



POLITECNICO DI BARI

DEPARTMENT OF ELECTRICAL AND INFORMATION ENGINEERING

MASTER'S DEGREE IN AUTOMATION ENGINEERING

MASTER THESIS IN
Dynamical Systems Theory

PREDICTIVE CONTROL OF A ROBOTIC MANIPULATOR IN AN ENVIRONMENT WITH DYNAMIC OBSTACLES

Supervisor:

Prof. Engr. Mariagrazia Dotoli

Co-Supervisors:

Prof. Engr. Raffaele Carli

Engr. Fabio Mastromarino

Candidate:

Francesco Savino

ACADEMIC YEAR 2024-2025



Politecnico
di Bari

LIBERATORIA ALLA CONSULTAZIONE DELLA TESI DI LAUREA DI CUI ALL'ART.4 DEL REGOLAMENTO DI ATENEIO PER LA CONSULTAZIONE DELLE TESI DI LAUREA (D.R. n. 479 del 14/11/2016).

Il sottoscritto **Francesco Savino** matricola **589466**

Corso di Laurea: **Ingegneria dell'Automazione**

Autore della presente tesi di Laurea dal titolo: **PREDICTIVE CONTROL OF A ROBOTIC MANIPULATOR IN AN ENVIRONMENT WITH DYNAMIC OBSTACLES**

Parola chiave: **MPC, Manipulator, Obstacles, ROS2, MoveIt2**

Abstract: This work falls within the field of robotics and industrial automation, with a focus on advanced manipulator control. The thesis addresses the problem of motion planning in dynamic environments, where moving obstacles may interfere with the robot's task execution. The objective is to develop and validate a Model Predictive Control (MPC) system capable of following a desired trajectory while actively avoiding dynamic obstacles. The proposed strategy integrates kinematic prediction models, a Generalized Proportional Integral Observer (GPIO) for estimating obstacle motion, and safety constraints based on Control Barrier Functions (CBFs). In particular, flexible safety criteria are designed through Control Barrier Function-based Safety Constraints (CBFSCs) with dynamically optimized decay rates, which are included as decision variables within the control problem. This formulation enables real-time adaptation of the safety margins, ensuring a balance between efficiency and robustness. The system is implemented and tested on the Niryo Ned2 robotic arm within the ROS2 and MoveIt2 frameworks. The results demonstrate accurate trajectory tracking and effective management of potential collisions in dynamic scenarios.

☒ Autorizzo

☐ Non autorizzo

la consultazione della presente tesi, fatto divieto a chiunque di riprodurre in tutto o in parte quanto in essa contenuto.

Bari, data 12/07/2025

Firma

A tutti coloro che hanno reso possibile questo traguardo.

Ai miei genitori e a mio fratello, per esserci sempre stati, in ogni passo del mio cammino.

Alle mie nonne, per l'affetto e la saggezza che sanno donare con semplicità.

Agli zii e alle zie, per la presenza affettuosa e discreta.

Agli amici di sempre, con cui condivido momenti preziosi e molto di più.

Agli amici universitari, compagni di viaggio in ogni fase di questo percorso.

Ai miei relatori, per la guida, la disponibilità e l'ispirazione.

A chi non c'è più, ma continua a vivere nei miei ricordi.

Abstract

This work falls within the field of robotics and industrial automation, with a focus on advanced manipulator control. The thesis addresses the problem of motion planning in dynamic environments, where moving obstacles may interfere with the robot's task execution. The objective is to develop and validate a Model Predictive Control (MPC) system capable of following a desired trajectory while actively avoiding dynamic obstacles. The proposed strategy integrates kinematic prediction models, a Generalized Proportional Integral Observer (GPIO) for estimating obstacle motion, and safety constraints based on Control Barrier Functions (CBFs). In particular, flexible safety criteria are designed through Control Barrier Function-based Safety Constraints (CBFSCs) with dynamically optimized decay rates, which are included as decision variables within the control problem. This formulation enables real-time adaptation of the safety margins, ensuring a balance between efficiency and robustness. The system is implemented and tested on the Niryo Ned2 robotic arm within the ROS2 and MoveIt2 frameworks. The results demonstrate accurate trajectory tracking and effective management of potential collisions in dynamic scenarios.

Contents

1	Introduction	1
1.1	Motivation	1
1.2	Definition and operating principle of MPC	3
1.3	Internship experience	5
1.4	Discrete-time CBFs	6
1.4.1	Definition of discrete-time CBFs	6
1.4.2	Motivation for the use of CBFs in obstacle avoidance	7
1.5	Dynamic obstacle state estimation	7
1.5.1	Constant acceleration prediction model	8
1.5.2	GPIO design and error dynamics	8
1.5.3	Stability analysis and error bound	9
1.6	Quaternion product (Hamilton product)	10
1.7	Tools	10
1.7.1	Robot operating system 2	10
1.7.2	MoveIt 2	12
1.7.3	RViz 2	12
1.7.4	URDF and SDF	12
1.8	The Niryo Ned2 robotic arm	13
1.9	Thesis objectives	15
1.10	Thesis structure	16
2	State of the art	17
2.1	Motion planning for obstacle avoidance	17
2.2	MPC for dynamic environments	18
2.3	State estimation for dynamic obstacles	19
2.4	CBFs for safety-critical motion	19
2.5	Analysis of an advanced integrated approach	20
2.6	Research gaps and thesis contribution	21
3	Problem statement	22
3.1	Implementation of a dynamic obstacle	22

3.1.1	Implementation in ROS 2	23
3.2	MPC formulation	24
3.2.1	The prediction model	24
3.2.2	The cost function	27
3.2.3	The constraints	27
3.2.4	The MPC optimization problem	28
3.3	MPC for trajectory tracking in an obstacle-free environment	29
3.3.1	Decision variables and prediction model	29
3.3.2	Reference trajectory generation	30
3.3.3	Position and orientation error computation	32
3.3.4	Joint constraints	34
3.3.5	Optimization problem formulation	34
3.3.6	Execution logic of the MPC controller	35
3.4	MPC for dynamic obstacle avoidance	38
3.4.1	GPIO design and implementation	38
3.4.2	Constraint-based safety control	40
3.4.3	Overview of the MPC software implementation	44
4	System architecture and implementation	46
4.1	System overview	46
4.2	Launch file and ROS2 nodes	48
4.3	Implementation considerations	49
4.3.1	Development environment and platform constraints	49
4.3.2	Real-time performance	49
4.3.3	Modular design	50
5	Numerical simulations	51
5.1	Performance metrics	52
5.2	Trajectory tracking in obstacle-free environments	53
5.2.1	Parameter tuning methodology	53
5.2.2	Results with the selected configuration	55
5.3	Obstacle avoidance in dynamic environments	58
5.3.1	Simulation setup	58
5.3.2	Obstacle state estimation	59
5.3.3	MPC with dynamic obstacle avoidance	61
6	Conclusions and future works	65
6.1	Contribution	65
6.2	Challenges and limitations	65
6.3	Future research directions	66
	References	67
	Source code Appendix	71

List of Figures

1.1	A basic working principle of Model Predictive Control [6].	3
1.2	General architecture of an MPC control system, illustrating the interaction between the controller, the state estimator, and the process [7].	4
1.3	RViz2 visualization of the static obstacle avoidance test using CHOMP.	5
1.4	Comparison between traditional MPC safety criteria and CBF-based criteria.	7
1.5	Overview and joint configuration of the Niryo Ned2 manipulator.	14
1.6	Link structure of the Niryo Ned2 as visualized in RViz2.	15
3.1	Visualization of the end-effector trajectory in RViz.	31
3.2	Execution timing of the MPC logic over consecutive control cycles.	36
3.3	Timing diagram illustrating the relationship between simulation timesteps and MPC control cycles.	37
3.4	Graphical illustration of the safety distance between the end-effector and the obstacle. . .	42
4.1	Block diagram of the system architecture showing data flow between the modules. . . .	46
5.1	Tracking of the end-effector position with respect to the reference trajectory.	54
5.2	End-effector orientation tracking: quaternion components.	55
5.3	End-effector position error over time for each Cartesian component.	56
5.4	Algebraic tracking error of the quaternion components.	56
5.5	3D visualization of the robot trajectory with orientation error vectors overlaid.	57
5.6	Control inputs: joint velocity commands generated by the MPC over time.	57
5.7	Initial environment setup in RViz: robot manipulator, dynamic obstacle, and ground plane.	58
5.8	Measured obstacle position (blue) vs estimated position (red dashed) along the x -, y -, and z -axes.	60
5.9	Estimation error on the y -component of the obstacle's position.	60
5.10	Estimated velocity of the obstacle along the y -axis.	61
5.11	Estimated acceleration of the obstacle along the y -axis.	61
5.12	Obstacle avoidance behaviour with high penalty $P_\gamma = 1000$	62
5.13	Obstacle avoidance behaviour with low penalty $P_\gamma = 25$ and more aggressive γ_k dynamics.	63
5.14	3D view of the executed trajectory under $P_\gamma = 25$	64

List of Tables

1.1	Niryo Ned2 technical specifications	14
5.1	Fixed parameters used in all MPC simulations	51

Introduction

1.1 Motivation

Robotics is one of the most dynamic and rapidly evolving engineering disciplines, with a constantly expanding range of applications, from advanced manufacturing to healthcare and domestic assistance.

The growing use of robotic manipulators outside traditional industrial contexts, such as collaborative logistics, personal assistance and robotic surgery, makes it increasingly urgent to develop control techniques capable of operating in dynamic and unstructured environments. In these scenarios, robots must be able to plan and adapt their behaviour in real time, ensuring not only the achievement of the objective but also the safety of interactions. This requires the adoption of advanced control strategies that account for the presence of mobile obstacles and the uncertainties of the operating environment. The ability to effectively manage such dynamic scenarios is crucial not only for safety but also to unlock the full potential of manipulators in terms of flexibility and industrial productivity.

Motion planning and control in the presence of obstacles have long been the subject of study in robotics. Traditional techniques are typically divided into two categories: offline approaches, such as sampling-based (RRT, PRM), grid-based (A*, D*) methods, and optimisation-based methods, and online approaches, including artificial potential fields (APF), fuzzy methods, and reinforcement learning-based methods. However, many of these approaches have significant limitations in dynamic environments: offline methods do not guarantee responsiveness, while online methods often struggle to comply with the physical constraints of the system or do not consistently integrate the prediction of obstacle behaviour.

MPC is an advanced solution for controlling complex systems in dynamic contexts, distinguished by its unified management of planning, control and constraint adherence. The operating principle of MPC is to solve, at each control step, a constrained optimisation problem defined over a future time horizon. Using an internal model of the system, MPC predicts its behaviour and calculates a sequence of control commands deemed optimal concerning the specified objectives and imposed constraints. This feature makes it particularly suitable for avoiding dynamic obstacles, especially when enhanced with modules for updated environmental perception and the implementation of active safety criteria.

Numerous recent studies have proposed extending the MPC framework with modules dedicated to safety management. An interesting first approach is illustrated in [1], which presents a two-level MPC integrated with APF. In this architecture, the upper level generates safe predictive trajectories in the

presence of dynamic obstacles, while the lower level acts as a trajectory tracker with rigid constraints to ensure collision avoidance. The system also employs a Super-Twisting Observer (STO) to estimate the speed of obstacles in real time, improving the quality of predictions.

A second advanced approach is described in [2], which proposes an MPC controller integrated with time-discrete CBFs with dynamically variable decay rates. This formulation allows the safety margin to be adapted in real time according to the current situation, while ensuring stability and responsiveness. The system also adopts a Generalised Proportional Integral Observer (GPIO) to estimate the position, velocity and acceleration of obstacles. In both of the aforementioned works, a set of critical points distributed along the robot’s kinematic structure is used to monitor the risk of collision across the entire body of the manipulator.

In several studies, including [1] and [2], the orientation error is often modelled as a simple difference between quaternions, neglecting the intrinsic geometric properties of this representation. This approximation has been analysed and criticised in [3], which proposes a more rigorous formulation of quaternion differential kinematics for MPC applications, based on the quaternion product and symbolic normalisation to prevent numerical drift and ensure the correct evolution of orientation.

Furthermore, the validation scenarios in these reference works often rely on simplified tasks, such as tracking a reference trajectory with a constant orientation for the end-effector. This simplification sidesteps the challenge of tracking fully spatial trajectories where the orientation must also evolve dynamically, a common requirement in many real-world manipulation tasks.

The combination of these observations (the predictive capabilities of MPC, the formal safety guarantees provided by CBFs and the role of advanced observers, along with the notable lack of publicly available, real-world implementations for such advanced control schemes) motivated the development of a new predictive control architecture. This thesis builds upon these premises and aims to implement an MPC controller for the Niryo Ned2 manipulator, capable of planning and executing safe, smooth, and dynamically adaptable trajectories in real time, while simultaneously ensuring accurate trajectory tracking and obstacle avoidance.

The activities carried out during the curricular internship further consolidated the research direction and provided a concrete application context. This experience focused on integrating the Niryo Ned2 manipulator into the ROS 2 ecosystem and configuring the system using MoveIt 2. A motion planning system based on the CHOMP planner was implemented, allowing the manipulator to generate collision-free trajectories in the presence of static obstacles. However, this approach revealed several limitations, most notably the inability to handle dynamic obstacles, which is crucial in realistic and collaborative scenarios. Additionally, CHOMP operates in joint space, which complicates inverse kinematics management when control in the operational space is required under dynamic conditions.

These limitations highlighted the need for a more adaptive and predictive control framework. This motivated the adoption of MPC as the methodological core of this thesis, in line with overcoming the identified shortcomings.

So this thesis falls within the field of predictive control for robotic manipulators, with a particular focus on the integration of obstacle avoidance strategies in dynamic environments. To meet the prediction and optimisation requirements typical of such contexts, an approach based on MPC has been adopted, an advanced technique that allows control actions to be optimised over a finite time horizon, explicitly considering the constraints of the system and the objectives to be achieved.

The simulations performed showed stable and precise behaviour of the system, with good tracking accuracy and a robust ability to avoid collisions, while maintaining an update frequency compatible with computational constraints. Numerical analysis of tracking errors confirms the validity of the approach, suggesting the possibility of extending it to scenarios with multiple obstacles or more complex dynamics. Further future development may consist of experimental validation on a real robotic platform.

1.2 Definition and operating principle of MPC

Among modern control strategies, MPC stands out for its ability to manage constrained multivariable systems, especially in Multiple-Input Multiple-Output (MIMO) type. MPC is an advanced control strategy that computes control actions online by solving a constrained optimization problem at each time step [4] [5]. Unlike classical control methods, which calculate control laws based solely on the current state, MPC leverages a mathematical model of the system to predict its future behaviour over a finite horizon. This predictive capability gives the controller an intrinsically proactive nature, allows it to anticipate the system's evolution and optimize decisions.

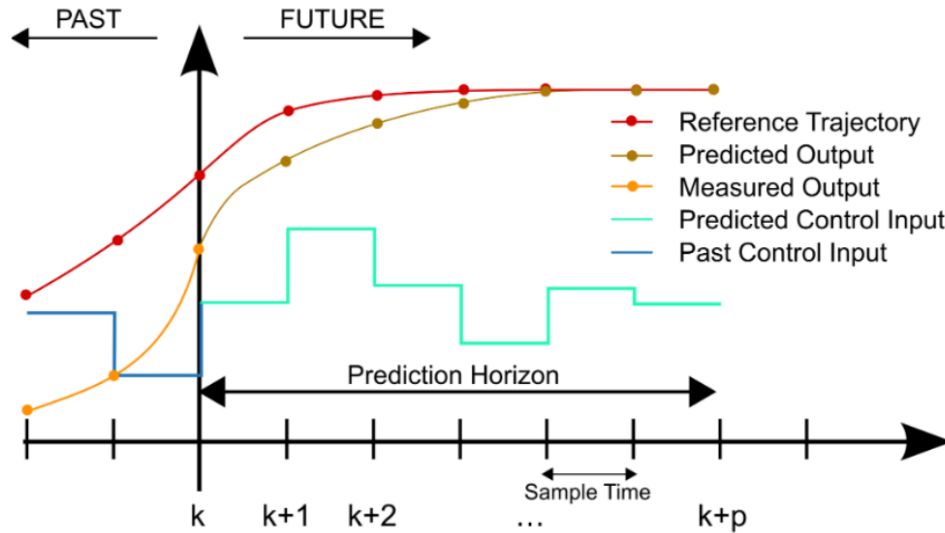


Figure 1.1: A basic working principle of Model Predictive Control [6].

The operating principle of MPC is an iterative process executed at each sampling instant. The control loop consists of three fundamental phases:

1. **Prediction:** Starting from the current system state, which is either measured or estimated, the controller uses an internal mathematical model to simulate and predict the future evolution of the states over a predefined time horizon, known as the prediction horizon (N_p). The length of this horizon, expressed in time steps, directly affects the controller's ability to anticipate constraints and disturbances.. This prediction is calculated as a function of a possible future sequence of control inputs.
2. **Optimization:** A constrained optimization problem is formulated and solved to determine a sequence of future control actions, defined over a control horizon (N_c , where $N_c \leq N_p$). The optimal sequence is the one that minimizes a predefined cost function (or objective function),

which typically penalizes the deviation of the system from a reference trajectory and the control effort. This function is typically quadratic in form for linear systems (yielding a QP problem), but can be non-linear in NMPC formulations. The optimization is performed subject to system constraints, such as actuator limits (e.g., maximum torque), state bounds (e.g., joint velocities), and environmental restrictions (e.g., obstacle avoidance).

3. **Application:** Of the computed optimal control sequence, only the first action is actually sent to the system's actuators.

At the next control step, the entire cycle is repeated. The system acquires the new state, and the prediction horizon “slides” forward in time. This strategy, known as the receding horizon principle (Fig. 1.1). By continuously replanning based on updated measurements, the controller can react to unforeseen disturbances and discrepancies between the model and the real system. This mechanism effectively transforms the open-loop optimization into a closed-loop control law, providing robustness to the entire control scheme.

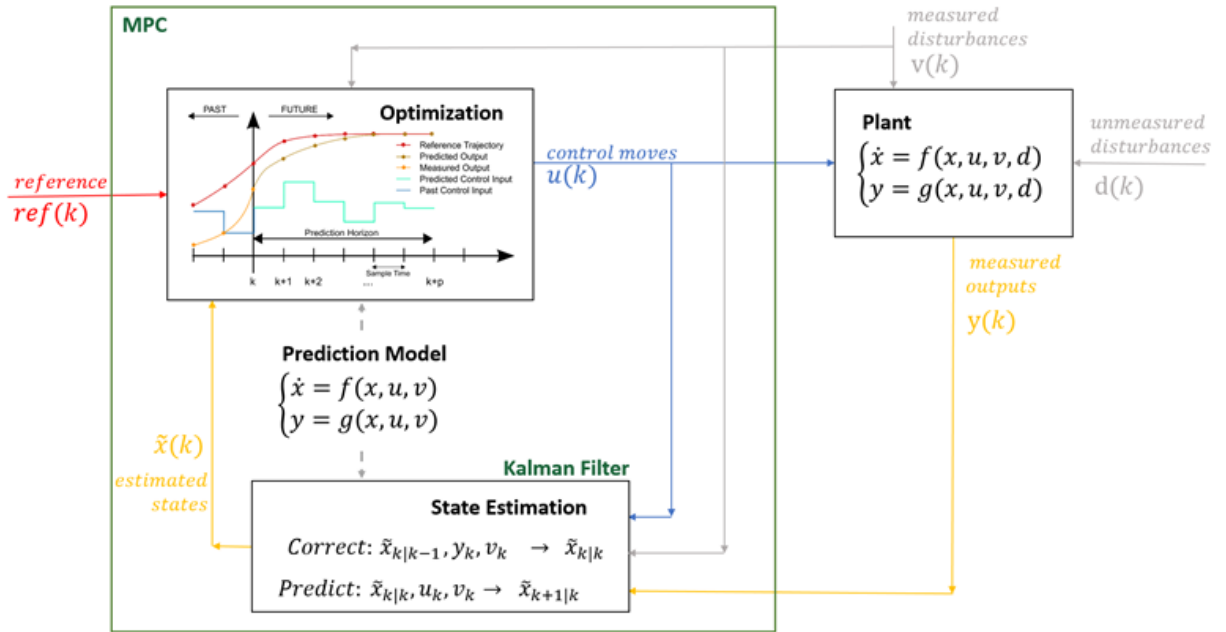


Figure 1.2: General architecture of an MPC control system, illustrating the interaction between the controller, the state estimator, and the process [7].

The combination of model-based prediction, constrained optimization, and the receding horizon strategy makes MPC a powerful and flexible framework. It is therefore particularly well-suited for managing complex, multivariable systems subject to stringent constraints, such as those that characterize modern robotics. In a typical implementation, the controller receives measured outputs from the process and takes into account external disturbances. It then uses its internal model for two fundamental tasks: first, to estimate the current state of the system (often via a state estimator like a Kalman Filter), which is crucial when dealing with noisy or unmeasurable signals; and second, to predict the future evolution of these states within the optimization problem. The overall control loop architecture (Fig. 1.2) manages both the feedback interaction and the internal predictive process. The effectiveness of MPC is strongly influenced by the design and tuning of its core components: the prediction model, the cost function, and

the system constraints. A comprehensive description of these elements, along with their application in this work, is provided in Chapter 3.

1.3 Internship experience

The work presented in this thesis builds upon the activities carried out during the curricular internship conducted at the *Decision and Control Laboratory (D&C Lab)*. The main objective of the internship was to create a modern software environment for the Niryo Ned 2 robotic manipulator by migrating its software stack from ROS 1 to ROS 2 and integrating it with the MoveIt 2 motion planning framework.

During this period, a complete trajectory planning pipeline was developed using the CHOMP optimizer [8], selected from among the planners available in MoveIt 2 for its ability to generate smooth, collision-free trajectories. The process involved configuring the robot model using the MoveIt 2 Setup Assistant, creating custom launch files for system initialization, and implementing a C++ node to manage motion planning tasks.

The system was successfully tested in simulation, with the robot's state and planned trajectories visualized in RViz 2. To validate the pipeline, a two-stage scenario was implemented: in the first stage, the robot planned a trajectory toward a target pose in an obstacle-free environment (the red path in Fig. 1.3); in the second stage, for the return path, a static obstacle was placed along the original trajectory (the green path in Fig. 1.3).

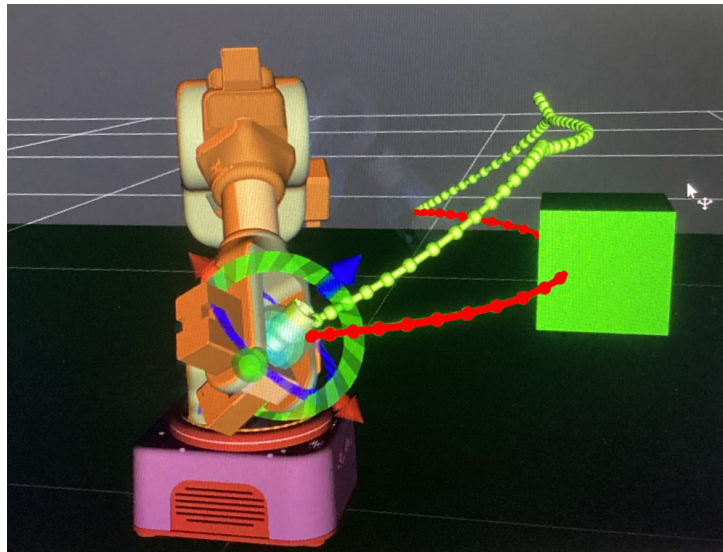


Figure 1.3: RViz2 visualization of the static obstacle avoidance test using CHOMP.

While the system successfully achieved its initial goals, the experiment also revealed two fundamental limitations:

- **Inability to handle dynamic obstacles:** The implemented pipeline operated in offline mode, generating a complete trajectory based on a static snapshot of the environment. This approach is ineffective in dynamic or unpredictable scenarios, where real-time replanning or reactive behaviour is required.
- **Limited control in operational space:** The CHOMP planner operates primarily in joint space, limiting direct control over the end-effector in Cartesian space. Although inverse kinematics allows

specification of the start and goal poses, it is difficult to enforce how the end-effector should behave along the path, such as maintaining a desired orientation or precisely following a Cartesian reference.

These insights highlighted the need for a more advanced control strategy capable of online adaptation, feedback integration, and constraint management. This motivated the development of the custom MPC that is the subject of this thesis, specifically designed to overcome the limitations of traditional motion planning approaches in ROS 2.

1.4 Discrete-time CBFs

In MPC frameworks, safety constraints (such as those required for obstacle avoidance) are often implemented by defining geometric boundaries that the robot must not cross. However, simply imposing these as hard constraints in the optimization problem can lead to issues, such as numerical instability or infeasibility when the robot gets too close to an obstacle. An increasingly popular and more elegant method for ensuring safety is the use of CBFs [2]. CBFs are a tool from control theory used to guarantee that a system remains within a predefined "safe set" for all time.

1.4.1 Definition of discrete-time CBFs

Let us consider a discrete-time nonlinear system of the form $\mathbf{x}_{k+1} = f(\mathbf{x}_k, \mathbf{u}_k)$, where $\mathbf{x}_k \in \mathcal{X}$ is the state and $\mathbf{u}_k \in \mathcal{U}$ is the control input. To formalize the concept of safety, we define a safe set as the superlevel set of a continuously differentiable scalar function $h(\mathbf{x})$:

$$\mathcal{C} = \{\mathbf{x} \in \mathcal{X} \mid h(\mathbf{x}) \geq 0\} \quad (1.1)$$

The boundary of this set is given by $\partial\mathcal{C} = \{\mathbf{x} \in \mathcal{X} \mid h(\mathbf{x}) = 0\}$. The set $\mathcal{C}$ is considered forward invariant if, for any initial state $\mathbf{x}_0 \in \mathcal{C}$, all future states $\mathbf{x}_k$ remain in the safe set $\mathcal{C}$ defined in Equation (1.1) for all $k > 0$.

To enforce this forward invariance, the function $h(\mathbf{x})$ must satisfy a specific condition, leading to the definition of a Discrete-Time Exponential CBF, as formally presented in [9]:

Definition (Discrete-time exponential CBF): A function $h(\mathbf{x})$ is a discrete-time exponential CBF for the system if for any state $\mathbf{x}_k \in \mathcal{C}$, there exists a control input $\mathbf{u}_k \in \mathcal{U}$ such that the following inequality holds:

$$h(\mathbf{x}_{k+1}) - h(\mathbf{x}_k) + \gamma h(\mathbf{x}_k) \geq 0 \quad (1.2)$$

where $0 < \gamma \leq 1$ is a user-defined decay rate. Condition (1.2) can also be rewritten in a more intuitive form, highlighting the recursive nature of the safety margin:

$$h(\mathbf{x}_{k+1}) \geq (1 - \gamma)h(\mathbf{x}_k) \quad (1.3)$$

This inequality is the core of the CBF. It does not simply require the next state to be safe (i.e., $h(\mathbf{x}_{k+1}) \geq 0$), but imposes a more structured condition: the "safety margin" at the next step, $h(\mathbf{x}_{k+1})$, must be at least a fraction $(1 - \gamma)$ of the safety margin at the current step, $h(\mathbf{x}_k)$. Importantly, this recursive inequality implies that if the system is initially in the safe set (i.e., $h(\mathbf{x}_k) \geq 0$), and the condition (1.3) is satisfied at each step, then the state will remain in the safe set for all future steps, i.e., $\mathbf{x}_{k+1} \in \mathcal{C}$ whenever $\mathbf{x}_k \in \mathcal{C}$, thus ensuring forward invariance.

1.4.2 Motivation for the use of CBFs in obstacle avoidance

The primary advantage of using a CBF constraint over a simple geometric constraint becomes evident in an obstacle avoidance scenario [2]. The practical difference between standard MPC constraints and CBF-based formulations in dynamic environments is illustrated through a representative example (Fig. 1.4).

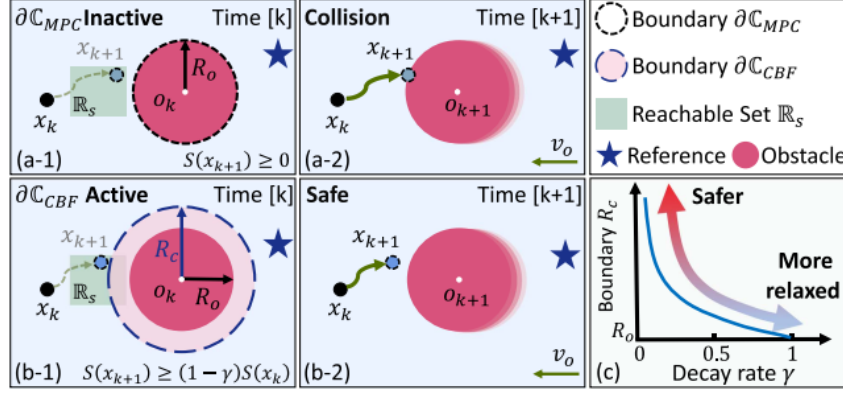


Figure 1.4: Comparison between traditional MPC safety criteria and CBF-based criteria.

In this context, the safety function can be defined as the distance to the obstacle, $h(\mathbf{x}_k) = \|\mathbf{x}_k - \mathbf{o}_k\| - R_o$, where $\mathbf{o}_k$ is the obstacle's centre and R_o its radius.

- **Traditional MPC safety criterion:** A standard MPC would impose the constraint that the predicted state must be outside the obstacle: $h(\mathbf{x}_{k+1}) \geq 0$. In this setting, the controller takes corrective action only when the robot's predicted reachable set $\mathbb{R}_s$ is about to touch the obstacle boundary $\partial\mathcal{C}_{MPC}$ (Fig. 1.4a). This can lead to aggressive, last-minute manoeuvres.
- **CBF-based Safety Criterion (CBFSC):** By using the CBF inequality (1.3), a much stronger condition is enforced: $h(\mathbf{x}_{k+1}) \geq (1 - \gamma)h(\mathbf{x}_k)$. This results in an expanded, state-dependent safety boundary $\partial\mathcal{C}_{CBF}$ (Fig. 1.4b). When the robot is far from the obstacle, $h(\mathbf{x}_k)$ is large, and the constraint forces the robot to maintain a significant safety margin, initiating avoidance manoeuvres much earlier and more smoothly.

The decay rate γ acts as a crucial tuning parameter. A small γ forces the safety margin to decrease very slowly, resulting in a highly conservative and "stricter" avoidance behaviour, while a larger γ (closer to 1) permits the robot to get closer to the obstacle (Fig. 1.4c).

The integration of these CBF-based constraints into the MPC optimization problem is a central aspect of this thesis, as it provides a formally robust and flexible method for guaranteeing safety in dynamic environments. The theoretical background of discrete-time CBFs presented here provides the foundation for the safety guarantees implemented in the MPC framework proposed in this thesis. Their practical integration into the optimization loop will be discussed in detail in Chapter 3.4.2.

1.5 Dynamic obstacle state estimation

In a dynamic environment, predicting the motion of moving obstacles is essential for safe robot navigation. While the current position of an obstacle may be measured directly using sensors (e.g., vision or LiDAR),

its velocity and acceleration are typically not observable. This lack of information limits the controller's ability to anticipate future positions. To address this, a dedicated state observer is introduced to reconstruct the full dynamic state of the obstacle from position measurements only.

A suitable method for this task is the GPIO, which estimates the derivatives of a signal based on its measured values and provides asymptotically convergent estimates under mild assumptions [10].

1.5.1 Constant acceleration prediction model

Let us define the obstacle's dynamic state at time step k , following the approach in [2], as composed of its 3D position $\mathbf{o}_k$, velocity $\mathbf{v}_k$, and acceleration $\mathbf{a}_k$, given respectively by:

$$\begin{cases} \mathbf{o}_k = [o_{x,k}, o_{y,k}, o_{z,k}]^\top \\ \mathbf{v}_k = [v_{x,k}, v_{y,k}, v_{z,k}]^\top \\ \mathbf{a}_k = [a_{x,k}, a_{y,k}, a_{z,k}]^\top \end{cases} \quad (1.4)$$

These quantities are grouped into an augmented state vector $\mathbf{D}_k = [\mathbf{o}_k^\top, \mathbf{v}_k^\top, \mathbf{a}_k^\top]^\top$.

The obstacle is modelled as a third-order integrator, assuming constant acceleration over the prediction horizon. The discrete-time prediction model is:

$$\begin{cases} \mathbf{o}_{k+1} = \mathbf{o}_k + T_s \mathbf{v}_k \\ \mathbf{v}_{k+1} = \mathbf{v}_k + T_s \mathbf{a}_k \\ \mathbf{a}_{k+1} = \mathbf{a}_k \end{cases} \quad (1.5)$$

Remark: In a more accurate model of motion, the position update equation would also include the second-order term $\frac{1}{2}T_s^2 \mathbf{a}_k$. In this implementation, however, it is omitted to simplify the structure of the observer and maintain compatibility with the linear state-space formulation.

This system, defined in equations (1.5), can be written compactly for each axis (x, y, z) in state-space form as:

$$\mathbf{D}_{k+1} = \mathbf{A}\mathbf{D}_k, \quad \mathbf{o}_k = \mathbf{C}\mathbf{D}_k \quad (1.6)$$

with:

$$\mathbf{A} = \begin{bmatrix} 1 & T_s & 0 \\ 0 & 1 & T_s \\ 0 & 0 & 1 \end{bmatrix}, \quad \mathbf{C} = \begin{bmatrix} 1 & 0 & 0 \end{bmatrix} \quad (1.7)$$

This formulation applies independently to each Cartesian axis and forms the basis for designing the GPIO.

1.5.2 GPIO design and error dynamics

Recall that the obstacle dynamics can be compactly represented in state-space form as in equations (1.6) and (1.7). Since the true velocity $\mathbf{v}_k$ and acceleration $\mathbf{a}_k$ are not directly measurable, the GPIO is employed to estimate these quantities from the measured position $\mathbf{o}_k$, yielding the reconstructed state $\hat{\mathbf{D}}_k$. The estimation dynamics of the GPIO are described by the following discrete-time equations [10]:

$$\begin{cases} \hat{\mathbf{o}}_{k+1} = \hat{\mathbf{o}}_k + T_s (\hat{\mathbf{v}}_k + \alpha_1(\mathbf{o}_k - \hat{\mathbf{o}}_k)) \\ \hat{\mathbf{v}}_{k+1} = \hat{\mathbf{v}}_k + T_s (\hat{\mathbf{a}}_k + \alpha_2(\mathbf{o}_k - \hat{\mathbf{o}}_k)) \\ \hat{\mathbf{a}}_{k+1} = \hat{\mathbf{a}}_k + T_s \alpha_3(\mathbf{o}_k - \hat{\mathbf{o}}_k) \end{cases} \quad (1.8)$$

where α_1 , α_2 , and α_3 are positive observer gains, and the estimates are denoted by hats.

Let the estimation errors be defined as:

$$\begin{cases} \varepsilon_{1,k} = o_k - \hat{o}_k \\ \varepsilon_{2,k} = v_k - \hat{v}_k \\ \varepsilon_{3,k} = a_k - \hat{a}_k \end{cases} \quad (1.9)$$

These are collected in the error vector $\mathbf{E}_k = [\varepsilon_{1,k}^\top, \varepsilon_{2,k}^\top, \varepsilon_{3,k}^\top]^\top$. The error evolves according to the linear system:

$$\mathbf{E}_{k+1} = \Phi \mathbf{E}_k \quad (1.10)$$

with error dynamics matrix:

$$\Phi = \begin{bmatrix} 1 - T_s \alpha_1 & T_s & 0 \\ -T_s \alpha_2 & 1 & T_s \\ -T_s \alpha_3 & 0 & 1 \end{bmatrix} \quad (1.11)$$

The observer gains must be selected such that the spectral radius $\rho(\Phi) < 1$, ensuring that the estimation error $\mathbf{E}_k$, composed of the components defined in equation (1.9), evolves according to the dynamics in equations (1.10), converges asymptotically to zero regardless of the initial condition. The following analysis formalizes this property by deriving an exponential bound on the error dynamics using a discrete-time Lyapunov approach.

1.5.3 Stability analysis and error bound

To analyse the stability of the observer, we examine the evolution of the estimation error over time. Let $\mathbf{E}_k$ be the estimation error vector defined previously. We define a Lyapunov function candidate as:

$$V(\mathbf{E}_k) = \|\mathbf{E}_k\|_W^2 = \mathbf{E}_k^\top \mathbf{W} \mathbf{E}_k \quad (1.12)$$

where $\mathbf{W}$ is a symmetric positive definite matrix. To ensure convergence of the estimation error, the matrix $\mathbf{W}$ must satisfy the following discrete-time Lyapunov inequality [2]:

$$\Phi^\top \mathbf{W} \Phi - \eta^2 \mathbf{W} \leq 0 \quad (1.13)$$

for some scalar η that satisfies $\rho(\Phi) < \eta < 1$, where η represents an upper bound on the rate of contraction.

Let $\lambda_{\min}(W) = c_1$ and $\lambda_{\max}(W) = c_2$ denote the minimum and maximum eigenvalues of W , respectively. The Lyapunov function is therefore bounded by the squared Euclidean norm of the error:

$$c_1 \|\mathbf{E}_k\|^2 \leq V(\mathbf{E}_k) \leq c_2 \|\mathbf{E}_k\|^2$$

The evolution of the Lyapunov function (1.12) over time can then be shown to be:

$$V(\mathbf{E}_{k+1}) = \mathbf{E}_{k+1}^\top \mathbf{W} \mathbf{E}_{k+1} = \mathbf{E}_k^\top \Phi^\top \mathbf{W} \Phi \mathbf{E}_k \leq \eta^2 \mathbf{E}_k^\top \mathbf{W} \mathbf{E}_k = \eta^2 V(\mathbf{E}_k)$$

which implies exponential convergence:

$$V(\mathbf{E}_k) \leq \eta^{2k} V(\mathbf{E}_0)$$

By combining these inequalities, we obtain a bound on the estimation error:

$$\|\mathbf{E}_k\|^2 \leq \frac{V(\mathbf{E}_k)}{c_1} \leq \frac{c_2}{c_1} \eta^{2k} \|\mathbf{E}_0\|^2$$

This can be expressed in compact form as:

$$\|\mathbf{E}_k\| \leq \phi_k \delta \quad \text{with} \quad \phi_k = \eta^k \sqrt{c_2/c_1} \quad (1.14)$$

where δ is an assumed upper bound on the initial estimation error $\|\mathbf{E}_0\|$.

This result confirms that, under a suitable tuning of the observer gains $\alpha_1, \alpha_2, \alpha_3$ (ensuring $\rho(\Phi) < 1$), the estimation error decays exponentially and the observer is asymptotically stable. The estimated state $\hat{\mathbf{D}}_k$ can then be used by the MPC to accurately predict the obstacle's future positions for robust and proactive avoidance manoeuvres.

1.6 Quaternion product (Hamilton product)

The integration of orientation in three-dimensional space, especially in robotic contexts, requires the use of the quaternion product (or Hamilton product). Given two quaternions, q_1 and q_2 , represented in the form $q = [x, y, z, w]^\top$ (where w is the scalar part and $[x, y, z]$ is the vector part), their product $q = q_1 \otimes q_2 = [x, y, z, w]^\top$ is defined as:

$$\begin{cases} x = w_1 x_2 + x_1 w_2 + y_1 z_2 - z_1 y_2 \\ y = w_1 y_2 - x_1 z_2 + y_1 w_2 + z_1 x_2 \\ z = w_1 z_2 + x_1 y_2 - y_1 x_2 + z_1 w_2 \\ w = w_1 w_2 - x_1 x_2 - y_1 y_2 - z_1 z_2 \end{cases} \quad (1.15)$$

This operation is fundamental for composing rotations and consistently updating the orientation within the system's kinematics and dynamics.

1.7 Tools

The development of the proposed control system relied on widely adopted technologies in the field of open-source robotics. This section provides an overview of the main software tools and frameworks integrated into the architecture, with a focus on their specific roles.

ROS 2 serves as the middleware responsible for inter-node communication, hardware abstraction, and real-time coordination. MoveIt 2 handles motion planning and kinematic computations, while RViz 2 offers a powerful platform for visualizing the robot's state and environment during simulation. The robot model is encoded using URDF and SDF formats, which define the manipulator's structure and physical properties. Each of these tools is discussed in more detail in the following subsections.

1.7.1 Robot operating system 2

The software framework used in this thesis is based on the Robot Operating System 2 (ROS 2), an open-source middleware that has emerged as a de facto standard in both academic research and industrial applications for the development of complex robotic systems [11]. Despite its name, ROS 2 is not an operating system in the traditional sense of process scheduling and memory management. Rather, it serves as a flexible and modular infrastructure built on top of a conventional OS (such as Linux), offering a standardized set of tools, libraries, and communication interfaces for robot software development.

ROS 2 is a complete redesign of the original ROS 1 framework, created to overcome fundamental limitations of its predecessor, particularly in terms of real-time performance, security, scalability for multi-robot systems, and suitability for production environments. A key architectural change in ROS 2 is the

adoption of the Data Distribution Service (DDS) as the core communication layer. This standard enables a decentralized, peer-to-peer message passing architecture with configurable Quality of Service (QoS) settings, thereby eliminating the need for a centralized master node and improving overall robustness and flexibility [11].

Core concepts and architecture

The ROS 2 architecture is designed to promote modularity, interoperability, and abstraction through a well-defined set of layers and communication primitives. Its most important features include:

- **Distributed modularity:** Applications are composed of independent *nodes*, each responsible for a specific task (e.g., sensor acquisition, planning, actuation). This structure enhances reusability, facilitates parallel development, and improves fault isolation.
- **Flexible communication interfaces:** Nodes communicate through message-passing mechanisms such as:
 - Topics, for asynchronous publish/subscribe communication;
 - Services, for synchronous request/reply interactions;
 - Actions, for long-running tasks that can be pre-empted (e.g., goal-based motion).

These interfaces are built on top of DDS, which manages message discovery, transport, and delivery.

- **Hardware abstraction:** High-level algorithms can be implemented independently of the underlying hardware, making them portable across different robot platforms with minimal adaptation.
- **Rich ecosystem and tooling:** ROS 2 is supported by a comprehensive ecosystem that includes a wide range of software packages for common robotic tasks, as well as integrated tools for development, visualization, debugging, and data analysis. This tooling greatly facilitates the implementation, inspection, and testing of complex robotic applications.
- **Multi-language and open source:** ROS 2 supports multiple programming languages, primarily C++ and Python. It is released under the permissive Apache 2.0 license, enabling wide adoption in both academic and industrial settings.

These elements can be interpreted along three conceptual levels: a filesystem layer for modular software organization, a runtime computation graph for inter-process communication, and a community infrastructure that fosters collaboration and reuse.

Version selection: Humble Hawksbill

This thesis is developed using the ROS 2 Humble Hawksbill distribution, released in May 2022. Humble is a Long-Term Support (LTS) version, officially maintained until May 2027. It was chosen for its stability, compatibility with widely used packages like MoveIt 2, and strong community support.

At the time this project commenced, newer standard distributions were available. However, *Iron Irwini* (May 2023) was still considered to be maturing and had reported stability issues, making it a less reliable choice. The subsequent release, *Jazzy Jalisco* (May 2024), was too recent to be considered a stable candidate for development. For the purposes of a master’s thesis (where reproducibility, a

mature ecosystem, and long-term reliability are essential) the well-established LTS version, Humble, was therefore the most suitable and prudent choice. Thanks to its flexibility, modularity, and robust middleware foundation, ROS 2 has become a key enabling technology in a wide range of domains, including industrial automation, autonomous vehicles, and advanced academic research. [11].

1.7.2 MoveIt 2

MoveIt 2 is a widely used open-source motion planning framework for robotic manipulation, built on top of ROS 2 [12]. It offers essential functionalities such as motion planning, inverse and forward kinematics, trajectory interpolation, collision checking, and environment representation. Thanks to its modular and extensible architecture, it supports a broad range of robotic platforms and is employed in both research and industrial contexts.

Its seamless integration with ROS 2 makes it particularly well-suited for the development of complex robotic systems, enabling developers to interact with robot models, perform planning queries, and manage the environment within a unified framework.

In this thesis, MoveIt 2 was selected as the standard tool to interface with the robot's model and the planning environment. The specific role and implementation details of MoveIt 2 within the developed architecture are described in a subsequent chapter.

1.7.3 RViz 2

RViz 2 is the standard 3D visualization tool for the ROS 2 ecosystem [13]. It provides a powerful and flexible interface for developers and researchers to visualize a robot's state, sensor data, and planning information in real time. As a crucial tool for debugging and introspection, RViz 2 plays a key role during both the development and validation phases of robotic applications.

The tool supports the visualization of various types of information, including robot models (URDF), coordinate frames (via TF2), sensor streams (e.g., LiDAR, RGB-D cameras), planned trajectories, and collision environments represented in the planning scene. These capabilities make it particularly valuable in systems that involve perception, motion planning, and interaction with dynamic environments.

In this thesis, RViz 2 was extensively used to visualize the robot's configuration, monitor the execution of planned trajectories, verify the representation of obstacles, and observe the predicted motion of both the robot and dynamic objects in the environment. Its integration was essential for verifying the correctness of the developed architecture and for supporting the analysis of the experimental results.

1.7.4 URDF and SDF

In robotic systems, a precise and standardized representation of the robot's structure is essential for simulation, control, and visualization. Two widely used formats in the ROS ecosystem for modelling robot descriptions are the *Unified Robot Description Format* (URDF) and the *Simulation Description Format* (SDF).

URDF

URDF is an XML-based format developed within the ROS ecosystem to describe the physical structure of a robot. It defines the robot as a set of interconnected rigid bodies (called *links*) connected via joints, and includes information such as dimensions, masses, inertias, and visual/collision meshes. URDF is

tightly integrated with tools like RViz and MoveIt2 and is the de facto standard for kinematic modelling in ROS-based applications.

Among its advantages:

- Readability and simplicity: the XML structure is straightforward and easy to edit.
- Compatibility: natively supported by ROS2, MoveIt2, and Gazebo (with SDF conversion).
- Modularity: it can be extended through Xacro macros to support parametric or reusable components.

However, URDF has some limitations:

- It does not support closed kinematic chains.
- It lacks full support for dynamics, sensors, and environment descriptions.
- It cannot define more advanced simulation parameters (e.g., friction coefficients, transmission delays).

SDF

SDF is a more expressive XML format originally developed for the Gazebo simulator. It expands on the capabilities of URDF by allowing the definition of complex environments, sensor models, physics properties, and plugins. Unlike URDF, SDF can describe entire simulation worlds, including multiple robots, lights, terrains, and dynamic elements.

Advantages of SDF include:

- Support for full physical simulation, including sensor noise and material properties.
- Ability to define and simulate entire scenes.
- Native integration with the Gazebo simulation engine.

While more powerful, SDF is less commonly used for motion planning or real-time control purposes, as many ROS2 tools (including MoveIt2) still rely primarily on URDF for robot modelling.

Comparison and use cases

In practice, URDF is preferred for control, planning, and visualization tasks within ROS2, while SDF is used for more accurate simulations in Gazebo. Conversion tools such as `gz_sdf` and `gz_urdf` allow interchanging formats when necessary.

1.8 The Niryo Ned2 robotic arm

The Niryo Ned2 is a six-axis collaborative robotic manipulator developed for educational, research, and prototyping purposes. It is manufactured by Niryo, a French robotics company based in Lille and founded in 2017. Over the years, Niryo has progressively improved its desktop robot lineup, evolving from the original *Niryo One* and the enhanced *Ned* to the *Ned2* (the third-generation model used in this thesis), and more recently, the fourth-generation *Ned3 Pro*. All models share a common design philosophy oriented towards education, research, and affordable access to industrial-grade robotic technologies[14].

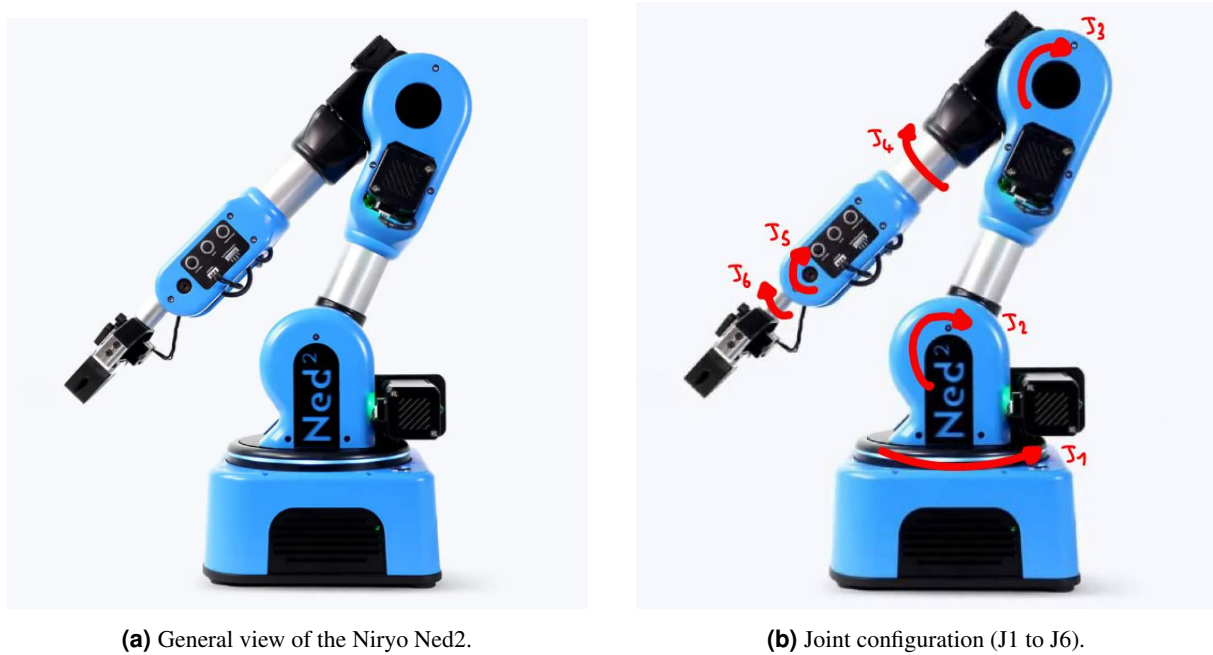


Figure 1.5: Overview and joint configuration of the Niryo Ned2 manipulator.

As a collaborative robot (cobot), the Ned2 is specifically designed to operate safely in proximity to humans. Collaborative robots offer several advantages over traditional industrial manipulators, including enhanced safety features, simplified programming interfaces, and greater flexibility for integration into human-centric environments. These characteristics make cobots particularly well suited for research, prototyping, and educational contexts where direct interaction with human operators is common.

Table 1.1: Niryo Ned2 technical specifications

Feature	Value
Degrees of Freedom	6 revolute joints
Payload	300 g
Repeatability	± 0.5 mm
Reach	490 mm
Weight	7 kg
Dimensions (W $\times$ D $\times$ H)	181.6 $\times$ 187.6 $\times$ 436.2 mm
Software Compatibility	ROS, Python, C++, Blockly, MATLAB
Processor	Raspberry Pi 4 (ARMv8, 64-bit, 4 GB RAM)

The Ned2 is a compact, desktop-scale robot arm featuring six revolute joints. Its structure is primarily made of aluminium, providing both mechanical robustness and smooth motion. Thanks to its open-source architecture and compatibility with the Robot Operating System (ROS), it offers a flexible environment for the development and testing of manipulation algorithms.

The robot is powered by a Raspberry Pi 4 embedded computer and runs Ubuntu 20.04 with ROS Noetic. It can be programmed using various tools and languages such as Python, C++, Blockly, and MATLAB. Its moderate payload and high repeatability make it suitable for experimentation in areas such as motion planning, visual servoing, and human-robot interaction. A summary of the main technical

specifications of the Niryo Ned2 is presented in Table 1.1.



Figure 1.6: Link structure of the Niryo Ned2 as visualized in RViz2.

The six revolute joints of the Niryo Ned2 define a serial structure commonly found in lightweight industrial arms, enabling a wide range of dexterous motions (Fig. 1.5).

The official software resources for the Niryo Ned2, developed for ROS 1, are publicly available through the company’s open-source GitHub repository[15]. These include the robot’s URDF model and control packages, which define its structure and capabilities.

Thanks to the visualization tools provided by RViz2 and MoveIt 2, the kinematic structure of the Niryo Ned2 can be clearly inspected. The distinction between virtual frames (e.g., `world` and `tool_link`) and physical links becomes evident through their graphical representation within the simulated environment. The main links, from base to end-effector, include `base_link`, `shoulder_link`, `arm_link`, `elbow_link`, `forearm_link`, `wrist_link`, and `hand_link` (Fig. 1.6).

1.9 Thesis objectives

The thesis aims to develop, implement and validate an MPC controller for the Niryo Ned2 robotic manipulator, designed to accurately track the desired trajectory while ensuring the avoidance of dynamic obstacles. The entire work was carried out in a simulated environment, using ROS 2 for robot architecture management, MoveIt 2 for robot model integration and access to kinematic functionality, RViz for visualisation, and MATLAB for numerical analysis of the results.

To achieve this goal, the activity was divided into three main phases. In the first phase, a software module was developed in C++ to simulate a moving obstacle, represented as a rectangular prism moving along a cyclic trajectory between two fixed points, generated by linear interpolation. In the second phase, a discrete and non-linear MPC controller was designed, based on a kinematic model of the manipulator, with the initial objective of accurately tracking the position and orientation of the end-effector in the absence of obstacles. The third phase introduced obstacle avoidance functionality into the MPC structure by including safety constraints formulated as CBFs, applied to the end-effector. To support the controller, an observer capable of estimating the state of the obstacle (position, velocity and acceleration) was also integrated, providing the predictive system with a proactive ability to anticipate potential collisions.

1.10 Thesis structure

This thesis is organised as follows, to guide the reader from the contextualization of the problem to the presentation and analysis of the obtained results.

- **Chapter 1 - Introduction:** This chapter provides a general introduction to the context of the thesis. It presents the fundamental technologies used, including the Robot Operating System 2 (ROS 2), the MoveIt 2 motion planning framework, CBFs, and the GPIO. The hardware platform, the Niryo Ned2 manipulator, and the internship experience that provided the initial motivation for this work are also described.
- **Chapter 2 - State of the art:** This chapter presents a critical analysis of the relevant scientific literature. The main existing approaches for motion control and obstacle avoidance for robotic manipulators are examined, with a particular focus on methodologies based on MPC and CBF. The objective is to contextualise the contribution of this thesis by highlighting the limitations of current approaches and the most promising research directions.
- **Chapter 3 - Problem statement:** This chapter constitutes the technical core of the thesis, where the problem formulation and the design of the proposed controller are described in detail. The discussion is divided into three main implementation phases: the creation of a dynamic obstacle for the simulation environment, the development of a basic MPC controller for trajectory tracking only, and finally, the integration of advanced obstacle avoidance functionalities, which include the obstacle state observer and the CBF-based safety constraints.
- **Chapter 4 - System architecture and implementation:** This chapter describes the concrete software architecture of the implemented system. The developed ROS 2 nodes, their functionalities, the variables they exchange, and the way they communicate with each other are illustrated, with a particular focus on the interaction between the MPC controller and the rest of the system.
- **Chapter 5 - Numerical simulations:** In this chapter, the results obtained from the simulation campaigns are presented and analysed. The performance of the proposed controller is validated in both obstacle-free scenarios, to test its tracking capabilities, and in scenarios with dynamic obstacles, to evaluate the effectiveness of the avoidance strategies. The parameters used are analysed, and the results are compared with theoretical expectations.
- **Chapter 6 - Conclusions and future works:** This final chapter summarises the main contributions of the thesis and discusses the obtained results. It also outlines possible extensions and future research directions stemming from this work.

State of the art

This chapter presents a critical review of the scientific literature relevant to real-time motion control and dynamic obstacle avoidance for robotic manipulators. It begins with an overview of classical motion planning techniques and then focuses on the core methodologies underpinning this thesis: MPC, Control Barrier Functions (CBFs), and dynamic state observers. The objective is to frame the context of the proposed work, identify current limitations, and motivate the need for new approaches.

MPC provides a unified framework for trajectory generation, control, and constraint satisfaction, while CBFs offer formal guarantees for maintaining safety. However, both rely on accurate knowledge of the system and its environment. In scenarios involving moving obstacles, this includes information about obstacle velocity and acceleration, which are often not directly measurable. For this reason, state observers are increasingly used to estimate the dynamic state of obstacles in real time and support proactive motion planning.

2.1 Motion planning for obstacle avoidance

A fundamental capability for robotic manipulators operating in cluttered or dynamic environments is obstacle avoidance. The techniques developed to address this challenge can be broadly classified into two categories: offline global path planning and online reactive planning [16].

Offline planners are well suited to static environments. These algorithms compute a full, collision-free path from an initial to a goal configuration prior to execution. Notable approaches include sampling-based methods such as Rapidly-exploring Random Trees [17], search-based methods like A* [18], and optimization-based planners such as CHOMP (Covariant Hamiltonian Optimization for Motion Planning) [8]. Although effective in structured and predictable scenarios, these techniques struggle in dynamic contexts where real-time adaptation is required. Moreover, the stochastic nature of sampling-based methods can result in unstable or oscillatory behaviours during frequent replanning.

For environments that change over time, reactive or online planning strategies are more appropriate. Among these, the Artificial Potential Field (APF) method is a classic approach that defines attractive forces toward the goal and repulsive forces around obstacles, guiding the robot as if moving in a virtual force field [19]. However, APF is prone to local minima, where forces cancel each other out and the robot becomes trapped.

Other reactive strategies have been proposed, including fuzzy logic controllers [20] and reinforcement learning techniques [21]. While these methods can offer adaptive behaviours, they often lack a unified

framework that integrates high-level planning, system constraints, and low-level control. Additionally, many do not provide formal safety guarantees or predictable performance when facing unforeseen obstacles.

These limitations have driven interest toward planning and control frameworks that can offer dynamic adaptability while explicitly handling constraints. This is an area where MPC has shown significant promise.

2.2 MPC for dynamic environments

MPC has been widely adopted in robotic systems due to its ability to handle multi-variable dynamics, enforce constraints, and incorporate prediction in a unified framework. Several formulations have been proposed depending on the task: for example, path-following MPC has been employed for continuous replanning in dynamic environments [22], MPC with integral compensation has been used to enhance disturbance rejection and stability in both joint and task spaces [23], and cooperative MPC strategies have been explored for manipulators sharing a common workspace [24]. These variants demonstrate the adaptability of the MPC paradigm to a range of objectives and system configurations.

Building on this flexibility, MPC has proven to be a particularly powerful framework for motion planning and control in dynamic environments. By solving a constrained optimization problem at each control step, it predicts the future behaviour of the system over a finite time horizon and computes an optimal sequence of control actions that respect both system dynamics and operational constraints [25]. This architecture unifies high-level planning, low-level control, and constraint enforcement in a single formulation, making it especially attractive for real-time applications such as obstacle avoidance and trajectory tracking in robotic manipulators [1, 26, 27, 28].

In the context of dynamic obstacle avoidance, MPC enables proactive motion adjustment based on anticipated future states. However, most existing MPC-based approaches define safety constraints using simple Euclidean distance thresholds. These constraints typically become active only when the robot is already close to an obstacle, limiting their effectiveness in scenarios involving fast-moving obstacles or abrupt environmental changes [2]. This reactive behaviour can lead to delayed avoidance manoeuvres and compromise safety.

Another challenge arises from the use of hard constraints. While these constraints provide strict guarantees, they can render the optimization problem infeasible in the presence of disturbances, model inaccuracies, or unexpected events. To preserve feasibility, many implementations soften these constraints through slack variables or penalty terms, at the cost of weakening safety guarantees.

Finally, the choice of prediction horizon plays a critical role: short horizons may result in short-sighted and discontinuous behaviour, while long horizons improve foresight but significantly increase computational burden. Balancing responsiveness and computational tractability remains an open problem in the real-time application of MPC to robotic systems operating in dynamic and uncertain environments.

From a practical standpoint, the implementation of MPC in robotic systems often relies on dedicated software tools. Among them, IMPACT [29] enables rapid deployment of MPC controllers through code generation and integration with ROS 2. However, its dependency on Conda environments and limited flexibility for defining nonlinear constraints may hinder integration with larger control architectures.

Other libraries such as ACADO and CasADi offer greater modelling freedom. In this thesis, CasADi [30, 31] was chosen for its symbolic formulation capabilities and seamless support for nonlinear

programming. CasADi provides automatic differentiation in both forward and reverse modes on sparse computational graphs, supports symbolic-to-numeric workflows, and allows integration with advanced solvers such as IPOPT. This design enables full control over the optimization problem structure and direct compatibility with the ROS 2 control stack.

In this project, the MPC controller uses CasADi’s symbolic API to model the system dynamics, cost function, and safety constraints. IPOPT is employed as the underlying nonlinear solver to handle constrained optimization at each control step. Both CasADi and IPOPT were built and installed from source to ensure proper linking with the C++ implementation and full compatibility with the system libraries.

2.3 State estimation for dynamic obstacles

In dynamic environments, effective obstacle avoidance relies not only on planning and control, but also on the ability to anticipate the future motion of surrounding objects. While obstacle positions can often be acquired from perception systems such as LiDAR or vision-based tracking, their velocities and accelerations are typically not directly observable and must be estimated.

Several observer designs have been proposed for this purpose, including classical Kalman Filters [32], Extended Kalman Filters [33], high-order differentiators [34], and Sliding Mode Observers such as the STO [35, 1]. Kalman-based approaches offer optimal estimation in the presence of Gaussian noise and linear (or linearized) dynamics, but they require careful tuning of noise covariances and may suffer from limited responsiveness when applied to nonlinear or highly dynamic systems. High-order differentiators can provide rapid convergence, but their sensitivity to measurement noise can degrade estimation quality in practice. Sliding mode observers ensure finite-time convergence and robustness to bounded disturbances, but can introduce chattering effects unless appropriately filtered.

In this thesis, the Generalized Proportional Integral Observer (GPIO) introduced in [2] is adopted to reconstruct the full dynamic state of moving obstacles, including velocity and acceleration, based solely on position measurements. The GPIO is a linear discrete-time observer designed to asymptotically track the derivatives of the input signal. Compared to nonlinear observers, it offers a simpler structure, guaranteed convergence under mild conditions, and easy integration with the constant-acceleration motion model used in the MPC formulation.

The use of a stable and responsive observer is essential to ensure that safety constraints remain meaningful throughout the MPC prediction horizon. By providing accurate motion estimates without introducing chattering or relying on probabilistic models, the GPIO improves the reliability and responsiveness of the overall control architecture in real-time applications.

2.4 CBFs for safety-critical motion

To address the limitations of simple distance-based constraints and to provide more formal safety guarantees, CBFs have gained significant attention in recent years [36]. Originating from control theory, a CBF defines a condition that, when satisfied, ensures the forward invariance of a predefined safe set. In robotics, this translates into the system’s ability to remain within a region of admissible states, thereby avoiding configurations that could lead to collisions or other unsafe situations.

The integration of CBFs with MPC has been explored in several studies as a means to embed real-time safety constraints within a predictive control framework [37, 38, 39]. This hybrid approach enables the controller to generate actions that are both optimal (in terms of task performance) and provably safe.

However, a critical review of these approaches reveals two common limitations. First, many methods employ CBFs with a fixed, manually-tuned decay rate [40, 41, 42]. This creates a trade-off: low decay rates result in conservative behaviour that may overly restrict motion, while high decay rates can weaken the safety guarantees, particularly in the presence of model uncertainties or sensor noise.

Second, a large portion of the literature focuses on static obstacle avoidance. In dynamic settings, maintaining forward invariance requires anticipating the future motion of moving obstacles, which necessitates velocity or acceleration estimation. As pointed out in [2], many existing approaches do not include an explicit observer or estimation strategy, reducing their practical effectiveness in real-world applications. An interesting partial exception is [42], which introduces adaptive safety margins in a teleoperation context, but does not explicitly address dynamic obstacles and lacks a dedicated estimation module.

2.5 Analysis of an advanced integrated approach

The most recent and promising research directions aim to unify the predictive capabilities of MPC with formal safety guarantees provided by CBFs, while also incorporating state estimation techniques for obstacle motion. This section analyses the work that forms the theoretical and methodological foundation of the control architecture developed in this thesis.

A primary source of inspiration is the work by Liu et al. [2], which introduces a *Flexible Active Safety Motion (FASM)* control framework. This method proposes a robust MPC scheme augmented with CBF-based safety constraints and internal state estimation to enable proactive, adaptive, and safe behaviour in the presence of dynamic obstacles.

The central innovation of the proposed architecture is the formulation of a flexible safety criterion, called a CBF-guided Safety Constraint (CBFSC), based on discrete-time exponential CBFs with dynamically optimised decay rates. Unlike classical CBFs with fixed tuning, the decay rate γ is treated as a decision variable within the MPC optimisation problem. This allows the controller to dynamically adjust its level of conservatism based on the environment, enabling a better trade-off between safety and performance. A dedicated penalty term is added to the cost function to regularise the evolution of the safety margin over time.

The architecture consists of three key components:

1. **Dynamic obstacle state estimation:** A GPIO reconstructs the full dynamic state of moving obstacles (position, velocity, and acceleration) based on position measurements alone. This allows the controller to anticipate obstacle trajectories even when velocity information is unavailable.
2. **CBF-based safety constraints:** The predicted trajectories are used to construct discrete-time exponential CBFs, which define forward-invariant safe sets and are directly embedded into the MPC problem.
3. **Adaptive decay rate optimization:** The CBF decay rate γ is introduced as a tunable variable within the optimization. The cost function penalises excessive tightening or loosening of the safety margin, promoting stability and flexibility.

A particularly relevant result is that the proposed method achieves successful dynamic obstacle avoidance even with a prediction horizon of $N = 1$, where classical MPC schemes typically fail to provide sufficient foresight. This leads to significantly reduced computational effort while preserving responsiveness and safety.

The FASM approach is validated both in simulation and on a physical robotic platform. Results confirm its effectiveness in avoiding moving obstacles, maintaining accurate trajectory tracking, and adapting to rapid environmental changes. Compared to traditional CBF-MPC schemes with fixed safety parameters, the proposed method exhibits enhanced flexibility, reduced conservatism, and improved real-time feasibility.

2.6 Research gaps and thesis contribution

From the analysis of the existing literature, it is clear that the combination of MPC, CBFs, and state estimation techniques constitutes a powerful framework for real-time obstacle avoidance. However, several research gaps remain open and motivate the contributions of this thesis.

First, while advanced MPC-CBF architectures such as the FASM controller proposed in [2] show strong theoretical potential, they often lack implementation transparency. In particular, the paper does not provide any source code or open-source implementation, which limits reproducibility and practical adoption. Moreover, the mathematical formulation omits critical details, especially regarding how the end-effector pose is predicted. Although the pose is represented as a 7-dimensional vector including orientation in quaternion form, the authors apply a linear discrete-time update using the geometric Jacobian, which yields a 6-dimensional velocity. This creates a dimensional inconsistency and suggests that the orientation may not be properly integrated using quaternion algebra. The omission of an explicit orientation update equation raises concerns about the geometric consistency of the predicted states.

Specifically, in [2], the orientation error is presented with the same notation used for the position error, without any distinction or justification. This suggests that the authors may compute the orientation error as a simple vector difference between quaternions. Such an approach disregards the geometric properties of unit quaternions and can lead to numerical inconsistencies. This issue has been highlighted and addressed in [3], which proposes a more rigorous formulation based on the quaternion product and symbolic normalisation to ensure proper evolution of orientation within an MPC framework.

Additionally, the reference trajectory used in [2] is held constant throughout the prediction horizon, whereas in this thesis a time-varying reference is employed. The reference poses, including both position and orientation, evolve dynamically, allowing the controller to track complex trajectories rather than converge to a static target.

Finally, despite the recent interest in combining MPC, CBFs, and observers, publicly available implementations compatible with modern robotic software stacks (such as ROS 2 and MoveIt 2) are still lacking. This further limits the integration of such advanced techniques into real robotic systems.

To address these limitations, this thesis proposes a fully integrated, ROS 2-compatible control architecture for dynamic obstacle avoidance, combining nonlinear MPC, discrete-time CBFs, and a GPIO for obstacle motion estimation. In contrast to previous works, the MPC formulation includes both time-varying position and orientation tracking, and the quaternion error is computed using a geometrically consistent method based on quaternion algebra. The developed framework is implemented and validated on the Niryo Ned2 robotic manipulator, demonstrating real-time feasibility and improved flexibility in dynamic environments.

Problem statement

This chapter constitutes the technical core of the thesis, detailing the problem formulation and the incremental design of the proposed controller. The discussion is structured into three main implementation phases: first, the creation of a dynamic obstacle to set up the simulation environment; second, the development of a baseline Model Predictive Controller focused solely on trajectory tracking; and finally, the integration of advanced obstacle avoidance functionalities, which include the obstacle state observer and the CBF based safety constraints with dynamically optimized decay rates.

3.1 Implementation of a dynamic obstacle

To evaluate the proposed predictive controller in dynamic scenarios, a moving obstacle was introduced into the planning scene using MoveIt 2. The obstacle is modelled as a `CollisionObject` of cubic shape and follows a predefined, time-dependent linear trajectory between two fixed waypoints in the workspace. Its position is updated at runtime using the `MOVE` operation of the `CollisionObject` message, which incrementally modifies the `PlanningScene` at each control cycle. This method is significantly more efficient and less error-prone than repeatedly removing and re-adding the object using `ADD/REMOVE` operations.

Two strategies were evaluated for generating obstacle motion:

1. A discrete approach based on a fixed number of steps (*num_steps*) and a constant update rate;
2. A continuous, time-based interpolation based on the relative time elapsed since the beginning of the motion.

The second approach was adopted, as it ensures smoother and resolution-independent motion. While in Section 1.5.1 the obstacle's dynamic state is defined at discrete time steps k (see equations (1.4)), here we describe the continuous-time interpolation parameter $t \in [0, 1]$, representing the relative elapsed time within a motion segment.

The obstacle's position $\mathbf{o}(t) = (o_x(t), o_y(t), o_z(t))$ at time t is computed through linear interpolation between two reference poses, $\mathbf{o}_{\text{start}}$ and $\mathbf{o}_{\text{end}}$:

$$\mathbf{o}(t) = \mathbf{o}_{\text{start}} + (\mathbf{o}_{\text{end}} - \mathbf{o}_{\text{start}}) \cdot t \quad (3.1)$$

where t is calculated as the relative elapsed time with respect to the total segment duration. The orientation remains constant throughout the motion and is set to the identity quaternion:

$$\text{orientation} = (x = 0, y = 0, z = 0, w = 1)$$

which corresponds to zero rotation in quaternion representation.

3.1.1 Implementation in ROS 2

A dedicated ROS 2 node, `dynamic_scene_setup`, was implemented in C++ to manage the lifecycle of the dynamic obstacle within the simulation. Its source code is provided in Appendix 1, enabling reproducible testing of the full control pipeline. This node is responsible for three main tasks:

1. **Scene initialization:** Upon startup, the node adds both a static floor (representing the supporting surface of the robot) and the dynamic obstacle to the planning scene. The floor is modelled as a rectangular parallelepiped with dimensions $3.0 \text{ m} \times 3.0 \text{ m} \times 0.1 \text{ m}$, positioned so that its upper surface aligns with the base of the manipulator. The obstacle is instantiated as a cube of 10 cm side length, initially placed at point A: $(0.31, 0.5, 0.25) \text{ m}$. Both elements are added using the `applyCollisionObjects()` method from the `PlanningSceneInterface`.
2. **Obstacle motion via pose updates:** The obstacle's pose is updated at fixed time intervals using linear interpolation (3.1). It moves back and forth along a 1 m linear segment between points A and B: $(0.31, -0.5, 0.25) \text{ m}$, resulting in a one-dimensional trajectory along the Y-axis. The motion speed v_y can be adjusted by modifying the `total_duration` parameter in the code, which defines the time required to traverse one segment. The desired value is computed using the relation:

$$\text{total_duration} = \frac{d}{v_y}, \quad \text{with } d = 1 \text{ m.} \quad (3.2)$$

For instance, to simulate $v_y = 2 \text{ m/s}$, one should set `total_duration` to 0.5 s.

3. **External pose publication:** In parallel with the motion updates, the node publishes the current pose of the obstacle on the `/obstacle_pose` topic using a `geometry_msgs::msg::PoseStamped` message. This provides a lightweight and decoupled interface for other components (particularly the observer and the MPC controller) to retrieve obstacle state information without parsing the full `PlanningScene`.

Time management

The initial implementation relied on the default system clock (`RCL_ROS_TIME`) for computing the elapsed time during obstacle motion. However, this approach led to inconsistencies caused by its non-monotonic behaviour (i.e., the clock may occasionally move backward) resulting in temporal discontinuities in the obstacle's trajectory. These anomalies were observable in RViz and corroborated by warning messages such as:

```
[robot_state_publisher]: Moved backwards in time, re-publishing
joint transforms!
```

To ensure consistent and forward-progressing timing, a monotonic clock (`RCL_STEADY_TIME`) was adopted. Since ROS 2 Humble does not support assigning a steady clock as the primary time source of a node, a hybrid strategy was employed: the node continues to operate with `RCL_ROS_TIME`, while a separate instance of `rclcpp::Clock` configured with `RCL_STEADY_TIME` is used internally to compute the relative elapsed time from motion start.

The periodic updates are triggered using `create_wall_timer()` with a fixed update period of 40 ms, corresponding to a frequency of 25 Hz. Within each timer callback, the relative time is retrieved from the monotonic clock, and the new interpolated obstacle pose is computed and published accordingly.

This configuration ensures smooth, deterministic, and temporally coherent obstacle motion, which are key requirements for reliably evaluating:

- The GPIO module’s ability to estimate the obstacle’s velocity and acceleration based on the published pose data;
- The MPC controller’s effectiveness in integrating these estimates into its constraint formulation for real-time, collision-free trajectory generation.

3.2 MPC formulation

This chapter provides a detailed overview of the specific MPC formulation adopted in this thesis. First, the fundamental components of a standard MPC controller are described, followed by the customizations required for the robotic application under consideration.

The formulation of an MPC controller is based on three key components that define its behaviour and performance: the prediction model, the cost function, and the constraints. The careful design and tuning of these three elements determine the controller’s responsiveness, robustness, and computational efficiency in real-time robotic scenarios [4].

3.2.1 The prediction model

The model is the core of the MPC controller, as the ability to accurately predict the future behaviour of the system depends on it. The quality and nature of the model directly influence both the controller’s performance and its computational complexity. In robotic manipulators, where non-linear dynamics and constraints play a central role, selecting an appropriate prediction model is essential to ensure stability and accurate reference tracking.

A common approach is to use a discrete-time state-space representation, which for a general non-linear system is described by the following state transition and output equations:

$$\begin{cases} \mathbf{x}_{k+1} = f(\mathbf{x}_k, \mathbf{u}_k) \\ \mathbf{y}_k = h(\mathbf{x}_k) \end{cases} \quad (3.3)$$

where $\mathbf{x}_k$ is the state vector at time step k , $\mathbf{x}_{k+1}$ is the state vector at the next time step $k + 1$, $\mathbf{u}_k$ is the control input vector, and $\mathbf{y}_k$ is the output vector. The functions f and h represent the non-linear state transition and output mappings, respectively.

The nature of the prediction model is defined by the mathematical properties and origin of the functions f and h that constitute it. Two main classifications are typically considered:

Linear and non-linear models: A model is linear if both the state transition function f and the output function h are linear. A common example is the linear time-invariant (LTI) model:

$$\begin{cases} \mathbf{x}_{k+1} = \mathbf{A}\mathbf{x}_k + \mathbf{B}\mathbf{u}_k \\ \mathbf{y}_k = \mathbf{C}\mathbf{x}_k + \mathbf{D}\mathbf{u}_k \end{cases} \quad (3.4)$$

In this formulation, the state vector $\mathbf{x}$ contains the minimal set of variables needed to fully describe the internal state of the system. The first equation of (3.4) models the evolution of the state, where $\mathbf{A}$ defines the internal dynamics and $\mathbf{B}$ the influence of the inputs. The second equation of (3.4) defines how the outputs $\mathbf{y}$ are obtained from the states and inputs via the output matrix $\mathbf{C}$ and the feedthrough matrix $\mathbf{D}$.

Linear models lead to convex optimization problems (typically quadratic programming, QP), which are computationally efficient to solve. However, they may not accurately capture the behaviour of complex systems such as robotic manipulators. In contrast, non-linear models (where f and/or h are non-linear functions as in (3.3)) better approximate real dynamics but lead to a non-linear Model Predictive Control (NMPC) formulation, which is non-convex and more computationally demanding.

Physics-based and empirical models: This classification concerns the origin of the model equations. Physics-based models are derived from first principles, such as rigid-body dynamics or kinematics. For robotic manipulators, this includes:

- The forward kinematics maps the joint positions $\boldsymbol{\theta}_k \in \mathbb{R}^6$ to the end-effector pose $\mathbf{x}_{e,k} \in \mathbb{R}^7$, consisting of position and orientation. Specifically, the first three components represent the Cartesian position $\mathbf{p}_k \in \mathbb{R}^3$, while the remaining four represent the orientation in quaternion form $\mathbf{q}_k \in \mathbb{R}^4$:

$$\mathbf{x}_{e,k} = k(\boldsymbol{\theta}_k) \quad (3.5)$$

- The differential kinematics expresses the Cartesian velocity of the end-effector $\dot{\mathbf{x}}_e \in \mathbb{R}^6$ as a linear function of the joint velocities $\mathbf{u}_k \in \mathbb{R}^6$, via the geometric Jacobian $\mathbf{J}_{ee}(\boldsymbol{\theta})$. The velocity vector $\dot{\mathbf{x}}_e$ includes both linear velocity $\mathbf{v}_k \in \mathbb{R}^3$ and angular velocity $\boldsymbol{\omega}_k \in \mathbb{R}^3$:

$$\dot{\mathbf{x}}_e = \mathbf{J}_{ee}(\boldsymbol{\theta}) \mathbf{u}_k \quad (3.6)$$

Although in this thesis the Jacobian is evaluated at the end-effector, the same formulation applies to any point rigidly attached to the manipulator. The specific structure of the Jacobian depends on the choice of the reference point and frame, but the relation between joint velocities and Cartesian velocity remains linear and configuration-dependent.

- By applying a forward Euler discretization with sampling period T_s , we obtain the discrete-time prediction model used in the MPC:

$$\mathbf{x}_{e,k+1} = \mathbf{x}_{e,k} + T_s \mathbf{J}_{ee}(\boldsymbol{\theta}_k) \mathbf{u}_k \quad (3.7)$$

Equation (3.7), as introduced, describes the full Cartesian velocity of the end-effector in $\mathbb{R}^6$, including both its linear and angular components. However, in this work, only the translational part is used to predict the end-effector position. The orientation is instead handled separately through quaternion kinematics.

To represent the end-effector orientation, a unit quaternion $\mathbf{q} \in \mathbb{R}^4$ is adopted. Its time evolution is governed by the standard quaternion differential equation:

$$\dot{\mathbf{q}}(t) = \frac{1}{2} \boldsymbol{\omega}_{\text{pure}}(t) \otimes \mathbf{q}(t) \quad (3.8)$$

where $\boldsymbol{\omega}_{\text{pure}} = [\boldsymbol{\omega}; 0] \in \mathbb{R}^4$ is a pure quaternion formed from the angular velocity vector $\boldsymbol{\omega} \in \mathbb{R}^3$, and $\otimes$ denotes the quaternion product as defined in equation (1.15). The angular velocity is obtained as $\boldsymbol{\omega} = \mathbf{J}_o(\boldsymbol{\theta})\mathbf{u}$, where $\mathbf{J}_o$ is the rotational part of the geometric Jacobian.

To integrate this equation in discrete time using the forward Euler method, we obtain:

$$\mathbf{q}_{k+1}^{\text{raw}} = \mathbf{q}_k + \frac{1}{2} T_s (\boldsymbol{\omega}_{\text{pure},k} \otimes \mathbf{q}_k) \quad (3.9)$$

To preserve the unit-norm constraint of the quaternion, symbolic normalization is performed after the integration, as detailed in Section 3.3.

This quaternion-based approach is widely used in robotics due to its ability to represent arbitrary 3D orientations without singularities. Compared to Euler angles, quaternions avoid issues such as gimbal lock and discontinuities, and are more suitable for optimization-based control frameworks [43].

- The full rigid-body dynamics for torque control:

$$\mathbf{M}(\boldsymbol{\theta})\ddot{\boldsymbol{\theta}} + \mathbf{C}(\boldsymbol{\theta}, \dot{\boldsymbol{\theta}})\dot{\boldsymbol{\theta}} + \mathbf{F}_v(\dot{\boldsymbol{\theta}}) + \mathbf{g}(\boldsymbol{\theta}) = \boldsymbol{\tau} \quad (3.10)$$

where $\mathbf{M}$ is the mass matrix, $\mathbf{C}$ captures Coriolis and centrifugal effects, $\mathbf{F}_v$ models viscous friction, and $\mathbf{g}$ accounts for gravity. These equations can be discretized to construct the functions f and h in the MPC formulation.

Alternatively, empirical (or data-driven) models are identified from experimental data, using parametric models such as ARX, ARMAX, or NARX, or more complex structures like neural networks. These models can be effective when the physical system is difficult to model analytically, though they may require large amounts of data and may lack interpretability.

The choice of the model represents a fundamental trade-off between the accuracy of the system representation and the computational tractability of the optimization problem to be solved in real-time.

In practical applications, especially when dealing with resource-constrained hardware or strict real-time requirements, it is often necessary to favour simpler models over highly accurate ones. In this work, a kinematic model is adopted, in which the robot dynamics are approximated by its first-order differential kinematics. This simplification enables fast optimization and real-time execution, at the cost of a less precise representation of physical effects such as inertia, Coriolis forces, and joint friction.

While full rigid-body dynamics provide higher fidelity and allow for accurate force and torque prediction, their inclusion leads to a significant increase in computational burden due to the nonlinearities and the larger number of states involved. Conversely, kinematic models are less demanding and can still offer satisfactory tracking performance for motion planning and obstacle avoidance, particularly in moderately dynamic contexts.

This trade-off reflects a common design principle in control architecture design: balancing model fidelity with the computational feasibility of the online optimization problem.

3.2.2 The cost function

The cost function, denoted by J , encodes the high-level control objectives into a scalar quantity to be minimized by the optimization algorithm at each sampling instant. Its formulation is central to shaping the desired system behaviour by balancing possibly conflicting goals, such as tracking accuracy, energy efficiency, and control smoothness.

A common structure in the MPC literature, especially in industrial applications, separates the prediction horizon N_p and control horizon N_c , and penalizes both the tracking error and the rate of change of the control inputs:

$$J(k) = \sum_{i=1}^{N_p} \|\hat{\mathbf{y}}_{k+i|k} - \mathbf{r}_{k+i|k}\|_{\mathbf{Q}}^2 + \sum_{i=0}^{N_c-1} \|\Delta \mathbf{u}_{k+i|k}\|_{\mathbf{R}}^2 \quad (3.11)$$

where $\Delta \mathbf{u}_{k+i|k} = \mathbf{u}_{k+i|k} - \mathbf{u}_{k+i-1|k}$ denotes the change in control input between successive steps. Penalizing this term encourages smoother control profiles, reduces mechanical stress on actuators, and helps mitigate abrupt changes in torque or velocity that could compromise stability or hardware longevity.

In this work, however, and consistently with the literature on NMPC for robotics [2], a different quadratic cost function is adopted. No separate control horizon is defined; instead, a single prediction horizon of length N is used to optimize both the state and input trajectories. This function directly penalizes the state tracking error and the control input magnitude at each stage of the prediction horizon. The overall cost is the sum of these stage costs, F , over the horizon N .

The resulting cost function directly penalizes the state tracking error and the magnitude of the control inputs at each stage:

$$J(k) = \sum_{i=0}^N F(\mathbf{e}_{i|k}, \mathbf{u}_{i|k}), \quad \text{with} \quad F(\mathbf{e}, \mathbf{u}) = \mathbf{e}^\top \mathbf{Q} \mathbf{e} + \mathbf{u}^\top \mathbf{R} \mathbf{u} \quad (3.12)$$

where:

- $\mathbf{e}_{i|k} = \hat{\mathbf{y}}_{i|k} - \mathbf{r}_{i|k}$ is the output tracking error at prediction step i , based on information available at time k ;
- $\mathbf{u}_{i|k}$ is the control input applied at step i of the prediction horizon;
- $\mathbf{Q}$ and $\mathbf{R}$ are symmetric positive definite weighting matrices that tune the relative importance of tracking performance and control effort.

Although the cost function encodes the primary performance objectives, its solution must also comply with the system's physical and operational constraints, which are discussed in the next section.

3.2.3 The constraints

One of the main reasons for the widespread adoption of MPC in control applications is its ability to explicitly incorporate system constraints into the optimization problem. This feature is particularly valuable in robotics, where the system must operate within strict physical limitations, such as joint position and velocity bounds, actuator capabilities, and safety margins for collision avoidance.

Constraints define the set of admissible control actions and system trajectories over the prediction and control horizons. They are commonly expressed as inequality conditions of the form:

$$\begin{cases} \mathbf{u}_{\min} \leq \mathbf{u} \leq \mathbf{u}_{\max} \\ \mathbf{x}_{\min} \leq \mathbf{x} \leq \mathbf{x}_{\max} \\ \mathbf{y}_{\min} \leq \mathbf{y} \leq \mathbf{y}_{\max} \end{cases} \quad (3.13)$$

where $\mathbf{u}$, $\mathbf{x}$, and $\mathbf{y}$ denote the system inputs, states, and outputs, respectively.

These constraints can be classified as:

- **Hard constraints**, which must always be satisfied. Violation of these constraints typically implies physical infeasibility or safety hazards.
- **Soft constraints**, which allow limited violations through the introduction of non-negative slack variables $\varepsilon \geq 0$. For example, an output constraint may be relaxed as:

$$\mathbf{y} \leq \mathbf{y}_{\max} + \varepsilon$$

The slack variable is penalized in the cost function, ensuring that violations are allowed only when strictly necessary to preserve feasibility, such as in the presence of modelling errors or external disturbances.

For robotic manipulators, the most fundamental constraints arise from the physical structure of the robot. Let θ_i and u_i denote the angular position and velocity of the i -th joint, respectively. The most common constraints are:

Joint position limits ensure that each joint operates within its mechanical range, preventing damage to the actuators or the robot structure. Moreover, appropriately tuned joint limits can help reduce the risk of self-collision, especially in highly articulated manipulators:

$$\theta_{i,\min} \leq \theta_i \leq \theta_{i,\max} \quad (3.14)$$

Joint velocity limits constrain the rate at which each joint can move:

$$u_{i,\min} \leq u_i \leq u_{i,\max} \quad (3.15)$$

In some scenarios, additional constraints (3.13) may be introduced to ensure that the end-effector maintains a safe clearance above the supporting surface (e.g., by enforcing a minimum height threshold to prevent collisions with the table or ground plane). However, such constraints were not included in the present implementation.

Such constraints play a critical role in ensuring safe and reliable operation, particularly during tasks that involve high-speed motion or operate near the workspace boundaries. Additional constraints related to obstacle avoidance and interaction with dynamic environments will be addressed in later sections of this chapter.

3.2.4 The MPC optimization problem

By combining the previously discussed components (the prediction model, the cost function, and the constraints) the MPC strategy can be formulated as a finite-horizon optimal control problem, solved online at each sampling instant k .

The optimization problem takes the following form:

$$\begin{aligned} \min_{\mathbf{u}(k)} \quad & J(k) = \sum_{i=0}^N F(\mathbf{e}_{i|k}, \mathbf{u}_{i|k}) \\ \text{subject to} \quad & \mathbf{x}_{i+1|k} = f(\mathbf{x}_{i|k}, \mathbf{u}_{i|k}), \quad i = 0, \dots, N-1 \\ & \mathbf{x}_{0|k} = \hat{\mathbf{x}}_k \\ & \mathbf{u}_{\min} \leq \mathbf{u}_{i|k} \leq \mathbf{u}_{\max}, \quad i = 0, \dots, N-1 \\ & \mathbf{x}_{\min} \leq \mathbf{x}_{i|k} \leq \mathbf{x}_{\max}, \quad i = 0, \dots, N \end{aligned} \quad (3.16)$$

Here, $F(\mathbf{e}_{i|k}, \mathbf{u}_{i|k})$ is the stage cost introduced in (3.12), and $\hat{\mathbf{x}}_k$ is the current estimated state. The system dynamics, defined by $\mathbf{x}_{i+1|k} = f(\mathbf{x}_{i|k}, \mathbf{u}_{i|k})$, incorporate both the differential kinematics for the end-effector position (3.7) and the quaternion-based orientation model (3.9). Input and state constraints are enforced according to (3.15) and (3.14), respectively.

Only the first control input of the optimal sequence, $\mathbf{u}_{0|k}^*$, is applied to the system. At the next time step, the entire problem is re-solved using updated state information. This receding horizon strategy enables the controller to dynamically react to disturbances and model uncertainties.

3.3 MPC for trajectory tracking in an obstacle-free environment

The first step in developing the complete controller is to design and validate a baseline MPC focused solely on trajectory tracking. This initial formulation, for an obstacle-free environment, allows us to establish the core components of the controller: the robot's prediction model, the cost function for tracking, and the physical constraints of the manipulator.

3.3.1 Decision variables and prediction model

The optimization problem is formulated over a finite prediction horizon of length N , starting from the current time step k . The decision variables to be optimized are the control inputs and the predicted states:

Control inputs: The control input sequence $\mathbf{U} = [\mathbf{u}_{0|k}, \dots, \mathbf{u}_{N-1|k}]$ is represented as a column vector $\mathbf{U} \in \mathbb{R}^{6N}$, where each element $\mathbf{u}_{i|k} \in \mathbb{R}^6$ corresponds to the joint velocity command predicted for step i ahead of time k .

State trajectory: Similarly, the predicted state trajectory $\mathbf{X} = [\mathbf{x}_{0|k}, \dots, \mathbf{x}_{N|k}]$ is represented as a column vector $\mathbf{X} \in \mathbb{R}^{13(N+1)}$. Each predicted state $\mathbf{x}_{i|k} \in \mathbb{R}^{13}$ contains both the robot's joint configuration and its end-effector pose:

$$\mathbf{x}_{i|k} = \begin{bmatrix} \mathbf{x}_{e,i|k} \\ \boldsymbol{\theta}_{i|k} \end{bmatrix} \quad (3.17)$$

where $\mathbf{x}_{e,i|k} \in \mathbb{R}^7$ is the predicted end-effector pose at stage i (composed of a 3D position vector $\mathbf{p}_{i|k}$ and a 4D unit quaternion $\mathbf{q}_{i|k}$), and $\boldsymbol{\theta}_{i|k} \in \mathbb{R}^6$ are the predicted joint positions.

$$\mathbf{x}_{e,i|k} = \begin{bmatrix} \mathbf{p}_{i|k} \\ \mathbf{q}_{i|k} \end{bmatrix} \quad (3.18)$$

The state transition model defines the kinematic constraints that link the predicted states over the horizon. It predicts the state at step $i+1$ based on the predicted state and control at step i , all conditioned on the information available at time k . This model is obtained by discretizing the robot's differential kinematics using a sampling time T_s .

Joint position prediction: The future joint positions are obtained by integrating the joint velocities (the control inputs $\mathbf{u}_i$):

$$\boldsymbol{\theta}_{i+1|k} = \boldsymbol{\theta}_{i|k} + T_s \mathbf{u}_{i|k} \quad (3.19)$$

End-effector pose prediction: The future end-effector pose is predicted by integrating its Cartesian velocity, which is derived from the joint velocities via the geometric Jacobian $\mathbf{J}_{ee}$. The Jacobian is split into its linear part $\mathbf{J}_p \in \mathbb{R}^{3 \times 6}$ and its angular part $\mathbf{J}_o \in \mathbb{R}^{3 \times 6}$. To reduce the computational burden, the Jacobians $\mathbf{J}_p$ and $\mathbf{J}_o$ are computed once at the beginning of each control cycle based on the current joint configuration, and then assumed constant throughout the prediction horizon. This approximation avoids the need to recompute the Jacobian matrices at each stage, allowing for a faster optimization without significantly compromising accuracy.

The position part of the pose is updated as:

$$\mathbf{p}_{i+1|k} = \mathbf{p}_{i|k} + T_s \mathbf{J}_p(\boldsymbol{\theta}_k) \mathbf{u}_{i|k} \quad (3.20)$$

The orientation, represented by a unit quaternion $\mathbf{q}_i = [q_x, q_y, q_z, q_w]^\top$, is updated using the relationship between quaternion rate and angular velocity $\boldsymbol{\omega}_{i|k} = \mathbf{J}_o(\boldsymbol{\theta}_k) \mathbf{u}_{i|k}$. The pure quaternion corresponding to the angular velocity is formed as $\boldsymbol{\omega}_{\text{pure},i|k} = [\boldsymbol{\omega}_{i|k}, 0]^\top$, and the orientation is then integrated as:

$$\mathbf{q}_{i+1|k}^{\text{raw}} = \mathbf{q}_{i|k} + \frac{1}{2} T_s (\boldsymbol{\omega}_{\text{pure},i|k} \otimes \mathbf{q}_{i|k}) \quad (3.21)$$

where $\otimes$ denotes the quaternion product as defined in equation (1.15), and the convention $[q_x, q_y, q_z, q_w]$ is used.

To counteract numerical drift from the integration, which can cause the quaternion to lose its unit-norm property, the resulting quaternion is symbolically normalized at each step within the optimization problem:

$$\mathbf{q}_{i+1|k} = \frac{\mathbf{q}_{i+1|k}^{\text{raw}}}{\sqrt{\|\mathbf{q}_{i+1|k}^{\text{raw}}\|^2 + \epsilon}} \quad (3.22)$$

where $\|\cdot\|^2$ denotes the squared Euclidean norm, and a small epsilon term ($\epsilon \approx 10^{-12}$) is added to the denominator to ensure numerical stability and avoid division by zero.

Equations (3.20), (3.22), and (3.19) define the discrete-time state transition function $f(\mathbf{x}_{i|k}, \mathbf{u}_{i|k})$ used in the MPC optimization problem to impose the kinematic constraints $\mathbf{x}_{i+1|k} = f(\mathbf{x}_{i|k}, \mathbf{u}_{i|k})$ at each step of the prediction horizon.

$$\mathbf{x}_{i+1|k} = \begin{bmatrix} \mathbf{x}_{e,i+1|k} \\ \boldsymbol{\theta}_{i+1|k} \end{bmatrix} = \begin{bmatrix} \mathbf{p}_{i+1|k} \\ \mathbf{q}_{i+1|k} \\ \boldsymbol{\theta}_{i+1|k} \end{bmatrix} = \begin{bmatrix} \mathbf{p}_{i|k} + T_s \mathbf{J}_p(\boldsymbol{\theta}_k) \mathbf{u}_{i|k} \\ \frac{\mathbf{q}_{i+1|k}^{\text{raw}}}{\sqrt{\|\mathbf{q}_{i+1|k}^{\text{raw}}\|^2 + \epsilon}} \\ \boldsymbol{\theta}_{i|k} + T_s \mathbf{u}_{i|k} \end{bmatrix} \quad (3.23)$$

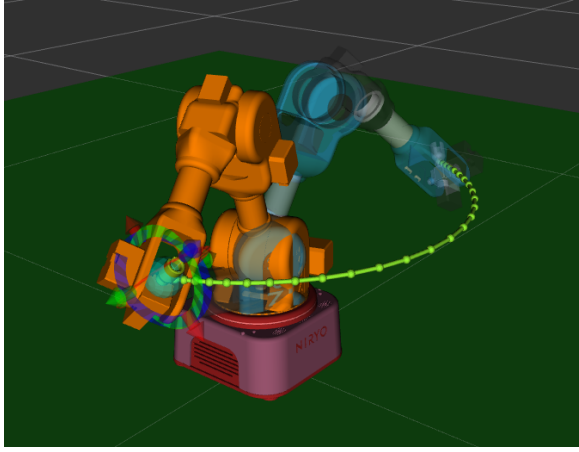
These constraints are imposed on all elements of the predicted state trajectory $\mathbf{X}$, except for the initial state $\mathbf{x}_{0|0}$, which is known at the beginning of each control cycle. This formulation reflects the decomposition of the kinematic prediction model introduced in Section 3.2, where end-effector motion is modelled by combining differential kinematics for position and quaternion integration for orientation.

3.3.2 Reference trajectory generation

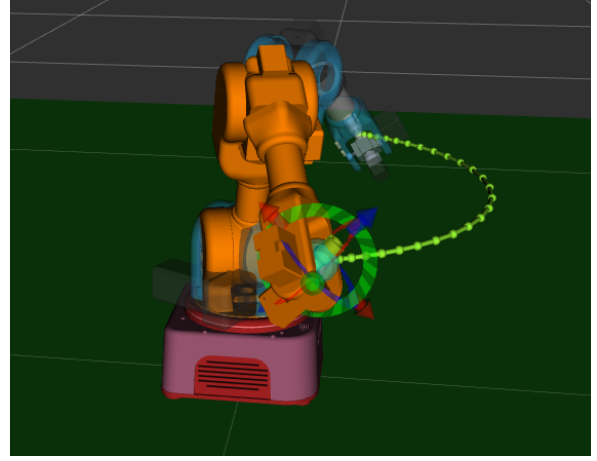
The generation of the reference trajectory plays a key role in the proposed control pipeline, as it provides the desired end-effector poses to be tracked by the MPC controller. Each pose consists of a 3D position and an orientation expressed as a unit quaternion.

To generate this trajectory, a dedicated C++ node named `desired_trajectory_sampling` was developed (see Appendix 2). The node uses MoveIt 2's `MoveGroupInterface` and the `CHOMP`

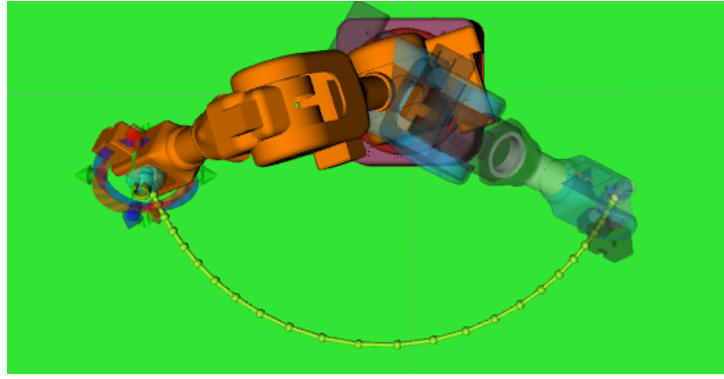
planner to compute a Cartesian path from the current robot configuration to a manually defined target pose. This pose is expressed in the world reference frame and is hard-coded in the script as a `geometry_msgs::msg::PoseStamped` message, containing both position and orientation in quaternion form.



(a) Isometric view



(b) Lateral view



(c) Top view

Figure 3.1: Visualization of the end-effector trajectory in RViz.

This approach was selected because it guarantees that the reference trajectory is kinematically feasible. The underlying planning and visualization logic was originally developed during the internship project carried out prior to this thesis, and has been extended here to support real-time sampling and offline storage for closed-loop control.

The trajectory is sampled at a fixed rate of 10 Hz (i.e., every 100 ms), using forward kinematics to compute the end-effector pose at each sampled time step. This sampling rate was intentionally chosen to match the execution frequency of the MPC controller, ensuring temporal consistency between reference updates and prediction steps.

The resulting sequence of poses is stored in a plain-text file named `ref_ee_traj_100ms.txt`. Each line of the file contains seven numerical values: the first three represent the Cartesian position, and the remaining four define the orientation as a unit quaternion. An extract is provided in Appendix 3 to illustrate its structure. This file is pre-generated before controller execution and remains static during runtime; the reference is therefore not updated online.

The end-effector trajectory, as generated by the motion planner, consists of a continuous path and a set of discretized reference poses. In the RViz 2 environment, this information is visualized through a green curve representing the smooth trajectory, while discrete waypoints are marked by green spheres, offering a clear depiction of the planned motion over time (Fig. 3.1).

Trajectory usage in the MPC pipeline: The reference trajectory is loaded at runtime from a text file and stored as a 31×7 matrix, representing 3.1 seconds of motion sampled at 100 ms. Although the original trajectory contains only 31 time steps, the simulation is configured to run for a longer duration, defined by a constant `max_timesteps` set to 100. This allows the controller’s behavior to be observed not only during transient motion but also in steady-state conditions.

To ensure sufficient reference data throughout the simulation, the last pose of the trajectory is repeated from step 31 onward, effectively holding the target constant beyond the planned motion. At each control cycle k , the optimizer does not use the entire trajectory, but only the segment corresponding to the prediction horizon. Specifically, a block of N consecutive poses starting at time index k is extracted, flattened into a $7N \times 1$ column vector, and passed as part of the parameter vector to the solver. This approach ensures time alignment between the predicted states and the corresponding reference values. The reference extraction is implemented efficiently in C++ as follows:

- The file is read line-by-line, and each pose is stored in a `std::vector` named `reference_ee_traj`.
- If the file contains fewer than `max_timesteps + N - 1` entries, the last pose is duplicated as padding to reach the required length.
- At each iteration, only the next N poses are selected and reshaped into a flat vector to be passed to the NLP solver.

This design guarantees that the optimizer always receives a consistent set of reference poses over the horizon, even beyond the duration of the original trajectory.

3.3.3 Position and orientation error computation

At each step of the prediction horizon, the MPC controller minimizes the pose tracking error between the predicted end-effector pose and the corresponding reference pose. This section details the formulation of both the position and orientation error vectors used in the cost function.

At each stage i of the prediction horizon, the MPC controller minimizes the tracking error between the predicted pose $\mathbf{p}_{i|k}, \mathbf{q}_{i|k}$ and the corresponding reference pose $\mathbf{p}_{\text{ref},i}, \mathbf{q}_{\text{ref},i}$, which are extracted from the pre-sampled trajectory.

Position error: This corresponds to a direct subtraction in $\mathbb{R}^3$, and yields the translational deviation to be minimized by the cost function. The position error is defined as the difference between the predicted and reference Cartesian positions:

$$\mathbf{e}_{\text{pos},i} = \mathbf{p}_{i|k} - \mathbf{p}_{\text{ref},i} \quad (3.24)$$

Orientation error: When tracking a desired pose in 3D space, the orientation component of the reference must be compared with the predicted one in a way that respects the geometric structure of rotations. Unlike positions, quaternions cannot be simply subtracted to obtain a meaningful error. As

explained in [3], subtracting two unit quaternions does not yield a geometrically interpretable difference and can lead to singularities or inconsistencies, particularly due to the double-cover property of unit quaternions.

To compute a meaningful orientation error, the quaternion error is defined as the relative rotation between the predicted and the reference orientation. Given the predicted unit quaternion $\mathbf{q}_{i|k}$ and the reference quaternion $\mathbf{q}_{\text{ref},i}$, the following steps are performed:

1. Sign ambiguity handling

Due to the double covering of $\text{SO}(3)$ by unit quaternions, a quaternion and its negation represent the same rotation. However, numerically they are distinct. To avoid discontinuities, the predicted quaternion is aligned with the reference as follows:

$$\mathbf{q}_{\text{pred,aligned},i} = \begin{cases} \mathbf{q}_{i|k} & \text{if } \mathbf{q}_{\text{ref},i}^\top \mathbf{q}_{i|k} \geq 0 \\ -\mathbf{q}_{i|k} & \text{otherwise} \end{cases}$$

2. Quaternion error computation

The orientation error is defined as the quaternion that rotates from the predicted orientation to the reference one. This is obtained via the quaternion product as defined in equation (1.15):

$$\mathbf{q}_{\text{err},i} = \mathbf{q}_{\text{ref},i} \otimes \mathbf{q}_{\text{pred,aligned},i}^*$$

where $(\cdot)^*$ denotes the quaternion conjugate and $\otimes$ is the quaternion product. This formulation yields a unit quaternion $\mathbf{q}_{\text{err},i}$ encoding the rotational difference.

3. Vector error extraction

The error quaternion can be interpreted as:

$$\mathbf{q}_{\text{err},i} = \begin{bmatrix} \sin(\theta/2) \mathbf{u} \\ \cos(\theta/2) \end{bmatrix}$$

where θ is the rotation angle and $\mathbf{u}$ is the unit axis of rotation. For small-angle deviations (which is typically the case in tracking), the approximation $\sin(\theta/2) \approx \theta/2$ holds, so the **vector part** of the quaternion approximates the axis-angle representation:

$$\mathbf{e}_{\text{orient},i} \approx 2 \cdot \text{vec}(\mathbf{q}_{\text{err},i}) \quad (3.25)$$

where $\text{vec}(\cdot)$ extracts the first three elements of the quaternion. The result is a 3D orientation error vector expressed in minimal form.

This formulation provides a smooth and singularity-free way to define orientation errors, suitable for optimization and consistent with the geometry of $\text{SO}(3)$. The final 6D pose tracking error is then constructed by concatenating the position (3.24) and orientation (3.25) components:

$$\mathbf{e}_i = \begin{bmatrix} \mathbf{e}_{\text{pos},i} \\ \mathbf{e}_{\text{orient},i} \end{bmatrix} \quad (3.26)$$

This vector is used to define the tracking cost in the MPC formulation.

3.3.4 Joint constraints

To ensure safe and physically feasible motion, the MPC formulation explicitly incorporates joint-level constraints on both angular position and velocity. These constraints correspond to the general theoretical expressions introduced in Section 3.2, namely (3.14) and (3.15).

The numerical bounds were obtained by analyzing the robot description file (`niryo_ned2.urdf`), which defines the `<limit>` tag for each revolute joint. The joint position constraints were slightly approximated to simplify the numerical implementation. Specifically, lower bounds were approximated by excess and upper bounds by defect. The resulting limits used in the MPC problem are:

$$\begin{cases} -2.9998 \leq \theta_1 \leq 2.9998 \\ -1.8325 \leq \theta_2 \leq 0.6101 \\ -1.3400 \leq \theta_3 \leq 1.5700 \\ -2.0900 \leq \theta_4 \leq 2.0900 \\ -1.9200 \leq \theta_5 \leq 1.9228 \\ -2.5300 \leq \theta_6 \leq 2.5300 \end{cases} \quad (3.27)$$

$$\begin{cases} -10.0 \leq u_1 \leq 10.0 \\ -8.0 \leq u_2 \leq 8.0 \\ -8.0 \leq u_3 \leq 8.0 \\ -2.0 \leq u_4 \leq 2.0 \\ -2.0 \leq u_5 \leq 2.0 \\ -2.0 \leq u_6 \leq 2.0 \end{cases} \quad (3.28)$$

These constraints are enforced at every stage of the prediction horizon, ensuring that all predicted joint configurations and control inputs remain within the robot's feasible workspace. They also contribute to reducing the risk of self-collision and help maintain compatibility with the hardware limitations of the Niryo Ned2. All joint positions θ_i are expressed in radians (rad), and joint velocities u_i in radians per second (rad/s), as defined in the `niryo_ned2.urdf` file.

3.3.5 Optimization problem formulation

The goal of this MPC controller is to compute a sequence of joint velocity commands $\mathbf{u}_{i|k}$ that minimize the tracking error between the robot's intended motion and a predefined reference trajectory. This specific formulation instantiates the general structure introduced in Equation (3.16), by explicitly detailing the end-effector's kinematic prediction model, while preserving the same quadratic cost function used for pose tracking. More formally, given the discrete-time kinematic model described in Equation (3.23), the control policy is designed to ensure that the instantaneous pose tracking error asymptotically vanishes:

$$\lim_{k \rightarrow \infty} \mathbf{e}_k = \mathbf{0}$$

where $\mathbf{e}_k := \mathbf{e}_{0|k} \in \mathbb{R}^6$ is the pose tracking error at the current control cycle, defined in Equation (3.26).

At each control cycle k , the controller solves a non-linear optimization problem to compute the optimal sequence $\{\mathbf{u}_{0|k}, \dots, \mathbf{u}_{N-1|k}\}$ over a finite prediction horizon of length N . The optimization problem is formulated as:

$$\begin{aligned}
 \min_{\mathbf{u}_{0|k}, \dots, \mathbf{u}_{N-1|k}} \quad & J(k) = \sum_{i=0}^{N-1} \left(\mathbf{e}_{i|k}^\top \mathbf{Q} \mathbf{e}_{i|k} + \mathbf{u}_{i|k}^\top \mathbf{R} \mathbf{u}_{i|k} \right) \\
 \text{subject to} \quad & \mathbf{p}_{i+1|k} = \mathbf{p}_{i|k} + T_s \mathbf{J}_p(\boldsymbol{\theta}_k) \mathbf{u}_{i|k} \\
 & \mathbf{q}_{i+1|k} = \frac{\mathbf{q}_{i+1|k}^{\text{raw}}}{\sqrt{\|\mathbf{q}_{i+1|k}^{\text{raw}}\|^2 + \epsilon}} \\
 & \boldsymbol{\theta}_{i+1|k} = \boldsymbol{\theta}_{i|k} + T_s \mathbf{u}_{i|k} \\
 & \mathbf{x}_{0|k} = \hat{\mathbf{x}}_k \\
 & \theta_{\min}^{(j)} \leq \theta_{i+1|k}^{(j)} \leq \theta_{\max}^{(j)} \\
 & -u_{\max}^{(j)} \leq u_{i|k}^{(j)} \leq u_{\max}^{(j)} \\
 & \text{for } j = 1, \dots, 6; \quad i = 0, \dots, N-1
 \end{aligned} \tag{3.29}$$

In this formulation:

- $\mathbf{p}_{i|k}, \mathbf{q}_{i|k}$ represent the position and orientation (as a quaternion) of the end-effector at stage i of the prediction horizon (3.18);
- $\boldsymbol{\theta}_{i|k} \in \mathbb{R}^6$ are the joint positions;
- $\mathbf{u}_{i|k} \in \mathbb{R}^6$ are the joint velocities;
- $\mathbf{e}_{i|k} \in \mathbb{R}^6$ is the 6D pose tracking error defined in Equation (3.26);
- $\mathbf{Q}, \mathbf{R} \in \mathbb{R}^{6 \times 6}$ are diagonal, positive-definite weighting matrices;
- The condition $\mathbf{x}_{0|k} = \hat{\mathbf{x}}_k$ initializes the predicted trajectory with the current robot state;
- Joint constraints are imposed at each prediction step, as specified in (3.27) and (3.28).

The implementation of this formulation is provided in Appendix 4. Symbolic expressions for the cost, dynamics, and constraints were defined using CasADi, and the non-linear program is solved online using the IPOPT solver.

In CasADi, the optimization problem is formulated as a non-linear program (NLP) characterized by the tuple $\{\mathbf{x}, \mathbf{f}, \mathbf{g}, \mathbf{p}\}$. In this formulation, $\mathbf{x}$ denotes the concatenated vector of decision variables, which includes both the joint velocities and the predicted states over the prediction horizon. The scalar function $\mathbf{f}$ represents the cost to be minimized, while $\mathbf{g}$ collects the set of constraints that the solution must satisfy. The vector $\mathbf{p}$ contains the parameters that are provided to the solver at runtime.

Specifically, the parameters passed during execution consist of the current state estimate $\hat{\mathbf{x}}_k$, the geometric Jacobians $\mathbf{J}_p$ and $\mathbf{J}_o$ evaluated at the current joint configuration $\boldsymbol{\theta}_k$, and the flattened reference trajectory $\mathbf{S}_{\text{ref}}$. This formulation enables real-time trajectory tracking in free space, ensuring both joint-level feasibility and strict adherence to the robot's kinematic constraints.

3.3.6 Execution logic of the MPC controller

The high-level execution logic of the MPC controller involves computing an optimal velocity command $\mathbf{u}_{0|k}$ at each control cycle k , which is then stored and applied at the beginning of the following cycle $k+1$,

thus decoupling computation from actuation timing. This scheme ensures that the optimization process can be completed within the fixed control period without compromising real-time execution requirements (Fig. 3.2).

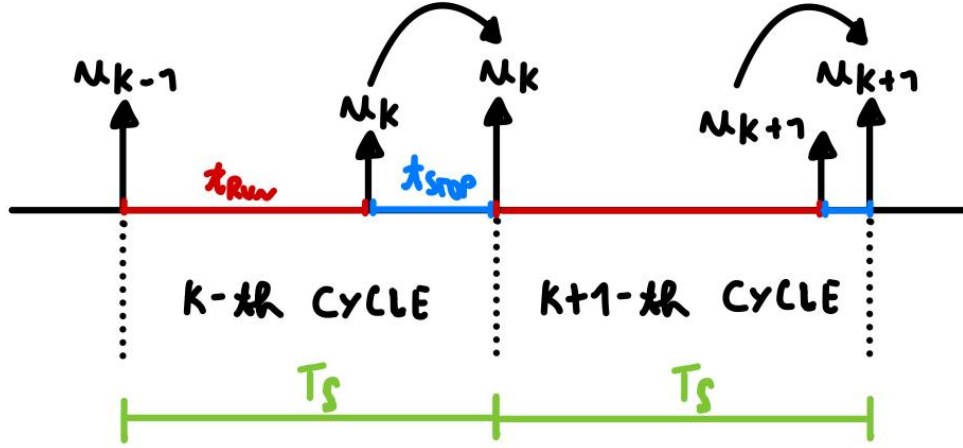


Figure 3.2: Execution timing of the MPC logic over consecutive control cycles.

Each control cycle is divided into two time intervals:

- t_{run} : the time allocated for executing all necessary computations, including acquiring the current joint state and end-effector pose via MoveIt 2, computing the Jacobians (also via MoveIt 2), updating the reference trajectory, and solving the non-linear optimization problem using CasADi and IPOPT;
- t_{stop} : the remaining time until the fixed sampling period T_s elapses.

The duration of t_{run} is variable, depending on system latency and computational load, while t_{stop} is adjusted accordingly to ensure that each control cycle maintains a constant duration of T_s .

The controller is structured so that all required data are acquired and processed during t_{run} , ensuring that the control input computed at cycle k is available for execution at cycle $k + 1$.

The execution of the MPC control loop is organized into three conceptual phases, repeated at each control step k . The goal of each iteration is to compute and later apply the optimal joint velocity command $\mathbf{u}_{0|k}$, based on the current robot state and the reference trajectory.

Phase 1: Timing management and control command application

- 1.1) A timestamp t_{end} is recorded, and the loop checks whether the elapsed time since the last control step ($t_{\text{end}} - t_{\text{start}}$) is below 100 ms. If so, the node waits to maintain a fixed control frequency of 10 Hz. Once the condition is met, t_{start} is updated to begin the new cycle.
- 1.2) The previously computed control command $\mathbf{u}_{0|k-1}$ is published to the low-level controller for execution.

Phase 2: Data acquisition and problem setup

- 2.1) The robot state is updated by querying the current joint angles and end-effector pose using the MoveIt 2 API functions `getCurrentJointValues()` and `getCurrentPose()`, respectively. These values are used to construct the current state estimate $\hat{\mathbf{x}}_k = [\mathbf{x}_{e,k}^\top, \boldsymbol{\theta}_k^\top]^\top$.
- 2.2) The geometric Jacobians $\mathbf{J}_p$ and $\mathbf{J}_o$ for the end-effector are extracted from the full 6×6 Jacobian matrix returned by MoveIt 2's `getJacobian()` function.
- 2.3) A segment of the pre-sampled reference trajectory is selected starting at index k , and the next N poses are flattened into a column vector of size $7N$, which serves as a parameter for the solver.
- 2.4) All relevant data (initial state, Jacobians, reference trajectory) are collected in the CasADi dictionary `arg["p"]`, which defines the non-linear program.

Phase 3: Optimization and result management

- 3.1) The non-linear program is solved using IPOPT, producing a decision vector that includes the full sequence of optimal joint velocities $\{\mathbf{u}_{i|k}^*\}_{i=0}^{N-1}$ and the predicted states.
- 3.2) The first control input $\mathbf{u}_{0|k}^*$ is extracted and stored to be published in the subsequent control cycle. The entire solution vector is saved to warm-start the optimization in the next iteration, reducing solver time.

Special case - first iteration: No control input is applied during the very first iteration, since no previous solution exists. The process begins directly from Phase 2.2, where the robot state and Jacobians are computed, and proceeds with problem setup and optimization (Phases 2 and 3). In the second iteration, all three phases are executed, except that the wait in Phase 1.1 is skipped, since the first loop only serves to prepare the initial control input.

It is important to note that, except during the first iteration, the MPC control cycle and the `for` loop iteration are generally not aligned in real time. They coincide only if the total computation time exactly matches the desired 100 ms period, corresponding to a 10 Hz control frequency.

As already done in Section 3.1.1 to address the same issue in the context of dynamic obstacle motion, a monotonic clock (`RCL_STEADY_TIME`) is also used here to accurately measure the elapsed time and ensure consistent control timing.

This structured control loop guarantees synchronization between sensing, planning, and actuation, while respecting the timing constraints necessary for real-time MPC execution.

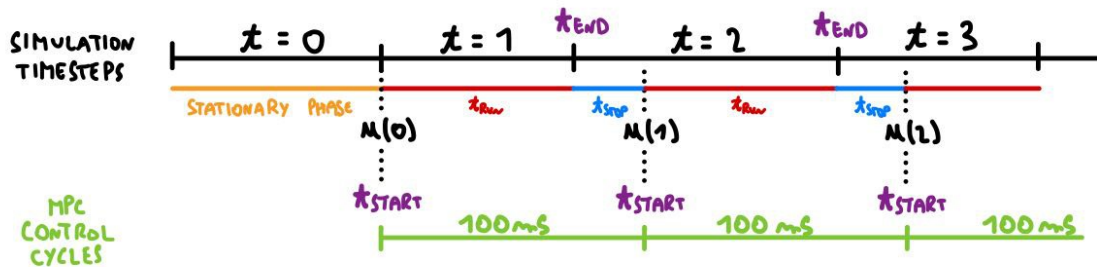


Figure 3.3: Timing diagram illustrating the relationship between simulation timesteps and MPC control cycles.

Each simulation timestep (corresponding to an iteration of the `for` loop) starts at t_{start} and ends at t_{end} . The MPC control cycle, however, extends beyond t_{end} by an additional waiting period t_{stop} , ensuring

a fixed control period of 100 ms. The interval corresponding to the first simulation timestep (iteration $t = 0$) represents a stationary phase during which the manipulator remains still while the initial control input $\mathbf{u}(0)$ is computed and subsequently applied and held constant throughout the first MPC cycle.

3.4 MPC for dynamic obstacle avoidance

Having completed the implementation of a baseline MPC for reference tracking in an obstacle-free environment, along with a complementary node responsible for moving a box-shaped obstacle along a predefined trajectory within the MoveIt 2 simulation scene, we are now ready to address the core challenge of this thesis: the integration of dynamic obstacle avoidance into the MPC framework.

To enable this functionality, two key components must be incorporated:

- A GPIO to estimate in real time the velocity and acceleration of the dynamic obstacle from its measured position;
- A set of CBF constraints applied to the end-effector, ensuring safe interaction with the environment.

These modules will be integrated into the existing MPC control pipeline to enable robust and proactive obstacle avoidance. The GPIO allows the controller to anticipate the future motion of the obstacle, while the CBFs introduce formal safety guarantees directly into the optimization problem, preventing collisions at the end-effector level.

The next sections describe the implementation details of both components. Section 3.4.1 addresses the design of the obstacle state estimator and its integration into the controller. Section 3.4.2 presents the formulation and implementation of the CBF constraints within the MPC, including the strategy used to dynamically tune the decay rate γ .

3.4.1 GPIO design and implementation

In order to enable the MPC to react to the dynamic obstacle, its current position must be provided to the optimizer in real time. This requires coordination between the node responsible for simulating the obstacle's motion and the MPC controller node. Specifically, a publisher has been added to the obstacle node to continuously broadcast its position using geometry messages on a dedicated topic. Correspondingly, a subscriber has been implemented within the MPC node to receive and store these updates. Finally, the solver parameters are updated at each control step so that the most recent obstacle position is passed to the non-linear problem (NLP) before solving.

This setup ensures that the obstacle's state is integrated into the MPC loop in a consistent and timely manner. The obstacle node publishes its position at a rate of 25 Hz (i.e., every 40 ms), while the MPC operates at a slower control frequency of 10 Hz (i.e., every 100 ms). As a result, each MPC iteration receives one of the most recent position measurements from the obstacle node. Although not every published obstacle position is used, the high publishing frequency ensures that the data available at each MPC step is up-to-date and reduces the impact of communication delays.

Importantly, the obstacle is already in motion when the MPC controller is first activated. Therefore, from the first iteration onward, the MPC receives a position that reflects the dynamic evolution of the environment. This measured position is denoted as $\mathbf{o}_{k|k}$, representing the true position of the obstacle at the current control step k . It is used as input to the observer module, which estimates the obstacle's future states (position, velocity, and acceleration) according to the GPIO formulation introduced in Chapter 3.

An example of the obstacle positions passed to the NLP at each control cycle is reported in Appendix 5.

Observer gain selection: The GPIO described by equation (1.8) depends on three scalar gains α_1 , α_2 , and α_3 , which must be carefully chosen to ensure stability and satisfactory estimation performance. These parameters influence the speed and responsiveness of the estimation and ultimately affect the quality of the obstacle motion prediction.

In [2], the authors propose a tuned set of gains for a sampling time $T_s = 0.04$ s, yielding the values $\alpha_1 = 5$, $\alpha_2 = 10$, and $\alpha_3 = 2$. However, since in this implementation the MPC operates with a slower sampling time of $T_s = 0.1$ s, a direct reuse of those values would result in overly aggressive behaviour or even instability. Rather than performing a full retuning of the observer for the new sampling time, which could be time-consuming and require empirical adjustment, a scaling approach was adopted to reuse the original design while preserving stability. To address this, a scaling approach has been adopted. Given that the gains in [2] were tuned for a time step t , the new gains are computed by applying the following transformation:

$$\begin{cases} \alpha_1 = a_1 \cdot S \\ \alpha_2 = a_2 \cdot S^2 \\ \alpha_3 = a_3 \cdot S^3 \end{cases}$$

where $S = \frac{t}{T_s}$ is the ratio between the original and the current sampling times, and a_1 , a_2 , and a_3 are the original gains from the literature. For $t = 0.04$ s and $T_s = 0.1$ s, we have $S = 0.4$, which leads to the following scaled gains:

$$\alpha_1 = 2.0, \quad \alpha_2 = 1.6, \quad \alpha_3 = 0.128 \quad (3.30)$$

To verify the stability of the observer under these gains, we compute the error dynamics matrix Φ as defined in (1.11). The spectral radius $\rho(\Phi)$ is then evaluated by computing the eigenvalues of the matrix and checking whether their modulus is strictly less than 1, as required by the convergence condition stated in Chapter 3.

This verification was carried out in MATLAB by constructing the matrix Φ and computing its eigenvalues. The script used for this analysis is reported in Appendix 6. The results confirmed that the chosen gains lead to a stable observer, with $\rho(\Phi) < 1$, and thus guarantee the asymptotic convergence of the estimation error $\mathbf{E}_k$.

GPIO implementation in C++: Unlike the approach proposed in [2], where the obstacle prediction model is embedded directly into the MPC optimization problem, this implementation handles both the observer and the trajectory prediction externally. At each control cycle, the predicted future positions of the obstacle over the control horizon are computed in a separate routine and then passed to the MPC solver as an additional parameter. This design choice does not compromise the effectiveness of the method (since the predicted trajectory is still available at optimization time) but leads to a cleaner software architecture and facilitates real-time logging and debugging of the observer's performance. The implementation follows these steps:

1. As previously introduced, a ROS 2 subscriber is created to receive the obstacle pose in real time from the `/obstacle_pose` topic. The most recent message is stored in the variable `latest_obstacle_pose_msg`, protected by a mutex to ensure thread safety. This variable directly represents the measured position $\mathbf{o}_{k|k}$.

2. Internal variables are defined to maintain the estimated state of the obstacle: position $\hat{\mathbf{o}}_{k|k-1}$, velocity $\hat{\mathbf{v}}_{k|k-1}$, and acceleration $\hat{\mathbf{a}}_{k|k-1}$. The observer operates with the same sampling time T_s as the MPC and uses the gains α_1 , α_2 , and α_3 described in before (3.30).
3. The symbolic formulation of the CasADi NLP is extended to include a new parameter `O_pred_flat_sym`, which encodes the predicted trajectory of the obstacle over the horizon. This variable is flattened into a vector of size $3(N + 1)$, representing $N + 1$ predicted 3D positions. At runtime, the controller constructs the corresponding CasADi DM vector `O_pred_flat_dm` and includes it in the overall parameter vector $\mathbf{p}$ passed to the solver.
4. At each MPC iteration, the observer is updated using the most recent obstacle measurement $\mathbf{o}_{k|k}$, if available, according to equations (1.8). If no new measurement has arrived, the observer uses the last estimate as input. This ensures continuity of operation even in the presence of occasional communication delays.
5. Once the observer produces the current estimated state $(\hat{\mathbf{o}}_{k|k}, \hat{\mathbf{v}}_{k|k}, \hat{\mathbf{a}}_{k|k})$, which serves as the initial condition, the obstacle trajectory is propagated over the entire prediction horizon using the discrete-time constant-acceleration model introduced in equations (1.5).

This model treats the obstacle as a third-order integrator and assumes constant acceleration across the horizon. The resulting sequence of predicted positions $\{\mathbf{o}_{i|k}\}_{i=0}^N$ is flattened into a CasADi DM vector and passed to the solver as part of the parameter dictionary.

This architecture offers a clear modular separation between state estimation and optimal control, while remaining fully compatible with the structure of the MPC problem and the future inclusion of CBF constraints. Additionally, logging both measured and predicted obstacle positions at each iteration enables real-time verification of the observer’s correctness and stability.

3.4.2 Constraint-based safety control

Ensuring collision-free motion in the presence of dynamic obstacles is a fundamental requirement for safe robot operation. To this end, constraint-based safety control methods enforce safety by explicitly restricting the robot’s motion within safe regions defined by suitable barrier functions.

In this work, Control Barrier Functions (CBFs) are employed to guarantee that the robot maintains a minimum safe distance from the obstacle throughout the motion planning and control process. The concept of a safety margin r_{safe} , formalizes the minimum clearance that must be preserved between the robot and the obstacle. By incorporating CBF constraints into the MPC problem, the controller actively prevents the robot from entering unsafe regions, even in the presence of uncertainties and disturbances. The next sections detail the mathematical formulation of the CBFs and their integration into the MPC framework. This constraint-based approach relies on the notion of forward invariance of a safe set, ensuring that if the system state starts in a safe region, the controller keeps it within that region at all future steps. In the discrete-time case, this is enforced via inequalities that ensure the safety function does not decay too rapidly, as described in the following sections.

Definition of the safe distance: In order to ensure that the robot maintains a sufficient clearance from the moving obstacle, a scalar quantity r_{safe} is defined. This value represents the minimum allowable distance between any critical point on the robot and the obstacle’s surface, accounting for both geometric and estimation-related uncertainties. In this work, the safety constraint is primarily enforced on the robot’s

end-effector position; however, as discussed in [2], the same principle can be applied to multiple critical points distributed along the robot's structure to guarantee overall collision avoidance:

$$r_{\text{safe}} = d_{\min} + R_o + r_d \quad (3.31)$$

where:

- $d_{\min}$ is a fixed safety margin (here, $d_{\min} = 0.001$ m, i.e., 1 mm) that ensures minimal clearance even under ideal estimation;
- R_o is the radius of the sphere that tightly bounds the obstacle geometry. For a cube of side $a = 0.1$ m, this is given by $R_o = \frac{a\sqrt{3}}{2} \approx 0.0866$ m;
- r_d is an additional margin that compensates for estimation errors introduced by the observer.

The value r_d is defined as:

$$r_d = \Phi_0 \cdot \delta$$

where Φ_0 is a scalar gain derived from the Lyapunov stability analysis presented in Section 1.5.3, and δ is an upper bound on the initial estimation error $\|\mathbf{E}_0\|$.

Following the method described in [2], the value of Φ_0 was computed by solving the Lyapunov inequality (1.13) using CVX in MATLAB (see Appendix 7 for implementation details). From equation (1.14), the resulting value of Φ_0 is:

$$\Phi_0 = \sqrt{\frac{c_2}{c_1}} \approx 2.4288$$

The value of δ was selected empirically based on simulations of the estimation error. Across multiple runs, the worst-case absolute error was approximately 0.018 m. To provide a conservative safety buffer, we set:

$$r_d = 0.02 \text{ m (i.e., 2 cm)} \Rightarrow \delta = \frac{r_d}{\Phi_0} \approx 0.00823 \text{ m (i.e., 0.823 cm)}$$

The final value used in this work is therefore computed according to equation (3.31):

$$r_{\text{safe}} = 0.001 + 0.0866 + 0.02 = 0.1076 \text{ m (i.e., 10.76 cm)}$$

Geometric interpretation: The geometric meaning of the safety distance can be understood by considering the obstacle as a sphere of radius R_o centered at position $\mathbf{o}_k$, and the end-effector located at position $\mathbf{p}_k$. The total distance between the end-effector and the obstacle center is decomposed into four components: the safety margin $h_{e,k}$, the obstacle radius R_o , the minimum clearance $d_{\min}$, and the uncertainty buffer r_d . This relationship can be expressed as:

$$\|\mathbf{p}_k - \mathbf{o}_k\| = h_{e,k} + R_o + d_{\min} + r_d$$

Solving for $h_{e,k}$ yields the signed safety margin:

$$h_{e,k} = \|\mathbf{p}_k - \mathbf{o}_k\| - r_{\text{safe}} \quad (3.32)$$

Enforcing the condition $h_{e,k} \geq 0$ ensures that the end-effector remains within the admissible safe region, maintaining the desired clearance. This threshold will be used in the next section to construct a Control Barrier Function (CBF) that actively enforces safety during motion (Fig. 3.4).

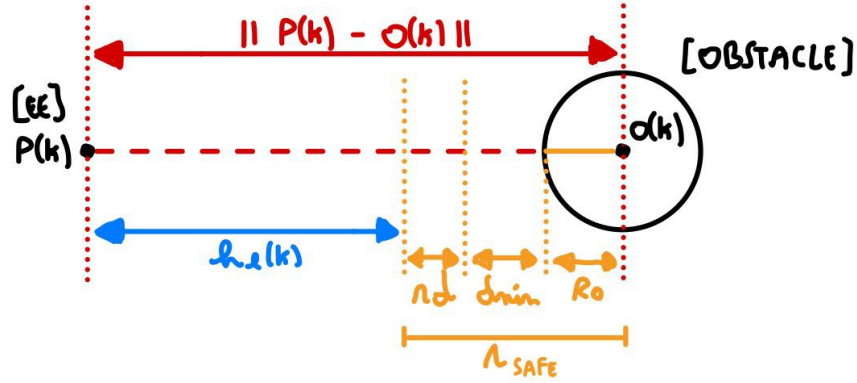


Figure 3.4: Graphical illustration of the safety distance between the end-effector and the obstacle.

Modification of the cost function to include decay rate penalty: Building upon the obstacle-free MPC formulation presented in Equation (3.29), the cost function is extended to incorporate a penalty on the CBF decay rates $\gamma_{i|k}$ along the prediction horizon.

The modified cost function is expressed as:

$$J(k) = \sum_{i=0}^{N-1} F(\mathbf{e}_{i|k}, \mathbf{u}_{i|k}, \gamma_{i|k}) = \sum_{i=0}^{N-1} \left(\mathbf{e}_{i|k}^\top \mathbf{Q} \mathbf{e}_{i|k} + \mathbf{u}_{i|k}^\top \mathbf{R} \mathbf{u}_{i|k} + P_\gamma \gamma_{i|k}^2 \right) \quad (3.33)$$

where:

- $\mathbf{e}_{i|k}$ and $\mathbf{u}_{i|k}$ are, respectively, the pose tracking error and the control input at prediction step i , as defined in the obstacle-free MPC formulation;
- $\gamma_{i|k} \in \mathbb{R}$ is the discrete-time CBF decay rate at step i , introduced as an additional decision variable to allow the controller to modulate safety conservatism dynamically along the prediction horizon;
- $P_\gamma \in \mathbb{R}$ is a scalar weighting parameter (a 1×1 matrix) tuned to balance the trade-off between control efficiency and safety enforcement. Specifically, P_γ is proportional to safety: higher values of P_γ correspond to a stronger emphasis on maintaining a safe distance from the obstacle, potentially at the cost of reduced control flexibility;

In [2], the MPC horizon is set to $N = 1$, so the decay rate γ_k varies only between MPC iterations but remains constant along the horizon.

In this thesis, we extend the formulation by considering $\gamma_k = [\gamma_0, \gamma_1, \dots, \gamma_{N-1}]^\top$ as a vector of N decision variables, allowing the optimizer to choose a different decay rate for each prediction step i . This approach enables more flexible and potentially less conservative safety enforcement.

Modification of optimization variables and initialization: The optimization variable vector $\mathbf{z}$, indicated in this formulation although it is implemented as `OPT_VARS` and assigned to the NLP input

field `nlp["x"]` in the CasADi-based code., is augmented to include the decay rates:

$$\mathbf{z} = \begin{bmatrix} \mathbf{u}_{0|k} \\ \vdots \\ \mathbf{u}_{N-1|k} \\ \mathbf{x}_{0|k} \\ \vdots \\ \mathbf{x}_{N|k} \\ \gamma_{0|k} \\ \vdots \\ \gamma_{N-1|k} \end{bmatrix} \in \mathbb{R}^{6N+13(N+1)+N}$$

where:

- $\mathbf{u}_{i|k} \in \mathbb{R}^6$ are the joint velocity commands at step i , for $i = 0, \dots, N-1$;
- $\mathbf{x}_{i|k} \in \mathbb{R}^{13}$ are the predicted states at step i , including position, orientation, and joint angles;
- $\gamma_{i|k} \in \mathbb{R}$ are the CBF decay rates at each stage of the prediction horizon.

This structure leads to a total of $6N + 13(N + 1) + N = 20N + 13$ decision variables. The warm start initialisation $\mathbf{z}^{(0)}$ includes a small positive initial assumption for each γ_i , set to a value of 0.001 to improve convergence and solver feasibility following the guidelines provided in [2].

To enforce safety within the MPC framework, two additional sets of constraints are introduced: bounds on the CBF decay rates $\gamma_{i|k}$, and the discrete-time Control Barrier Function Safety Constraint (CBFSC) that ensures the end-effector remains at a safe distance from the moving obstacle.

Decay rate bounds: The decay rates $\gamma_{i|k}$ must satisfy $\epsilon \leq \gamma_{i|k} \leq 1$, where $\epsilon = 10^{-6}$ is a small positive constant that ensures $\gamma_{i|k}$ remains strictly positive while not exceeding 1, thus preserving the theoretical guarantees of the CBF framework. These constraints are implemented explicitly at each horizon step.

Discrete-time CBF safety constraint: To guarantee safe separation between the end-effector and the moving obstacle (3.32), we impose a discrete-time CBF condition. Define:

$$\begin{cases} H_{i|k} = \|\mathbf{p}_{i|k} - \hat{\mathbf{o}}_{i|k}\| - r_{\text{safe}} \\ H_{i+1|k} = \|\mathbf{p}_{i+1|k} - \hat{\mathbf{o}}_{i+1|k}\| - r_{\text{safe}} \end{cases} \quad (3.34)$$

where $\hat{\mathbf{o}}_{i|k}$ and $\hat{\mathbf{o}}_{i+1|k}$ are the predicted obstacle positions, and r_{safe} is defined in before (3.31).

The safety is then enforced by imposing the discrete-time CBF condition defined in Equation (1.3):

$$H_{i+1|k} - (1 - \gamma_{i|k}) H_{i|k} \geq 0 \quad (3.35)$$

These constraints ensure that the safety margin does not shrink too abruptly over the horizon and that the end-effector maintains a dynamically appropriate clearance from the obstacle.

MPC formulation with CBF-based obstacle avoidance: To extend the obstacle-free MPC formulation to account for dynamic obstacle avoidance, the optimization problem is augmented with additional

decision variables and constraints derived from the Control Barrier Function Safety Constraint (CBFSC) mechanism. At each control cycle k , the controller solves a non-linear optimization problem to compute the optimal joint velocity inputs $\{\mathbf{u}_{0|k}, \dots, \mathbf{u}_{N-1|k}\}$ and the optimal decay rates $\{\gamma_{0|k}, \dots, \gamma_{N-1|k}\}$. Equation (3.33) shows the objective function, which adds a penalty on the decay rates $\gamma_{i|k}$ to balance safety and tracking. The modified optimization problem is formulated as:

$$\begin{aligned}
 \min_{\substack{\mathbf{u}_{0|k}, \dots, \mathbf{u}_{N-1|k} \\ \gamma_{0|k}, \dots, \gamma_{N-1|k}}} \quad & J(k) = \sum_{i=0}^{N-1} \left(\mathbf{e}_{i|k}^\top \mathbf{Q} \mathbf{e}_{i|k} + \mathbf{u}_{i|k}^\top \mathbf{R} \mathbf{u}_{i|k} + P_\gamma \gamma_{i|k}^2 \right) \\
 \text{subject to} \quad & \mathbf{p}_{i+1|k} = \mathbf{p}_{i|k} + T_s \mathbf{J}_p(\boldsymbol{\theta}_k) \mathbf{u}_{i|k} \\
 & \mathbf{q}_{i+1|k} = \frac{\mathbf{q}_{i+1|k}^{\text{raw}}}{\sqrt{\|\mathbf{q}_{i+1|k}^{\text{raw}}\|^2 + \epsilon}} \\
 & \boldsymbol{\theta}_{i+1|k} = \boldsymbol{\theta}_{i|k} + T_s \mathbf{u}_{i|k} \\
 & \mathbf{x}_{0|k} = \hat{\mathbf{x}}_k \\
 & \theta_{\min}^{(j)} \leq \theta_{i+1|k}^{(j)} \leq \theta_{\max}^{(j)} \\
 & -u_{\max}^{(j)} \leq u_{i|k}^{(j)} \leq u_{\max}^{(j)} \\
 & 0 < \gamma_{i|k} \leq 1 \\
 & H_{i+1|k} - (1 - \gamma_{i|k}) H_{i|k} \geq 0 \\
 & \text{for } j = 1, \dots, 6; \quad i = 0, \dots, N-1
 \end{aligned} \tag{3.36}$$

Here:

- $\gamma_{i|k} \in \mathbb{R}$ is the dynamically optimized CBF decay rate;
- $P_\gamma \in \mathbb{R}$ is a scalar weight that balances safety enforcement;
- $H_{i|k}$ and $H_{i+1|k}$ are defined in Equations (3.34) as the signed distance to the obstacle;
- All other variables retain the same interpretation as in the obstacle-free case, see Equation (3.29).

3.4.3 Overview of the MPC software implementation

The following is a high-level pseudocode outlining the main steps of the MPC controller software, including initialization, obstacle state estimation, non-linear optimization, and control loop execution.

1. SYSTEM INITIALIZATION

- 1.1. Initialize ROS2 and create main node.
- 1.2. Create publisher for sending joint velocities.
- 1.3. Create MoveIt interface for the manipulator.
- 1.4. Obtain initial robot state (position, orientation, joints).
- 1.5. Load reference end-effector trajectory from file.
- 1.6. Prepare variables and parameters for joint and velocity limits.

2. OBSTACLE OBSERVER INITIALIZATION

- 2.1. Define variables for estimated obstacle state (position,

- velocity, acceleration).
- 2.2. Define observer parameters (gains, timestep).
- 2.3. Create subscriber to receive obstacle position from ROS2.
- 2.4. Protect obstacle data access with mutex.
- 3. OPTIMIZATION PROBLEM DEFINITION (MPC)
 - 3.1. Define CasADi symbolic variables for:
 - Predicted states and controls over the horizon.
 - Jacobians.
 - Reference trajectory.
 - Predicted obstacle trajectory.
 - CBF parameters (γ).
 - 3.2. Define cost function (tracking + control penalty + CBF penalty).
 - 3.3. Define constraints:
 - Robot dynamics (position, orientation, joints).
 - Joint and velocity limits.
 - CBF safety constraints with respect to obstacle.
 - Constraints on γ .
 - 3.4. Build NLP dictionary for CasADi and create IPOPT solver.
- 4. MAIN MPC LOOP (for each timestep t)
 - 4.1. If not the first cycle:
 - Apply previously computed control command.
 - Update current robot state (pose and joints).
 - Wait for sampling time to elapse.
 - 4.2. Compute current Jacobian using MoveIt.
 - 4.3. Extract and prepare reference trajectory for the horizon.
 - 4.4. OBSTACLE OBSERVER:
 - If new measurement available, update observer.
 - Otherwise, use last estimate or fallback.
 - Predict obstacle trajectory for the entire horizon.
 - 4.5. Prepare parameters for the solver (current state, Jacobians, reference, obstacle trajectory).
 - 4.6. Solve NLP problem with warm-start from previous solution.
 - If successful: extract and save new control command.
 - If failure: publish zero velocities and terminate.
 - 4.7. Log debug and state information.
- 5. TERMINATION
 - 5.1. At the end of the timesteps, publish zero velocities to stop the robot.
 - 5.2. Close ROS2 threads and terminate program.

For the complete implementation, refer to the C++ source code in Appendix 8.

System architecture and implementation

This chapter describes the overall architecture of the implemented robotic control system. It covers the main software components, their interactions, and the communication framework used to integrate the MPC controller, obstacle observer, and robot interface within the ROS2 ecosystem.

The goal is to provide a clear understanding of how the various modules collaborate to achieve real-time motion planning and obstacle avoidance for the Niryo Ned2 manipulator.

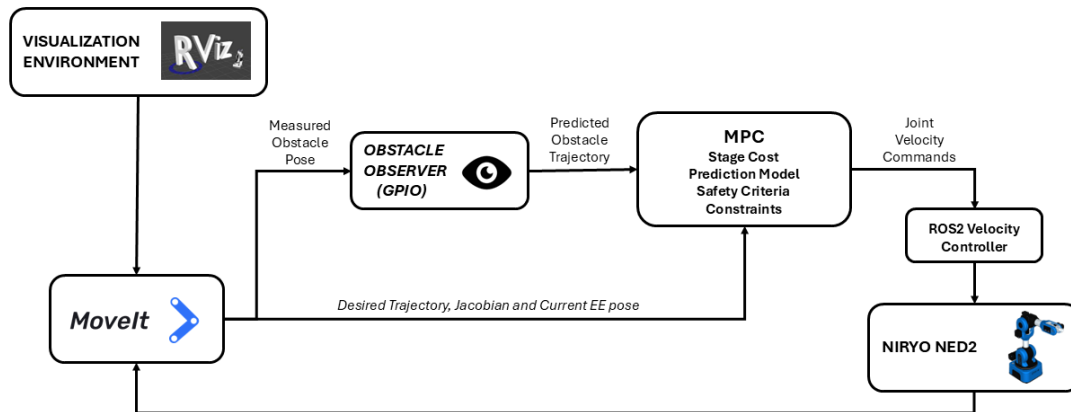


Figure 4.1: Block diagram of the system architecture showing data flow between the modules.

4.1 System overview

The high-level architecture of the developed MPC system for obstacle avoidance, designed for the Niryo Ned2 manipulator, consists of six main modules, each handling a specific function within the control pipeline (Fig. 4.1):

- **Rviz2:** This module is responsible for the real-time visualization of the robot, environment, and

dynamic obstacles. It provides an intuitive interface for monitoring the execution of the control system and the simulated scene.

- **MoveIt2:** This module is responsible for several key operations within the control system. In addition to generating the end effector reference trajectory using the CHOMP planner, it is used to retrieve the current state of the robot (in terms of pose and joint configuration) and compute the geometric Jacobian. It is also used to update the pose of a dynamic obstacle within the simulated scene. A dedicated ROS2 node computes the motion of the obstacle and publishes its current pose to a specific topic for the observer module.
- **Obstacle observer (GPIO):** This module receives the current measured pose of the obstacle and estimates its velocity and acceleration. The estimated state is then used to compute the predicted positions of the obstacle over the MPC horizon.
- **MPC controller:** Calculates optimal joint velocities based on the desired trajectory, the current state of the robot, and the predicted obstacle motion. Includes phase cost, prediction model, safety constraints (CBF), and constraints on joint angular position and joint velocity.
- **ROS2 velocity controller:** Receives the joint velocity commands computed by the MPC and forwards them to the robot actuators through ROS2 control infrastructure. The velocity commands computed by the MPC module at each control cycle are applied to the simulated Niryo Ned2 manipulator through the `ros2_control` framework. In particular, a `JointGroupVelocityController` was employed, allowing direct control of joint velocities via ROS2 topic communication.

This controller is properly configured to interact with a simulated hardware interface, provided by the `mock_components/GenericSystem` plugin. To enable a physically consistent response to velocity commands within the simulation, the file `niryo_ned2.ros2_control.xacro` was modified accordingly. The plug-in was configured to compute approximate joint dynamics and to use velocity as the active control interface. This was achieved by setting the `calculate_dynamics` parameter to true and by specifying `custom_interface_with_following_offset` as `velocity`. Additionally, each joint in the robot description was extended with an explicit velocity command interface.

The controller itself was instantiated and parametrized via the `ros2_controllers.yaml` configuration file. All six joints of the manipulator were associated with both position and velocity state interfaces, and the velocity command interface was selected as the input control mode.

At runtime, the ROS2 node responsible for the MPC publishes a vector of joint velocities on the topic `/niryo_velocity_controller/commands`. This topic is monitored by the `JointGroupVelocityController`, which applies the received commands to the simulated robot joints. This configuration enables the closed-loop execution of velocity-based control strategies in a purely simulated environment.

In summary, the MPC generates the optimal velocity control inputs, which are transmitted to the robot through a velocity controller specifically configured to operate on a simulated hardware abstraction. This setup ensures realistic actuator behaviour during the validation of motion planning and obstacle avoidance strategies.

- **Niryo Ned2 (simulated):** The robot simulated through the `mock_components/GenericSystem` interface, which receives the commanded joint velocities and provides the current joint and end-effector states for feedback and planning purposes.

4.2 Launch file and ROS2 nodes

The orchestration of the control system is handled through a custom Python-based launch file that initializes all the required ROS2 nodes and their configurations. This centralized setup ensures the correct initialization order and timing, especially important when working with simulated hardware and time-sensitive components such as the MPC controller.

The launch file includes the following nodes and actions:

- **RViz2:** Provides a 3D visualization interface for the robot, planning scene, and dynamic elements. It is initialized with a preconfigured RViz layout.
- **Static transform publisher:** Publishes a fixed transform between the `world` and `base_link` frames, which serves as the base reference for the robot and obstacle coordinates.
- **Robot state publisher:** Publishes the current state (positions and velocities) of all joints based on the robot's URDF model. This is essential for visualization and internal state consistency.
- **Move group node:** The main node of MoveIt2, responsible for motion planning, kinematic computations, and interaction with the planning scene. In this setup, the CHOMP planner is used as the default pipeline.
- **ros2_control node:** Initializes the controller manager with the simulated hardware configuration defined in the `ros2_controllers.yaml` file. It loads the velocity-based controller that receives commands from the MPC.
- **Joint state broadcaster:** Broadcasts the state (position and velocity) of each joint to ROS2 topics, enabling real-time introspection by other nodes or GUI tools.
- **JointGroupVelocityController:** A velocity-based controller configured to accept velocity commands for all six joints of the Niryo Ned2. It is responsible for forwarding the velocity commands to the simulated hardware interface.
- **Dynamic scene setup node:** A custom ROS2 node that adds a ground plane for the robot and places a dynamic obstacle in the simulated scene. It periodically updates the position of the obstacle and publishes its current pose to the `/obstacle_pose` topic, which is then used by the observer module for trajectory prediction.
- **MPC controller node:** Executes the custom Model Predictive Control algorithm developed for obstacle avoidance. It publishes joint velocity commands to the velocity controller.

To set up the entire simulation environment and control system, the main launch file is utilized:

```
ros2 launch niryo_moveit2_config dynamic_setup.launch.py
```

The MPC node is deliberately started with a delay to ensure that all relevant components (e.g., MoveIt2, robot state publishers, and velocity controllers) are properly initialized.

In order to construct a meaningful simulation scenario where the manipulator is required to perform obstacle avoidance, it was necessary to synchronize the motion of the robot with that of the dynamic obstacle. Specifically, the obstacle was programmed to follow a predefined trajectory that intersects the robot's desired path, resulting in a potential collision in the absence of any safety constraints.

This synchronization was achieved by tuning the delay used to launch the obstacle node (`dynamic_scene_setup`). The delay was adjusted according to the obstacle velocity v_y , which is configurable in the obstacle's C++ source file (see Equation (3.2)). This setup allowed for validation of the MPC-based safety mechanism under realistic dynamic conditions.

It is worth noting that the node responsible for generating the reference trajectory of the end-effector was launched separately using a simplified launch file named `ref_traj_launch.py`. This decision was made to decouple the trajectory planning logic from the main control system and facilitate independent execution. Details on the structure and implementation of this node are discussed in Section 3.3.2.

4.3 Implementation considerations

The development of the proposed control architecture required several practical and architectural decisions to ensure compatibility with the available hardware, efficient execution of time-critical components, and long-term maintainability. These considerations influenced the choice of development platform, the programming language used for control logic, and the overall software modularity.

This section outlines the main implementation aspects that shaped the system design. First, it discusses the challenges and motivations behind using a WSL 2-based development environment. Then, it presents the real-time constraints imposed by the MPC formulation and the solutions adopted to meet performance requirements. Finally, it highlights the modular structure of the software architecture, which enables flexibility, scalability, and ease of integration with future components.

4.3.1 Development environment and platform constraints

The project was developed entirely within a WSL 2 (Windows Subsystem for Linux) environment, using Ubuntu 22.04 as the base distribution and ROS 2 Humble Hawksbill as the robotics middleware. This setup enabled seamless integration between Windows and Linux tools while maintaining access to ROS utilities through a native Linux terminal.

However, the use of WSL 2 introduced some technical limitations, such as the lack of full support for USB passthrough and the inability to access real-time scheduling features. To overcome these constraints, a fully software-based simulation was adopted, avoiding the need to interface with physical hardware.

The entire system is designed to be modular and decoupled, allowing computationally intensive nodes (such as the MPC controller) and graphical components (like RViz2) to operate in coordination via the DDS middleware, without relying on the host operating system. This architecture also ensures that the system remains easily portable to native ROS 2 installations on Linux or other real-time-compatible environments.

4.3.2 Real-time performance

The predictive nature of MPC imposes tight timing constraints, as each control cycle must complete within a predefined interval to maintain reactivity. Initially, the MPC controller was designed to operate at a frequency of 25 Hz (i.e., with a control cycle of 40 ms), a common value in many real-time robotic applications. However, experimental tests revealed that most cycles systematically exceeded the 40 ms threshold.

This delay was caused by the cumulative time required for multiple operations: computing the geometric Jacobian (performed once at the beginning of each control cycle and reused across the prediction

horizon), retrieving the current end-effector pose and joints configuration from MoveIt 2, and solving the nonlinear optimization problem using CasADi and IPOPT.

Due to these bottlenecks and the limited performance of the development hardware (an HP Pavilion laptop equipped with an Intel Core i5-8250U processor and 8 GB of RAM) the system was unable to maintain real-time execution at 25 Hz. As a result, the control frequency was reduced to 10 Hz (i.e., 100 ms per cycle), which offered a better balance between performance and stability.

Even at this reduced rate, some control cycles occasionally exceeded the 100 ms deadline, confirming the computational intensity of MPC within a ROS 2 framework.

To meet these requirements, the MPC controller was implemented entirely in C++ using CasADi for symbolic modeling and IPOPT as the nonlinear solver.

The choice of C++ was motivated by its superior runtime efficiency and finer control over system resources compared to interpreted languages such as Python. This was crucial to meet the stringent real-time constraints of the MPC controller. Moreover, C++ offers seamless integration with native ROS 2 libraries and enables low-latency communication, making it the most suitable language for this application.

This compiled implementation provides superior runtime performance and direct integration with native ROS 2 data structures. Although implemented as a standalone ROS 2 node, the controller integrates seamlessly with the `ros2_control` framework by sending velocity commands to the `JointGroupVelocityController`, enabling low-latency communication with the simulated hardware interface.

4.3.3 Modular design

From the early stages of development, the system architecture was designed with modularity in mind. The main components (reference trajectory generation, dynamic obstacle estimation, and the MPC controller) are organized as independent or loosely coupled modules, each with clearly defined responsibilities.

The reference trajectory is generated by a standalone ROS 2 node, which is launched separately through a dedicated launch file. This node could be easily replaced by an alternative source, such as a GUI interface or an external planner, without requiring changes to the rest of the pipeline.

In contrast, the obstacle observer is implemented as an internal module within the MPC controller node. Unlike some literature approaches that embed the obstacle prediction model directly within the nonlinear optimization problem, this implementation decouples the estimation process from the solver. At each control cycle, the observer predicts the future trajectory of the obstacle and passes it to the MPC as an input parameter. This separation allows for independent tuning and logging of the estimation process, while maintaining compatibility with the overall control loop.

Finally, the use of parameterized URDF/Xacro files enables adaptation of the system to different fixed-base manipulators beyond the Niryo Ned2. This abstraction ensures that the software remains reusable and scalable for future experiments or alternative robotic platforms.

Numerical simulations

This chapter presents a set of numerical simulations aimed at evaluating the performance of the proposed MPC-based control architecture. Two scenarios are considered: (i) an ideal environment without obstacles, used to validate trajectory tracking accuracy; and (ii) a dynamic environment featuring a moving obstacle, used to assess the effectiveness of the safety mechanisms based on CBFs and the observer module. To analyse the behaviour of the system and extract relevant performance metrics, MATLAB was used as the primary post-processing and plotting tool. Specifically, the full simulation logs (approximately 8,500 lines for the trajectory tracking scenario, and about 8,700 lines for the dynamic obstacle avoidance test) were exported as plain text `.txt` files. This approach avoided manual data extraction and enabled fully automated parsing and analysis within MATLAB. A dedicated script was developed to import the log files, extract the relevant quantities, and generate the plots shown throughout this chapter. This post-processing workflow ensured accuracy, consistency, and reproducibility in the evaluation of all simulation results.

Across all simulations, certain parameters were kept fixed to ensure comparability. In particular, the MPC sampling period was set to $\Delta t = 0.1$ s, corresponding to a control frequency of 10 Hz. Each simulation was run for 100 timesteps, resulting in a total duration of 10 s. The reference trajectory, by contrast, spans only 31 time steps (i.e., 3.1 s); its final pose is held constant for the remainder of the simulation. It is important to note that the same reference trajectory (described in Section 3.3.2) is used across all simulations. This setup allows the controller’s behaviour to be evaluated both during active tracking and in steady-state conditions.

Table 5.1: Fixed parameters used in all MPC simulations

Parameter	Value
Sampling time (T_s)	0.1 s (10 Hz)
Simulation duration	100 timesteps (10 s)
Reference trajectory	Same in all tests: 31 timesteps (3.1 s), final pose held constant

Other key parameters, such as the prediction horizon N and the weighting matrices used in the MPC objective function, were subject to tuning. Their specific values are reported and discussed in the relevant sections of this chapter.

5.1 Performance metrics

All plots shown in this chapter span the full 10-second simulation window. A vertical dashed line is drawn at $t = 3.1$ s to indicate the end of the reference trajectory, at which point the robot is expected to have completed the planned motion and to remain at the final pose thereafter. To ensure a consistent evaluation framework across all MPC simulations, a fixed set of plots is generated for each test scenario. These plots are designed to highlight both tracking accuracy and control effort, and are used systematically in the following sections. Below is an overview of the most relevant graphical analyses.

End-effector position and orientation tracking: These plots illustrate the tracking behaviour of the end-effector concerning the reference trajectory in both position and orientation domains. The position tracking plot depicts the component-wise evolution of the Cartesian position (X, Y, Z), where each axis shows the desired trajectory and the corresponding actual path followed by the controller. This enables a clear assessment of how accurately the system follows the spatial trajectory and allows detection of transient deviations or steady-state offsets. The orientation tracking plot focuses on the quaternion representation of the end-effector orientation. Each of the four components (w, x, y, z) is plotted over time, comparing the desired quaternion with the measured one. While quaternion components lack immediate geometric interpretability, their analysis helps verify convergence trends and detect potential discontinuities due to sign flips or misalignment during motion. Overall, these two plots provide a comprehensive view of the controller's tracking performance across both translational and rotational dimensions.

End-effector position error: This plot shows the tracking error in each Cartesian component (X, Y, Z) of the end-effector position. Unlike a scalar norm, this component-wise view helps identify in which direction the tracking deviation is more pronounced and how it evolves during the motion.

Orientation error analysis: As introduced in Section 3.3.3, the orientation error is computed using quaternion algebra by taking the relative rotation between the predicted and reference orientation. The resulting quaternion error $\mathbf{q}_{\text{err},i}$ can be written in the form:

$$\mathbf{q}_{\text{err},i} = \begin{bmatrix} \sin(\theta/2) \mathbf{u} \\ \cos(\theta/2) \end{bmatrix}$$

where θ is the rotation angle required to align the predicted frame to the reference, and $\mathbf{u}$ is the axis of rotation.

To extract θ and $\mathbf{u}$ from the error quaternion in MATLAB, the following formulas were applied:

$$\theta = 2 \cdot \text{atan2}(\|\text{vec}(\mathbf{q}_{\text{err}})\|, \mathbf{q}_{\text{err}}^w), \quad \mathbf{u} = \frac{\text{vec}(\mathbf{q}_{\text{err}})}{\|\text{vec}(\mathbf{q}_{\text{err}})\|}$$

where $\text{vec}(\mathbf{q}_{\text{err}})$ denotes the vector part (x, y, z) and $\mathbf{q}_{\text{err}}^w$ is the scalar part.

This decomposition allows for separate visualization of:

- The magnitude of the orientation error (θ);
- The direction of the error axis ($\mathbf{u}$);
- Their combination as a 3D orientation error vector ($\theta \cdot \mathbf{u}$), which corresponds to the quantity minimized in the MPC cost function.

Additionally, the algebraic component-wise difference between the reference and measured quaternions was plotted. While this provides a direct comparison, it should be interpreted cautiously because component-wise subtraction of quaternions lacks direct geometric meaning due to their double-cover property and inherent ambiguities.

3D orientation error visualization: This plot depicts the robot's end-effector trajectory in 3D space, overlaid with red vectors sampled along the path. Each vector represents the orientation error at that instant, with its direction indicating the axis about which a rotation is needed to align the current orientation with the reference, and its length proportional to the rotation angle (the magnitude of the error). This visualization provides an intuitive geometric insight into how the orientation discrepancy evolves spatially during task execution.

Control actions: This plot shows the time evolution of the control commands $\mathbf{u}_k$ generated by the MPC at each time step, corresponding to the joint velocity inputs applied to the robot. Each curve represents one of the six joints of the manipulator, allowing assessment of how the controller distributes motion effort across the arm. The plot helps evaluate the smoothness and aggressiveness of the control strategy. Peaks in the velocity profiles can indicate periods of fast motion required to reduce large tracking errors or to react to dynamic obstacles (in scenarios where obstacle avoidance is active). Although joint velocity limits were enforced within the MPC formulation, visualizing the actual control signals over time supports verification of constraint adherence and provides insights into control saturation or redundancy.

Additional plots related to obstacle avoidance and safety constraints are presented only in the simulations involving dynamic obstacles and will be introduced in the corresponding section.

5.2 Trajectory tracking in obstacle-free environments

This section validates the performance of the implemented MPC controller in a simplified environment, free of dynamic obstacles. The primary goal is to verify the basic functionality of the tracking logic before introducing more complex elements such as obstacle avoidance and safety constraints.

5.2.1 Parameter tuning methodology

The tuning process focused on identifying a set of controller parameters that provided a reasonable compromise between tracking performance and real-time feasibility. The following parameters were considered:

- the prediction horizon N ,
- the control effort penalty matrix $\mathbf{R} \in \mathbb{R}^{6 \times 6}$,
- the state error penalty matrix $\mathbf{Q} \in \mathbb{R}^{6 \times 6}$, composed of:
 - $\mathbf{Q}_{\text{pos}} \in \mathbb{R}^{3 \times 3}$: weights for position tracking error,
 - $\mathbf{Q}_{\text{orient}} \in \mathbb{R}^{3 \times 3}$: weights for orientation tracking error.

The tuning strategy involved adjusting one parameter at a time while keeping the others fixed, in order to isolate and evaluate its impact on system behaviour.

Prediction horizon N : Increasing the prediction horizon improves the controller’s ability to anticipate future deviations, potentially leading to more accurate tracking. However, this comes at the cost of increased computational effort per iteration. Experimental results showed that for $N > 5$, the average computation time per control cycle exceeded 100 ms, preventing the controller from maintaining a 10 Hz update frequency. On the other hand, setting $N < 5$ resulted in degraded tracking accuracy or even failure to complete the task. Therefore, $N = 5$ was selected as the minimum value that ensured acceptable performance without violating real-time constraints.

Control penalty matrix $\mathbf{R}$: This matrix penalizes the magnitude of the control inputs. Higher values result in smoother, less aggressive motions, while smaller values yield faster but potentially more oscillatory responses. It was observed that with $\mathbf{R} = 50$, the system failed to complete the reference trajectory within the intended 3.1 s and continued to deviate even after 10 s, indicating excessive penalization of the control effort. As a compromise, $\mathbf{R} = 2 \cdot \mathbf{I}_6$ was chosen, which provided satisfactory tracking without excessive delays or instability.

State penalty matrix $\mathbf{Q}$: This matrix determines the relative importance of state errors in the cost function. Greater weight was assigned to position accuracy than to orientation, since preliminary experiments revealed that increasing $\mathbf{Q}_{\text{orient}}$ beyond a certain threshold led to a deterioration in position tracking. The final values selected were:

$$\mathbf{Q}_{\text{pos}} = 2000 \cdot \mathbf{I}_3, \quad \mathbf{Q}_{\text{orient}} = 50 \cdot \mathbf{I}_3.$$

These values represent the best trade-off identified during the tuning phase. Although further fine-tuning could potentially yield slight improvements in performance, the expected benefit did not justify the additional effort, especially considering the subsequent development steps involving dynamic obstacle management, obstacle state estimation, and CBF-based safety constraints. Therefore, the tuning phase was concluded upon reaching this configuration, which already demonstrated that the implemented MPC was effective and stable in nominal conditions.

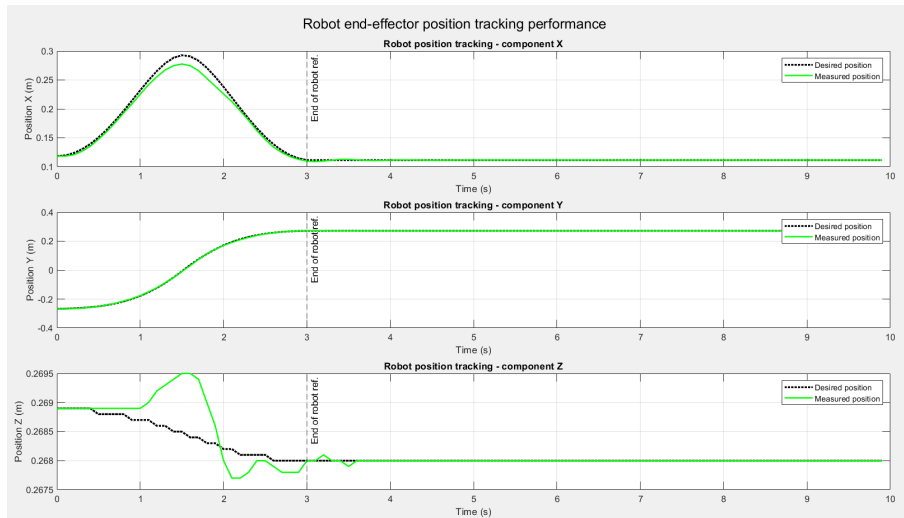


Figure 5.1: Tracking of the end-effector position with respect to the reference trajectory.

5.2.2 Results with the selected configuration

The plots presented in the following subsections refer to the configuration:

$$N = 5, \quad \mathbf{R} = 2 \cdot \mathbf{I}_6, \quad \mathbf{Q}_{\text{pos}} = 2000 \cdot \mathbf{I}_3, \quad \mathbf{Q}_{\text{orient}} = 50 \cdot \mathbf{I}_3.$$

The tracking performance achieved with the selected cost weights can be evaluated through the evolution of the end-effector's position and orientation over time (Figs. 5.1 and 5.2).

The comparison between the desired and measured positions along the three Cartesian axes reveals near-perfect tracking along the y -axis, as the reference and actual trajectories remain closely aligned. In contrast, the x - and z -components show more noticeable discrepancies, with the measured position lagging slightly behind the reference at specific intervals. These deviations reflect the increased emphasis on orientation accuracy in the cost function, where the weight $\mathbf{Q}_{\text{orient}} = 50 \cdot \mathbf{I}_3$ was selected to enhance alignment performance, potentially at the expense of positional precision in non-dominant directions.

Regarding orientation, the time evolution of the quaternion components demonstrates rapid convergence to the desired values, confirming effective regulation of the end-effector's attitude (Fig. 5.2). All components reach their target within the first 3 seconds of simulation, well before the end of the trajectory, indicating that the chosen orientation weight is adequate for ensuring accurate tracking.

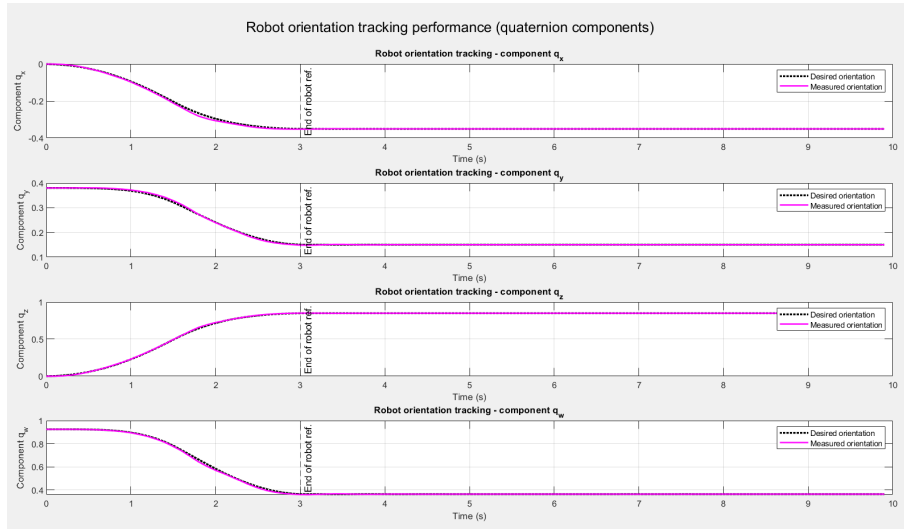


Figure 5.2: End-effector orientation tracking: quaternion components.

To better quantify these tracking results and validate the qualitative observations discussed earlier, the corresponding tracking errors are reported for both position and orientation (Figs. 5.3 and 5.4). This analysis allows for a more precise assessment of convergence time and peak deviations, occasionally correcting initial impressions—for instance, the tracking performance along the y -axis, initially deemed superior, is revealed to be comparable to or slightly worse than that of the z -axis.

The Cartesian position error along the three spatial directions exhibits distinct patterns (Fig. 5.3). The most significant deviation is observed along the x -axis, with a peak error of approximately 1.8 cm, which gradually decays to zero within the first 5 seconds. The y -axis presents a smaller peak (below 5 mm), showing a similar convergence trend. The z -axis demonstrates the best performance, with a maximum deviation slightly exceeding 1 mm, eliminated within 4 seconds. These results confirm the controller's effectiveness in ensuring accurate and stable convergence, even in the presence of moderate transient discrepancies.

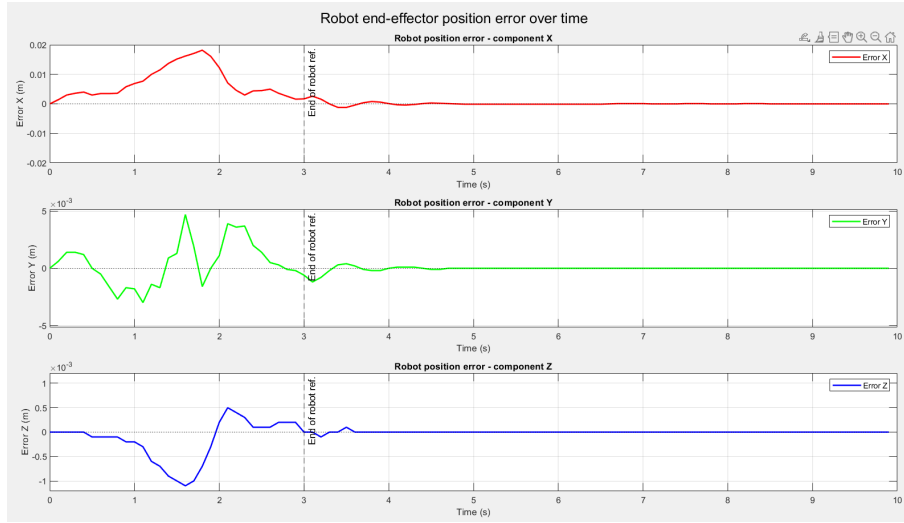


Figure 5.3: End-effector position error over time for each Cartesian component.

The orientation error is evaluated as the algebraic difference between the reference quaternion and the aligned measured quaternion (Fig. 5.4). The maximum deviations across components are approximately 0.015 for q_x , -0.006 for q_y , -0.014 for q_z , and slightly above 0.02 for q_w . All components exhibit smooth convergence, stabilizing within the first 5 seconds of simulation. These results confirm the effectiveness of the selected orientation cost weight in ensuring rapid and accurate alignment of the end-effector pose.

A spatial depiction of the end-effector trajectory and orientation accuracy is provided in the 3D visualization (Fig. 5.5). The reference path is represented by a black dotted line, while the executed trajectory appears in green. Notable discrepancies can be observed, particularly in the central portion of the motion, where the executed path tends to cut inside the intended curve, following a slightly tighter profile. This behaviour is likely attributable to the prioritization of orientation control over position tracking in the cost design.

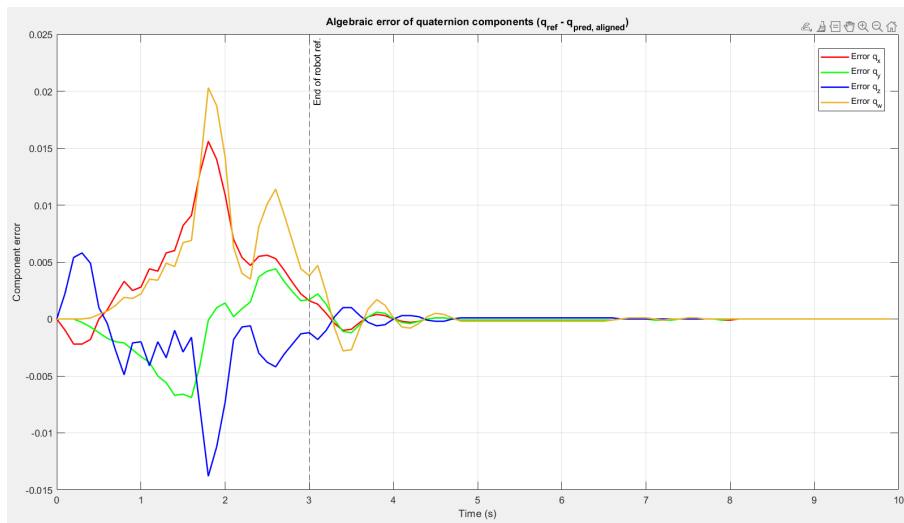


Figure 5.4: Algebraic tracking error of the quaternion components.

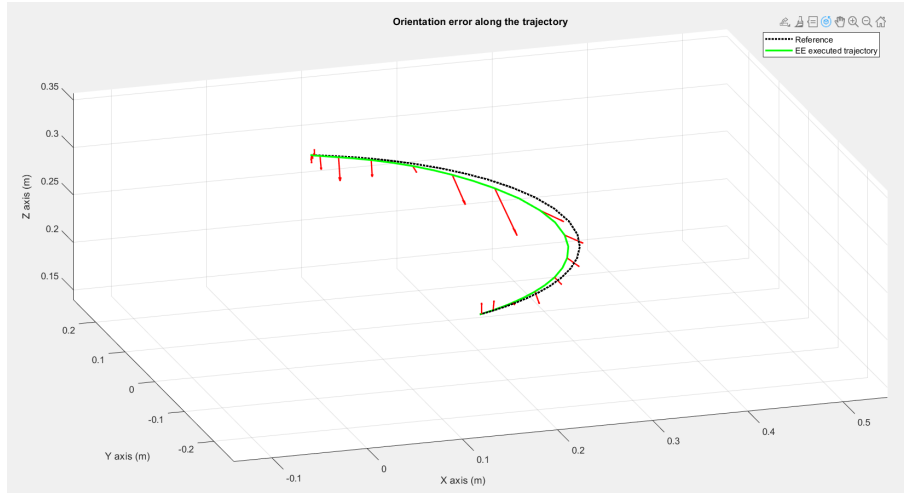


Figure 5.5: 3D visualization of the robot trajectory with orientation error vectors overlaid.

The red vectors represent the orientation error in axis-angle form, where the direction encodes the required rotation axis and the magnitude corresponds to the rotation angle, as already explained in Section 5.1. Notably, the axis is not fixed but varies along the trajectory, reflecting the changing rotational misalignment over time. The vectors grow in length during the central phase (where the trajectory deviation is more pronounced) and shrink toward the end, indicating improved alignment.

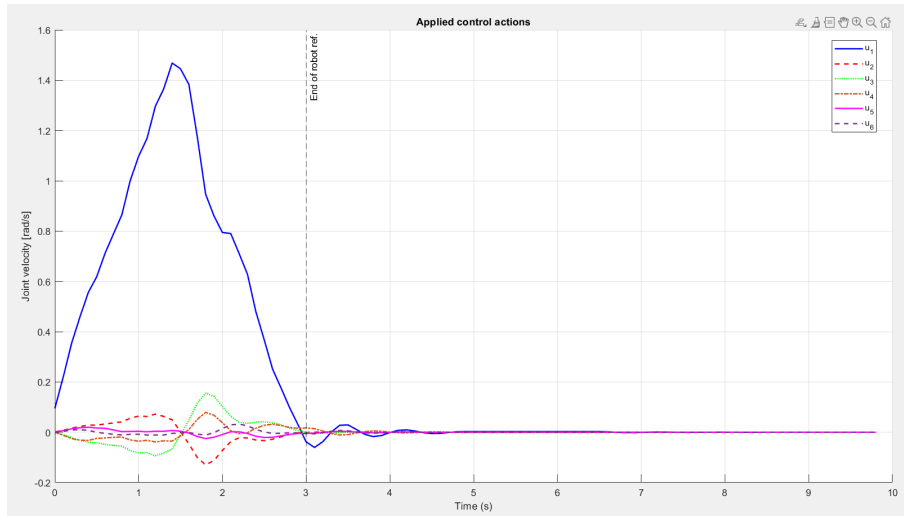


Figure 5.6: Control inputs: joint velocity commands generated by the MPC over time.

The joint velocity commands $\mathbf{u}_k$ generated by the MPC controller during the simulation exhibit distinct activation patterns across the six joints of the Niryo Ned2 manipulator (Fig. 5.6). The first joint, u_1 , is the most active, reaching a peak velocity of approximately 1.47 rad/s in the initial phase of motion. In contrast, the remaining joints operate within significantly lower ranges, typically below 0.2 rad/s. Notably, joints u_5 and u_6 show minimal activation, consistently remaining under 0.03 rad/s throughout the entire task. All control signals converge to zero within the first 5 seconds, confirming that the controller successfully stabilizes the end-effector once the reference trajectory has been completed. The smoothness and bounded amplitude of the control actions also indicate a well-conditioned optimization process and appropriate tuning of the cost function weights.

5.3 Obstacle avoidance in dynamic environments

After verifying the correct operation of the trajectory tracking MPC controller in an ideal, obstacle-free environment, the next step involves validating the complete control architecture in the presence of dynamic obstacles.

Before introducing safety constraints into the optimization problem, the performance of the internal observer module was evaluated in isolation. This preliminary step ensured that the obstacle’s position and predicted trajectory could be reliably estimated before being used in the CBF constraint. Although implemented directly within the MPC node, the observer operates independently from the solver: at every control cycle, it estimates the current state of the obstacle and forecasts its future positions over the prediction horizon. These predictions are then passed as external inputs to the optimization routine. The set of observer gains α_1 , α_2 , and α_3 used for this evaluation was introduced in Section 3.4.1.

To provide context for the simulations, this section begins with a recap of the initial scenario setup, including a visualization of the environment in RViz to clarify the spatial configuration of the robot, the obstacle, and the reference trajectory.

5.3.1 Simulation setup

The simulation environment used to evaluate the obstacle avoidance capabilities of the proposed controller was previously introduced in Chapter 4.2, where the launch structure and node synchronization logic were described. In particular, it was explained that the MPC node is intentionally launched with a delay to ensure that all essential ROS components are fully initialized.

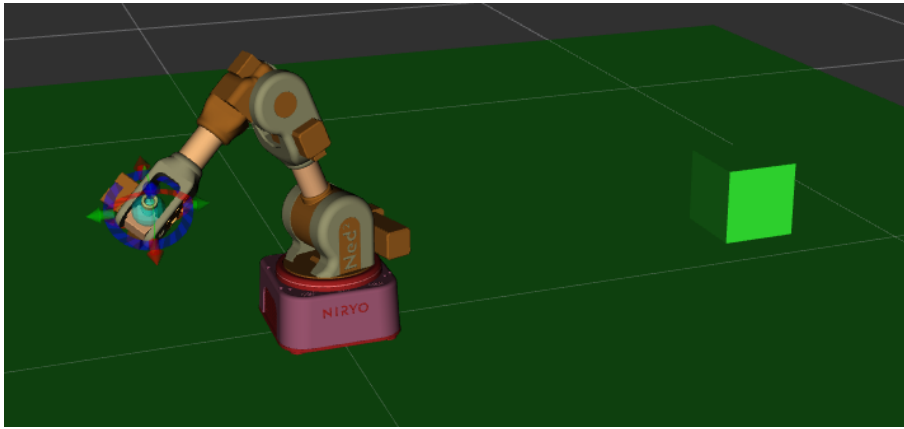


Figure 5.7: Initial environment setup in RViz: robot manipulator, dynamic obstacle, and ground plane.

Furthermore, to induce a potential collision scenario, the motion of the dynamic obstacle was deliberately synchronized with the robot’s reference trajectory. This was achieved by tuning the delay before launching the obstacle node (`dynamic_scene_setup`), based on its velocity, as specified in the corresponding C++ implementation. The initial configuration of the environment in RViz including the robot, the ground plane, and the obstacle. The robot’s initial joint configuration (Fig. 5.7) is defined in the file `initial_positions.yaml`, originally generated by the MoveIt Setup Assistant and subsequently modified to meet the requirements of this experiment.

The obstacle motion was constrained to a one-dimensional trajectory along the y -axis, as previously discussed in Chapter 4.2. For the experiments presented in this chapter, the obstacle moved at a constant velocity of $v_y = -0.065$ m/s, resulting in periodic back-and-forth motion between points A and B. This

specific velocity was selected to emulate a realistic yet computationally manageable dynamic scenario, allowing a reliable evaluation of the observer and the safety constraints. The reference trajectory of the end-effector, comprising the continuous path and the discretized waypoints used by the controller, was previously illustrated (Fig. 3.1).

This setup was intentionally designed to replicate the conditions of a previous internship experiment involving static obstacle avoidance using the CHOMP planner. In that case, a static obstacle was placed along the robot's path, and trajectory re-planning was employed to generate a feasible solution (Fig. 1.3). In contrast, the current scenario introduces increased complexity by simulating a dynamic obstacle that gradually approaches the reference trajectory. The purpose is to evaluate whether the controller can respond in real time to potential collisions, rather than reacting to a known, static obstacle.

5.3.2 Obstacle state estimation

The evaluation of the internal obstacle observer focused on its ability to estimate the current position of the dynamic obstacle and predict its motion over the control horizon. The observer was tested under the conditions described in Section 5.3, using the observer gains defined in Section 3.4.1, and its behaviour was analysed across multiple simulation runs to verify both convergence and robustness. All experiments were performed with a constant obstacle velocity of $v_y = -0.065$ m/s, as discussed earlier.

Initialization behaviour : Two distinct types of observer initialization were identified during testing.

- **Type 1 - Immediate initialization:** A valid obstacle measurement is received starting from the first control cycle. For instance:

```
Cycle  $t=0$ : o_measured = [0.310, 0.155, 0.250]
```

- **Type 2 - Delayed initialization:** The first valid measurement is received only after several control cycles, typically between $t = 5$ and $t = 7$. Until then, the observer assumes a default distant obstacle position:

```
Cycle  $t=0$ : Observer not initialized! Setting far.  
Cycle  $t=0$ : o_measured = [100.000, 100.000, 100.000]  
⋮  
Cycle  $t=5$ : o_measured = [0.310, 0.126, 0.250]
```

An analysis of logs from 11 independent simulations confirmed that the initial measured obstacle position can vary significantly between runs. In particular, the y -coordinate of the first valid measurement (which corresponds to the direction of motion of the obstacle) showed noticeable fluctuations. This variability is attributed to the asynchronous startup of ROS 2 nodes and the non-deterministic delay in topic discovery between the obstacle simulation node and the MPC controller. Nevertheless, in all test cases, the observer successfully initialized within a few control cycles and then continued operating as expected.

Estimation performance : The following plots illustrate the behaviour of the observer once initialization is complete. All data refer to a representative simulation in which initialization occurred without delay.

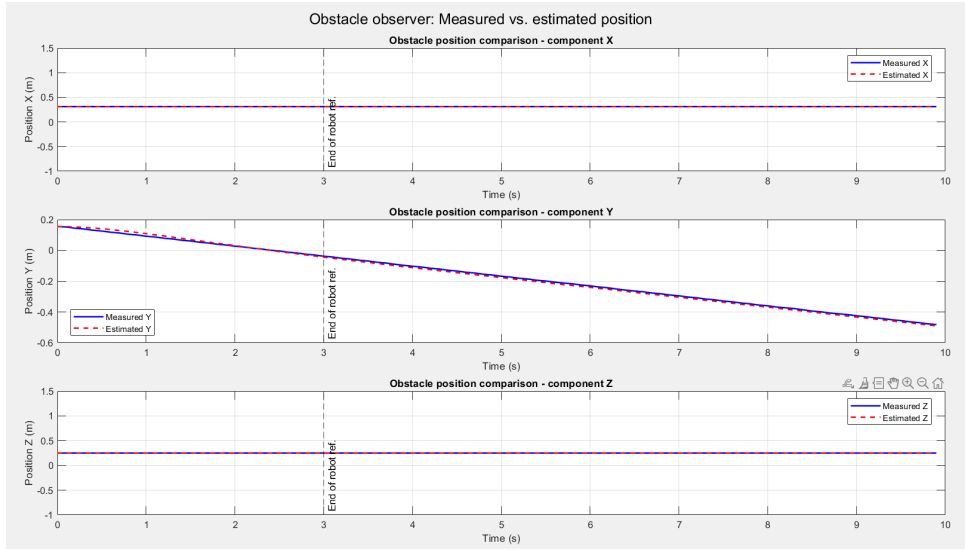


Figure 5.8: Measured obstacle position (blue) vs estimated position (red dashed) along the x -, y -, and z -axes.

The alignment between the obstacle position provided by the simulation and the estimate computed by the observer highlights a consistent match in the x - and z -components, which remain constant due to the one-dimensional motion along the y -axis (Fig. 5.8). Along the y -direction, both trajectories follow a monotonic decreasing trend, corresponding to the obstacle's movement from point A to point B. A slight persistent offset is observed, suggesting a small estimation lag relative to the actual trajectory.

The corresponding estimation error confirms this behaviour, stabilizing around $+1$ cm after an initial transient (Fig. 5.9). The staircase-like profile of the error curve results from the discrete-time operation of the observer, which integrates over a continuous signal in discrete intervals.

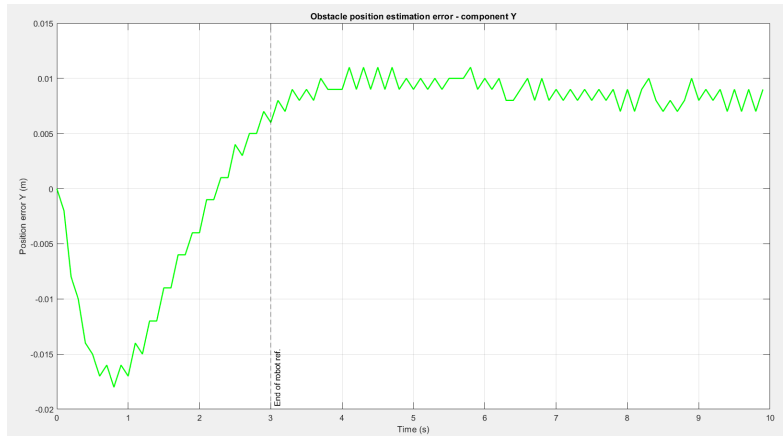


Figure 5.9: Estimation error on the y -component of the obstacle's position.

The estimated obstacle velocity reflects the expected behavior, with the x - and z -components identically zero due to the motion being restricted to the y -axis (Fig. 5.10). The estimated v_y initially starts from zero, momentarily undershoots the target value of -0.065 m/s, and then gradually approaches it from below. Although full convergence is not reached within the 10-second simulation window, the estimation error falls below 0.01 m/s after the transient. This performance is consistent with the observer design and the fixed gains $\alpha_1, \alpha_2, \alpha_3$ previously introduced (Section 5.3), and proves sufficient for enabling short-horizon motion prediction.

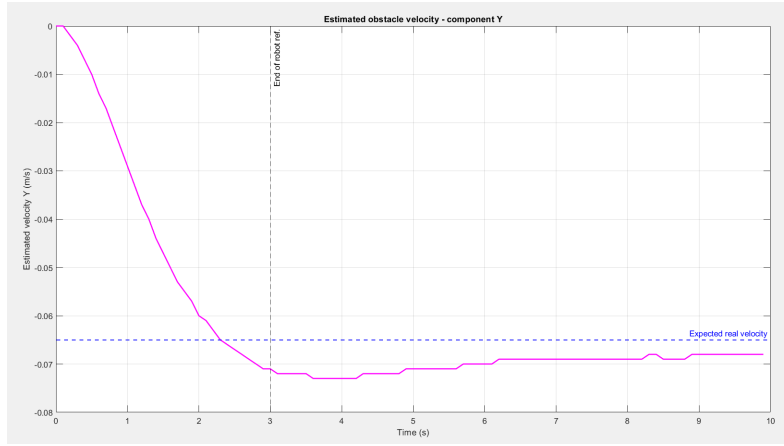


Figure 5.10: Estimated velocity of the obstacle along the y -axis.

The estimated acceleration remains close to zero, as expected under the assumption of constant-velocity motion (Fig. 5.11). Despite this, a small negative bias is introduced by the observer: following an initial sequence of discrete steps, the estimate settles temporarily near $-5 \times 10^{-3} \text{ m/s}^2$ before gradually approaching a steady-state value around $-3 \times 10^{-3} \text{ m/s}^2$. This behaviour reflects the inherent limitations of the discrete integration mechanism employed. Although the acceleration does not converge exactly to zero, the residual error remains sufficiently small to preserve the accuracy of the velocity and position predictions, and thus does not hinder the overall effectiveness of the observer.

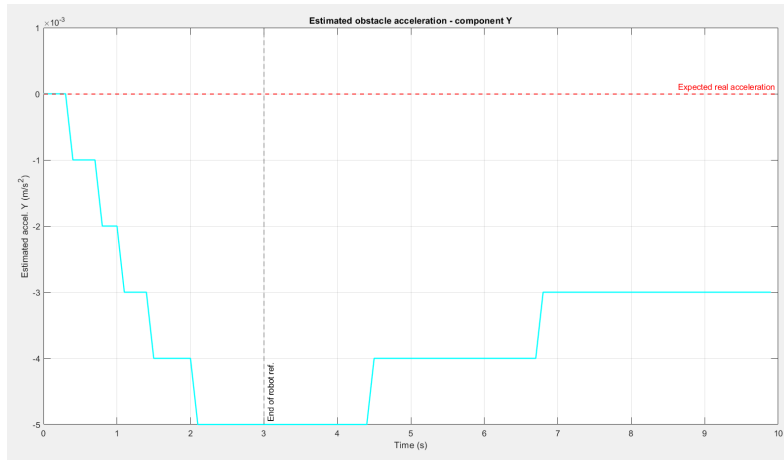


Figure 5.11: Estimated acceleration of the obstacle along the y -axis.

In summary, the observer exhibits consistent and robust performance across all tested scenarios, with sufficiently accurate state estimates to enable its integration into the safety-aware MPC framework described in the following section.

5.3.3 MPC with dynamic obstacle avoidance

This section provides a detailed analysis of the MPC controller's performance when a dynamic obstacle is introduced within the workspace. To handle this scenario, the original MPC formulation is extended by incorporating a single Control Barrier Function Safety Constraint (CBFSC) on the end-effector, described in Section 3.4.2. This constraint ensures a safe separation between the end-effector and the moving obstacle,

enabling the controller to actively adjust the end-effector's trajectory to avoid collisions. By integrating this CBFSC-based safety constraint, the controller maintains collision-free motion while following the reference trajectory, thereby improving its robustness and reliability in dynamic environments.

A new cost parameter P_γ is introduced in the CBF-related penalty term of the objective function. This scalar weight enables tuning the trade-off between safety and efficiency: increasing P_γ promotes greater conservativeness by encouraging the controller to maintain a larger distance from the obstacle. Additionally, a decay rate variable γ_k is introduced as part of the optimization process. This time-varying decision variable controls the relaxation level of the CBF constraints at each step. As discussed, small values of γ_k are favoured in the early stages of avoidance to trigger proactive behaviour, while larger values are permitted later to improve tracking efficiency.

The obstacle used in the simulation is a cube; it's modelled as a circumscribed sphere with radius $R_o = 0.0866$ m for safety assessment purposes. The minimum allowable distance between the end-effector and the obstacle surface is set to $d_{\min} = 0.001$ m. The plots in this section analyse the evolution of both the optimal decay rate γ_k and the distance maintained from the obstacle during the simulation. This distance is computed as the Euclidean norm between the end-effector and the obstacle centre, minus the obstacle radius, consistently with the formulation adopted in the CBF constraint.

Preliminary evaluation of the prediction horizon: Before investigating the effect of the parameter P_γ , a preliminary test was conducted to assess whether the selected prediction horizon $N = 5$ (previously tuned in the obstacle-free scenario) could be reduced without compromising the controller's ability to avoid dynamic obstacles.

Surprisingly, even with a shorter horizon $N = 2$, the robot was still able to perform safe avoidance manoeuvres. This behaviour aligns with findings reported in [2], where the authors emphasize the responsiveness of their flexible CBFSC strategy, which successfully avoids obstacles even with $N = 1$. According to their analysis, this is achieved by combining adjustable safety margins and the dynamic optimization of the decay rate γ_k , allowing early triggering of avoidance behaviour even when the obstacle is still far from the robot. In contrast, standard MPC formulations typically activate safety constraints only close to obstacles, which may be insufficient in fast-moving or highly dynamic environments.

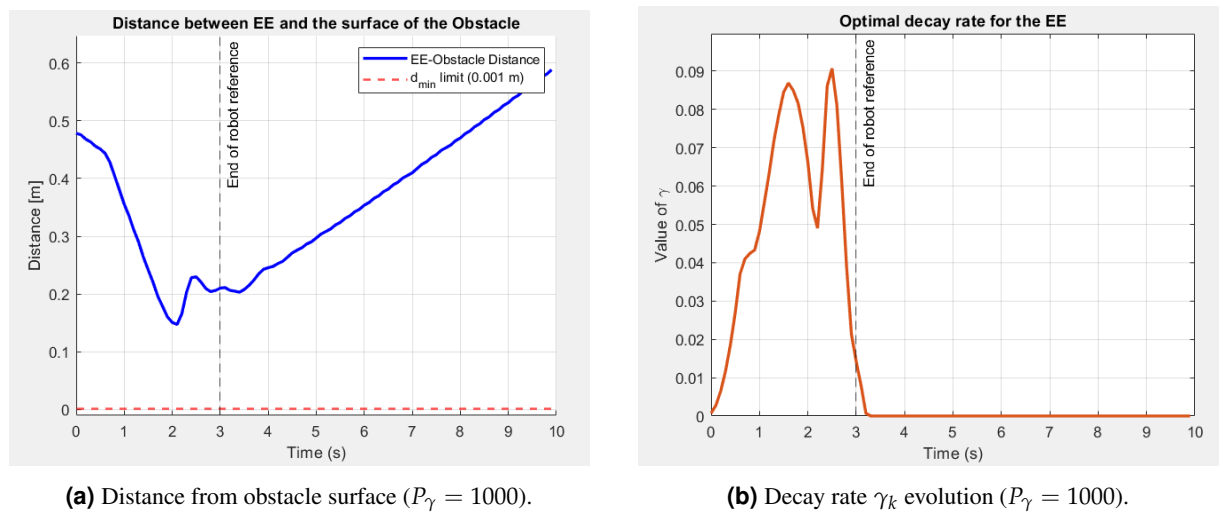


Figure 5.12: Obstacle avoidance behaviour with high penalty $P_\gamma = 1000$

In the present work, no systematic comparison with alternative MPC strategies was performed, as the focus is instead placed on evaluating the impact of P_γ on the trade-off between safety and efficiency during dynamic obstacle avoidance. All analyses presented in this section were carried out with $N = 2$, confirming that such a short prediction horizon is sufficient under the proposed strategy.

Impact of P_γ on safety margins and avoidance behaviour: This section explores how the parameter P_γ influences the trade-off between safety and efficiency during dynamic obstacle avoidance. Specifically, two values are compared: a high value $P_\gamma = 1000$ and a low value $P_\gamma = 25$, while keeping all other parameters fixed.

Case $P_\gamma = 1000$: The robot maintains a significant safety margin throughout the manoeuvre. The minimum distance between the end-effector and the obstacle surface reaches approximately 0.15 m, well above the threshold $d_{\min} = 0.001$ m (red dashed line in Fig. 5.12a). This behaviour reflects the conservative effect of a large P_γ , which strongly penalizes large values of the decay rate γ_k , encouraging the controller to maintain a greater distance from the obstacle and avoid aggressive avoidance manoeuvres. The evolution of γ_k over time shows (Fig. 5.12b) two distinct peaks: the first occurs at $t \approx 1.5$ s, reaching a value of about 0.085; the second, slightly higher peak reaches 0.09 at $t \approx 2.5$ s. Between the two peaks, the decay rate temporarily decreases to intermediate values (e.g., around 0.05), reflecting a partial relaxation of the avoidance constraint. After the second peak, γ_k rapidly returns to zero, indicating that the robot has safely cleared the obstacle.

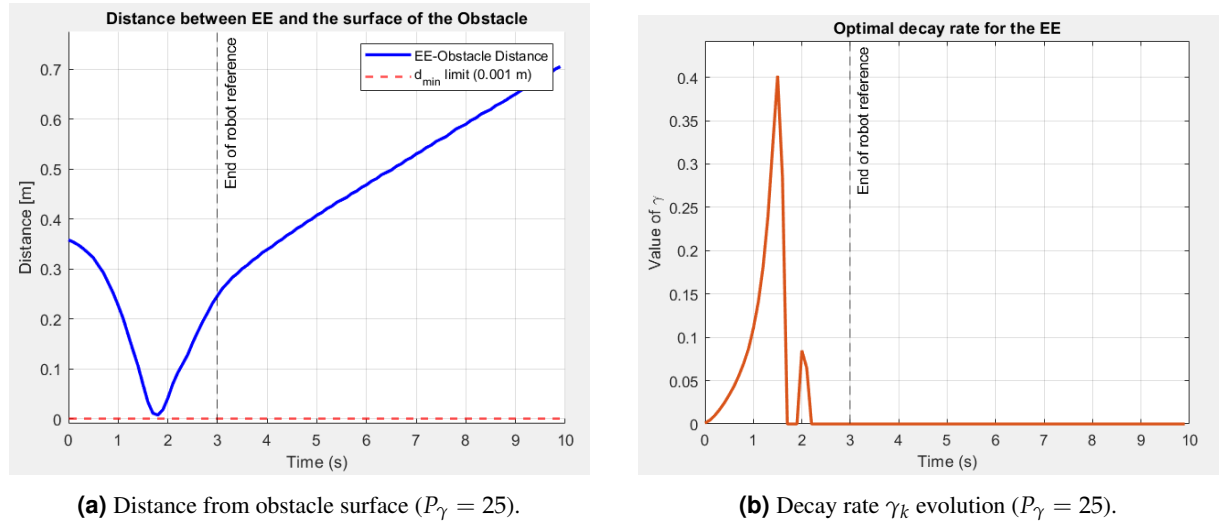


Figure 5.13: Obstacle avoidance behaviour with low penalty $P_\gamma = 25$ and more aggressive γ_k dynamics.

Case $P_\gamma = 25$: Reveal a markedly different avoidance behaviour. The end-effector approaches the obstacle much more closely (Fig. 5.13a), reaching a distance just slightly above $d_{\min}$. The distance curve features a clear minimum near the obstacle, followed by a gradual increase as the end-effector moves past it. The decay rate γ_k rises sharply (Fig. 5.13b) to about 0.4 at $t \approx 1.5$ s, followed by an almost instantaneous drop. A smaller secondary peak (about 0.085) is observed around $t = 2$ s, after which γ_k immediately falls to zero and remains at that value for the rest of the trajectory. These sharp dynamics reflect a more aggressive optimization, aimed at maximizing efficiency while satisfying only the strict minimum safety constraint, so the robot approaches closer to the obstacle.

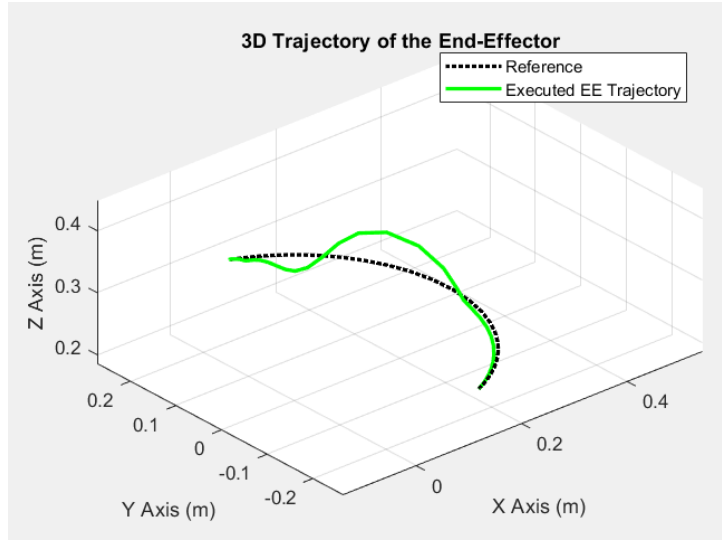


Figure 5.14: 3D view of the executed trajectory under $P_\gamma = 25$.

The executed trajectory adapts to the presence of the obstacle by rising along the z -axis to circumvent it before returning to the original motion plane (Fig. 5.14). This deviation from the nominal reference path indicates the correct activation of the dynamic avoidance mechanism. The trajectory deformation also provides geometric insight into the effect of Control Barrier Function constraints on path shaping, despite the absence of explicit time information in the spatial representation.

For higher values of P_γ , the end-effector tends to deviate earlier from the reference trajectory and maintains a larger clearance from the obstacle, thereby increasing the safety margin (Fig. 5.12a). However, this more conservative behavior results in reduced task effectiveness, as the end-effector diverges further from the nominal path, potentially compromising tracking accuracy.

This observation highlights an inherent trade-off between safety and tracking performance: increasing the safety margin by raising P_γ enhances obstacle avoidance, but at the expense of trajectory fidelity. Therefore, the selection of an appropriate P_γ value requires balancing these competing objectives according to the specific requirements and safety constraints of the application.

In practice, the optimal choice of P_γ depends on the operational context. In scenarios where safety is paramount, a higher P_γ may be preferable, even if it leads to less accurate trajectory tracking. Conversely, in applications where precise tracking is critical and the environment is less constrained, a lower P_γ may be more suitable. Thus, tuning P_γ enables the controller to be adapted to different priorities, ensuring both flexibility and robustness.

Conclusions and future works

This thesis contributes to the field of robotics and industrial automation, with a specific focus on advanced control techniques for robotic manipulators operating in dynamic environments. The main objective was to design, implement, and validate a Model Predictive Control (MPC) framework capable of accurately following a predefined trajectory while actively avoiding dynamic obstacles in real time.

6.1 Contribution

The system was developed and tested on the Niryo Ned2 robotic arm within the ROS2 and MoveIt2 frameworks, incorporating a Generalized Proportional Integral Observer (GPIO) for obstacle state estimation, kinematic models for control, and safety constraints formulated through Control Barrier Functions (CBFs). In particular, the use of Control Barrier Function-based Safety Constraints (CBFSCs) with dynamically optimized decay rates enabled the controller to adapt the safety margins during execution. These decay rates were treated as decision variables within the optimization problem, allowing the robot to balance tracking accuracy and safety depending on the proximity and velocity of surrounding obstacles.

Simulation results confirmed the effectiveness of the proposed architecture, showing good trajectory tracking, smooth control signals, and successful obstacle avoidance. Moreover, the work highlights the potential of combining predictive control and flexible safety criteria within modern robotic frameworks, providing a foundation for more adaptive and reliable robotic behaviours in complex and dynamic scenarios.

6.2 Challenges and limitations

The development of the proposed system also involved several technical challenges:

- **Computational complexity:** The real-time nature of the MPC optimization required careful tuning of the controller frequency (10–25 Hz), limiting the prediction horizon and overall control performance.
- **Lack of open-source reference material:** One of the most significant difficulties was the limited availability of open-source resources or documentation for integrating MPC within the ROS2 and MoveIt2 ecosystems. Most available implementations are still based on ROS1 or rely on custom, deprecated frameworks.

- **Controller design from scratch:** Due to the lack of existing tools, the complete MPC logic (including problem formulation, communication between nodes, and synchronisation with RViz) was implemented from scratch. This required extensive debugging and system design, but also contributed to a deeper understanding of the involved components and improved modularity.

Overall, this thesis establishes a solid foundation for real-time predictive control with adaptive safety in robotic manipulators and opens promising directions for future research and experimental validation on physical hardware.

6.3 Future research directions

Although the results are promising, several limitations and opportunities for further development remain:

- **Improved quaternion integration:** The current implementation relies on Euler’s method to integrate the orientation error. A more accurate approach (such as the Runge-Kutta method or exponential map integration) could improve the stability and precision of orientation tracking, particularly in scenarios with fast rotational dynamics.
- **Acceleration constraints:** No explicit bounds are currently imposed on joint accelerations. This may lead to abrupt control actions, which could affect the hardware’s durability or cause instability in physical implementations. Introducing second-order constraints would improve motion smoothness and feasibility.
- **Trajectory interpolation:** The reference trajectory is defined as a sequence of discrete waypoints. Applying linear or cubic interpolation techniques would generate smoother references and improve overall tracking quality.
- **Complex and fast-moving obstacles:** The current evaluation uses a relatively small and slow obstacle. Future work should investigate the system’s robustness against larger or faster-moving obstacles, possibly incorporating the prediction of obstacle dynamics.
- **Dynamic modelling:** The present control formulation is based purely on kinematics. Incorporating the full dynamic model of the manipulator would enable more accurate and realistic behaviour, especially in tasks involving high-speed motions or interactions with external forces.
- **Extension of safety constraints to additional robot parts:** Currently, the CBFSCs are applied only to the end-effector. Extending these constraints to other critical robot links or points along the manipulator could enhance overall safety by enabling the entire robot body to avoid collisions in dynamic environments actively.

References

- [1] Tianqi Zhu et al. “Real-Time Dynamic Obstacle Avoidance for Robot Manipulators Based on Cascaded Nonlinear MPC With Artificial Potential Field”. In: *IEEE Transactions on Industrial Electronics* 71.7 (2024), pp. 7424–7434. DOI: [10.1109/TIE.2023.3306405](https://doi.org/10.1109/TIE.2023.3306405).
- [2] Jinhao Liu et al. “Flexible Active Safety Motion Control for Robotic Obstacle Avoidance: A CBF-Guided MPC Approach”. In: *IEEE Robotics and Automation Letters* 10.3 (2025), pp. 2686–2693. DOI: [10.1109/LRA.2025.3534519](https://doi.org/10.1109/LRA.2025.3534519).
- [3] Maidul Islam, Mohamed Okasha, and Erwin Sulaeman. “A Model Predictive Control (MPC) Approach on Unit Quaternion Orientation Based Quadrotor for Trajectory Tracking”. In: *International Journal of Control, Automation and Systems* 17 (July 2019). DOI: [10.1007/s12555-018-0860-9](https://doi.org/10.1007/s12555-018-0860-9).
- [4] Max Schwenzer et al. “Review on model predictive control: an engineering perspective”. In: *The International Journal of Advanced Manufacturing Technology* 117 (Nov. 2021), pp. 1–23. DOI: [10.1007/s00170-021-07682-3](https://doi.org/10.1007/s00170-021-07682-3).
- [5] James B. Rawlings, David Q. Mayne, and Moritz M. Diehl. *Model Predictive Control: Theory, Computation, and Design*. 2nd. Madison, WI: Nob Hill Publishing, 2017. URL: <https://sites.engineering.ucsb.edu/~jbrow/mpc/MPC-book-2nd-edition-1st-printing.pdf>.
- [6] Daniel Simon. “Model Predictive Control in Flight Control Design - Stability and Reference Tracking”. PhD thesis. Mar. 2014. ISBN: 9789175194226. DOI: [10.3384/lic.diva-103742](https://doi.org/10.3384/lic.diva-103742).
- [7] MathWorks. *What Is Model Predictive Control?* Accessed: 2025-07-02. 2024. URL: <https://it.mathworks.com/help/mpc/gs/what-is-mpc.html>.
- [8] Nathan Ratliff et al. “CHOMP: Gradient Optimization Techniques for Efficient Motion Planning”. In: *Proceedings of (ICRA) International Conference on Robotics and Automation*. 2009, pp. 489–494.
- [9] Ayush Agrawal and Koushil Sreenath. “Discrete Control Barrier Functions for Safety-Critical Control of Discrete Systems with Application to Bipedal Robot Navigation”. In: *Robotics: Science and Systems (RSS)*. 2017. DOI: [10.15607/RSS.2017.XIII.073](https://doi.org/10.15607/RSS.2017.XIII.073).
- [10] Jinhao Liu et al. “Single-Loop Robust Model Predictive Speed Regulation of PMSM Based on Exogenous Signal Preview”. In: *IEEE Transactions on Industrial Electronics* 70.12 (2023), pp. 12719–12729. DOI: [10.1109/TIE.2023.3239938](https://doi.org/10.1109/TIE.2023.3239938).

REFERENCES

- [11] Steven Macenski et al. “Robot Operating System 2: Design, architecture, and uses in the wild”. In: *Science Robotics* 7.66 (2022), eabm6074. DOI: [10.1126/scirobotics.abm6074](https://doi.org/10.1126/scirobotics.abm6074). URL: <https://www.science.org/doi/abs/10.1126/scirobotics.abm6074>.
- [12] Michael Görner et al. “MoveIt! Task Constructor for Task-Level Motion Planning”. In: *2019 International Conference on Robotics and Automation (ICRA)*. May 2019, pp. 190–196. DOI: [10.1109/ICRA.2019.8793898](https://doi.org/10.1109/ICRA.2019.8793898).
- [13] ROS 2 Documentation Contributors. *RViz User Guide (Humble Hawksbill)*. <https://docs.ros.org/en/humble/Tutorials/Intermediate/RViz/RViz-User-Guide/RViz-User-Guide.html>. Accessed: June 13, 2025. 2022.
- [14] Niryo. *Ned2 - 6 axis collaborative robot for Education, Research and Industry*. <https://niryo.com/product/educational-desktop-robotic-arm/>.
- [15] Niryo Robotics. *Niryo ROS1 packages*. https://github.com/NiryoRobotics/ned_ros/.
- [16] Jean-Claude Latombe. *Robot Motion Planning*. Kluwer Academic Publishers, 1991.
- [17] Steven M. LaValle. “Rapidly-exploring random trees : a new tool for path planning”. In: *Technical Report 98-11, Computer Science Dept., Iowa State University* (1998).
- [18] Peter E. Hart, Nils J. Nilsson, and Bertram Raphael. “A Formal Basis for the Heuristic Determination of Minimum Cost Paths”. In: *IEEE Transactions on Systems Science and Cybernetics* 4.2 (1968), pp. 100–107. DOI: [10.1109/TSSC.1968.300136](https://doi.org/10.1109/TSSC.1968.300136).
- [19] Oussama Khatib. “Real-time obstacle avoidance for manipulators and mobile robots”. In: *Proceedings. 1985 IEEE International Conference on Robotics and Automation*. Vol. 2. 1985, pp. 500–505. DOI: [10.1109/ROBOT.1985.1087247](https://doi.org/10.1109/ROBOT.1985.1087247).
- [20] K.K. Kumbla and M. Jamshidi. “Control of robotic manipulator using fuzzy logic”. In: *Proceedings of 1994 IEEE 3rd International Fuzzy Systems Conference*. 1994, 518–523 vol.1. DOI: [10.1109/FUZZY.1994.343731](https://doi.org/10.1109/FUZZY.1994.343731).
- [21] Jens Kober, J. Andrew Bagnell, and Jan Peters. “Reinforcement learning in robotics: A survey”. In: *The International Journal of Robotics Research* 32.11 (2013), pp. 1238–1274. DOI: [10.1177/0278364913495721](https://doi.org/10.1177/0278364913495721).
- [22] Andreas Völz and Knut Graichen. “A Predictive Path-Following Controller for Continuous Replanning With Dynamic Roadmaps”. In: *IEEE Robotics and Automation Letters* 4.4 (2019), pp. 3963–3970. DOI: [10.1109/LRA.2019.2929990](https://doi.org/10.1109/LRA.2019.2929990).
- [23] Yuhan Chen et al. “Model Predictive Control With Integral Compensation for Motion Control of Robot Manipulator in Joint and Task Spaces”. In: *IEEE Access* 8 (2020), pp. 107063–107075. DOI: [10.1109/ACCESS.2020.3001044](https://doi.org/10.1109/ACCESS.2020.3001044).
- [24] Argitim Tika and Naim Bajcinca. “Predictive Control of Cooperative Robots Sharing Common Workspace”. In: *IEEE Transactions on Control Systems Technology* 32.2 (2024), pp. 456–471. DOI: [10.1109/TCST.2023.3331525](https://doi.org/10.1109/TCST.2023.3331525).
- [25] David Q. Mayne. “Model predictive control: Recent developments and future promise”. In: *Automatica* 50.12 (2014), pp. 2967–2986. DOI: [10.1016/j.automatica.2014.10.128](https://doi.org/10.1016/j.automatica.2014.10.128).

REFERENCES

- [26] Julian Nubert et al. “Safe and Fast Tracking on a Robot Manipulator: Robust MPC and Neural Network Control”. In: *IEEE Robotics and Automation Letters* 5.2 (2020), pp. 3050–3057. DOI: [10.1109/LRA.2020.2975727](https://doi.org/10.1109/LRA.2020.2975727).
- [27] Ajay Suresha Sathya et al. “Real-time Robot Arm Motion Planning and Control with Nonlinear Model Predictive Control using Augmented Lagrangian on a First-Order Solver”. In: *2020 European Control Conference (ECC)*. 2020, pp. 507–512. DOI: [10.23919/ECC51009.2020.9143732](https://doi.org/10.23919/ECC51009.2020.9143732).
- [28] Matteo Rubagotti et al. “Shared Control of Robot Manipulators With Obstacle Avoidance: A Deep Reinforcement Learning Approach”. In: *IEEE Control Systems Magazine* 43.1 (2023), pp. 44–63. DOI: [10.1109/MCS.2022.3216653](https://doi.org/10.1109/MCS.2022.3216653).
- [29] Alejandro Astudillo et al. “Rapid Deployment of Model Predictive Control for Robotic Systems: From IMPACT to ROS 2 Through Code Generation”. In: *2024 IEEE 18th International Conference on Advanced Motion Control (AMC)*. 2024, pp. 1–6. DOI: [10.1109/AMC58169.2024.10505632](https://doi.org/10.1109/AMC58169.2024.10505632).
- [30] Joel Andersson et al. “CasADi: a software framework for nonlinear optimization and optimal control”. In: *Mathematical Programming Computation* 11 (July 2018). DOI: [10.1007/s12532-018-0139-4](https://doi.org/10.1007/s12532-018-0139-4).
- [31] J. Andersson et al. *CasADi: a software framework for nonlinear optimization and optimal control*. <https://github.com/casadi/casadi>. Accessed: 2025-06-24.
- [32] Rudolf E Kalman. “A new approach to linear filtering and prediction problems”. In: *Journal of Basic Engineering* 82.1 (1960), pp. 35–45.
- [33] Simon J Julier and Jeffrey K Uhlmann. “A new extension of the Kalman filter to nonlinear systems”. In: *Proceedings of AeroSense: The 11th International Symposium on Aerospace/Defense Sensing, Simulation and Controls* 3.26 (1997), pp. 182–193.
- [34] Arie Levant. “Robust exact differentiation via sliding mode technique”. In: *Automatica* 34.3 (1998), pp. 379–384.
- [35] Arie Levant. “Higher-order sliding modes, differentiation and output-feedback control”. In: *International Journal of Control - INT J CONTR* 76 (June 2003), pp. 924–941. DOI: [10.1080/0020717031000099029](https://doi.org/10.1080/0020717031000099029).
- [36] Aaron D. Ames et al. “Control Barrier Functions: Theory and Applications”. In: *2019 18th European Control Conference (ECC)*. 2019, pp. 3420–3431. DOI: [10.23919/ECC.2019.8796030](https://doi.org/10.23919/ECC.2019.8796030).
- [37] Andrew Singletary, Shishir Kolathaya, and Aaron D. Ames. “Safety-Critical Kinematic Control of Robotic Systems”. In: *IEEE Control Systems Letters* 6 (2022), pp. 139–144. DOI: [10.1109/LCSYS.2021.3050609](https://doi.org/10.1109/LCSYS.2021.3050609).
- [38] Muhammad Ali Murtaza et al. “Safety Compliant Control for Robotic Manipulator With Task and Input Constraints”. In: *IEEE Robotics and Automation Letters* 7.4 (2022), pp. 10659–10664. DOI: [10.1109/LRA.2022.3179118](https://doi.org/10.1109/LRA.2022.3179118).
- [39] Yunxi Tang et al. “Learning-Based MPC With Safety Filter for Constrained Deformable Linear Object Manipulation”. In: *IEEE Robotics and Automation Letters* 9.3 (2024), pp. 2877–2884. DOI: [10.1109/LRA.2024.3362643](https://doi.org/10.1109/LRA.2024.3362643).

REFERENCES

- [40] Jun Zeng, Zhongyu Li, and Koushil Sreenath. “Enhancing Feasibility and Safety of Nonlinear Model Predictive Control with Discrete-Time Control Barrier Functions”. In: *2021 60th IEEE Conference on Decision and Control (CDC)*. 2021, pp. 6137–6144. DOI: [10.1109/CDC45484.2021.9683174](https://doi.org/10.1109/CDC45484.2021.9683174).
- [41] Jun Zeng et al. “Safety-Critical Control using Optimal-decay Control Barrier Function with Guaranteed Point-wise Feasibility”. In: *2021 American Control Conference (ACC)*. 2021, pp. 3856–3863. DOI: [10.23919/ACC50511.2021.9482626](https://doi.org/10.23919/ACC50511.2021.9482626).
- [42] Riccardo Periotto et al. “MPC-CBF with Adaptive Safety Margins for Safety-Critical Teleoperation over Imperfect Network Connections”. In: *2024 European Control Conference (ECC)*. 2024, pp. 1609–1615. DOI: [10.23919/ECC64448.2024.10590767](https://doi.org/10.23919/ECC64448.2024.10590767).
- [43] James Diebel. “Representing Attitude: Euler Angles, Unit Quaternions, and Rotation Vectors”. In: *Matrix* 58 (Jan. 2006).

Source code Appendix

Listing 1: C++ node responsible for creating the simulation environment (floor and dynamic obstacle)

```
1 #include <moveit/planning_scene_interface/planning_scene_interface.h>
2 #include <geometry_msgs/msg/pose_stamped.hpp>
3 #include <memory>
4 #include <rclcpp/rclcpp.hpp>
5 #include <thread>
6
7 int main(int argc, char * argv[])
8 {
9     // Start up ROS 2
10    rclcpp::init(argc, argv);
11
12    auto const node = std::make_shared<rclcpp::Node>(
13        "dynamic_scene_setup",
14        rclcpp::NodeOptions().automatically_declare_parameters_from_over
15        rides(true)
16    );
17
18    auto const logger = rclcpp::get_logger("dynamic_scene_setup");
19
20    rclcpp::executors::SingleThreadedExecutor executor;
21    executor.add_node(node);
22    auto spinner = std::thread([&executor]() { executor.spin(); });
23
24    // Publisher for the obstacle's pose
25    rclcpp::Publisher<geometry_msgs::msg::PoseStamped>::SharedPtr
26        obstacle_pose_publisher;
27    obstacle_pose_publisher = node->create_publisher<geometry_msgs::msg::Pose
28        Stamped>("/obstacle_pose", 10);
29
30    // Create the floor
31    auto const create_floor = [frame_id = "world", &node, &logger] {
32        moveit_msgs::msg::CollisionObject floor;
33        floor.header.frame_id = frame_id;
34        floor.header.stamp = node->now();
35        floor.id = "floor";
36        shape_msgs::msg::SolidPrimitive floor_primitive;
37        floor_primitive.type = shape_msgs::msg::SolidPrimitive::BOX;
38        floor_primitive.dimensions = {3.0, 3.0, 0.1};
39
40        geometry_msgs::msg::Pose floor_pose;
41        floor_pose.position.x = 0.0;
```

```
42     floor_pose.position.y = 0.0;
43     floor_pose.position.z = -(floor_primitive.dimensions[2] / 2.0);
44     floor_pose.orientation.w = 1.0;
45
46     floor.primitives.push_back(floor_primitive);
47     floor.primitive_poses.push_back(floor_pose);
48     floor.operation = floor.ADD;
49
50     RCLCPP_INFO(logger, "Floor added to the collision scene.");
51     return floor;
52 }();
53
54 // Apply the floor to the scene
55 moveit::planning_interface::PlanningSceneInterface planning_scene_interface;
56 planning_scene_interface.applyCollisionObjects({create_floor});
57 rclcpp::sleep_for(std::chrono::seconds(1));
58
59 // Create the box
60 auto const create_box = [frame_id = "world", &node, &logger] {
61     moveit_msgs::msg::CollisionObject box;
62     box.header.frame_id = frame_id;
63     box.header.stamp = node->now();
64     box.id = "box1";
65
66     shape_msgs::msg::SolidPrimitive box_primitive;
67     box_primitive.type = shape_msgs::msg::SolidPrimitive::BOX;
68     box_primitive.dimensions = {0.10, 0.10, 0.10}; // 10x10x10 cm cube
69
70     geometry_msgs::msg::Pose box_pose;
71     box_pose.position.x = 0.31;
72     box_pose.position.y = 0.5;
73     box_pose.position.z = 0.25;
74     box_pose.orientation.w = 1.0;
75
76     box.primitives.push_back(box_primitive);
77     box.primitive_poses.push_back(box_pose);
78     box.operation = box.ADD;
79
80     RCLCPP_INFO(logger, "Box added to the collision scene.");
81     return box;
82 }();
83
84 // Apply the box to the scene
85 planning_scene_interface.applyCollisionObjects({create_box});
86 rclcpp::sleep_for(std::chrono::seconds(1)); // Time for scene update
87
88 // (1) Create the publisher for PlanningScene
89 auto planning_scene_diff_publisher = node->create_publisher
90 <moveit_msgs::msg::PlanningScene>("planning_scene", 1);
91
92 // (2) Define points A and B
93 geometry_msgs::msg::Pose pose_A;
94 pose_A.position.x = 0.31;
95 pose_A.position.y = 0.5;
96 pose_A.position.z = 0.25;
```

```

97     pose_A.orientation.w = 1.0;
98
99     geometry_msgs::msg::Pose pose_B;
100     pose_B.position.x = 0.31;
101     pose_B.position.y = -0.5;
102     pose_B.position.z = 0.25;
103     pose_B.orientation.w = 1.0;
104
105     // (3) Initialize monotonic clock
106     rclcpp::Clock steady_clock(RCL_STEADY_TIME);
107     rclcpp::Time start_time = rclcpp::Time(0, 0, RCL_STEADY_TIME);
108     bool first_run = true;
109
110     // Global variable to save the previous relative time
111     static double previous_relative_time = 0.0;
112
113     // (4) Function to get relative time
114     auto getRelativeTime = [&start_time, &steady_clock, &node]() -> double {
115         return (steady_clock.now() - start_time).seconds();
116     };
117
118     // (5) Function to get interpolated pose
119     auto getInterpolatedPose = [](double t, const geometry_msgs::msg::Pose&
120         start_pose, const geometry_msgs::msg::Pose& end_pose) ->
121         geometry_msgs::msg::Pose {
122         geometry_msgs::msg::Pose pose;
123         pose.position.x = start_pose.position.x +
124             (end_pose.position.x - start_pose.position.x) * t;
125         pose.position.y = start_pose.position.y +
126             (end_pose.position.y - start_pose.position.y) * t;
127         pose.position.z = start_pose.position.z +
128             (end_pose.position.z - start_pose.position.z) * t;
129         pose.orientation.w = 1.0;
130         return pose;
131     };
132
133     // (6) Total duration of movement and number of cycles
134     // Total duration to go from A to B (and vice-versa)
135     double total_duration = 15.4;
136     int num_cycles = 1; // Number of A -> B -> A cycles
137     // Duration of a full cycle (A -> B -> A)
138     double cycle_duration = 2 * total_duration;
139
140     // (7) Function to update box position
141     auto updateDynamicBox = [&](double relative_time) {
142         // Approximate relative time to reduce numerical errors
143         double approx_relative_time = std::round(relative_time * 1e6) / 1e6;
144
145         // Calculate relative time within the current cycle
146         double cycle_relative_time = fmod(approx_relative_time, cycle_duration);
147
148         // Determine the current direction (with a tolerance margin)
149         bool current_direction = (cycle_relative_time + 0.001 < total_duration);
150
151         // Calculate the completed fraction of the movement t

```

```
152     double t = (current_direction ? cycle_relative_time :
153                 (cycle_relative_time - total_duration)) / total_duration;
154
155     // Clamp t to the interval [0.0, 1.0]
156     if (t < 0.0) t = 0.0;
157     if (t > 1.0) t = 1.0;
158
159     // Debugging: relative_time, cycle_relative_time, t and direction
160     RCLCPP_INFO(node->get_logger(), "Relative Time: %.6f,
161     Cycle Relative Time: %.6f, t: %.6f, Direction: %s",
162     approx_relative_time, cycle_relative_time, t, current_direction ?
163     "A -> B" : "B -> A");
164
165     // Calculate the interpolated pose
166     geometry_msgs::msg::Pose interpolated_pose;
167     if (current_direction) {
168         interpolated_pose = getInterpolatedPose(t, pose_A, pose_B);
169     } else {
170         interpolated_pose = getInterpolatedPose(t, pose_B, pose_A);
171     }
172
173     // Debugging: Print the interpolated pose
174     RCLCPP_INFO(node->get_logger(),
175     "Interpolated Pose: x=%.3f, y=%.3f, z=%.3f",
176     interpolated_pose.position.x,
177     interpolated_pose.position.y,
178     interpolated_pose.position.z);
179
180     // Create the CollisionObject message
181     moveit_msgs::msg::CollisionObject move_box;
182     move_box.header.frame_id = "world";
183     move_box.id = "box1";
184     move_box.operation = moveit_msgs::msg::CollisionObject::MOVE;
185     move_box.pose = interpolated_pose;
186
187     // Create the PlanningScene message
188     moveit_msgs::msg::PlanningScene planning_scene;
189     planning_scene.is_diff = true;
190     planning_scene.world.collision_objects.push_back(move_box);
191
192     // Publish the message
193     planning_scene_diff_publisher->publish(planning_scene);
194
195     // Publish the obstacle's pose on the dedicated topic
196     geometry_msgs::msg::PoseStamped obstacle_pose_msg;
197     obstacle_pose_msg.header.stamp = node->now();
198     obstacle_pose_msg.header.frame_id = "world";
199     obstacle_pose_msg.pose = interpolated_pose;
200
201     obstacle_pose_publisher->publish(obstacle_pose_msg);
202 };
203
204 // (8) Set up timer to update the box
205 rclcpp::TimerBase::SharedPtr timer_;
206 timer_ = node->create_wall_timer(
```

```

207     std::chrono::milliseconds(40),
208     [&]() {
209         if (first_run) {
210             // Initialize start_time only on the first run
211             start_time = steady_clock.now();
212             first_run = false;
213             return;
214         }
215
216         // Calculate the global relative time
217         double relative_time = getRelativeTime();
218
219         // Update the box's position
220         updateDynamicBox(relative_time);
221
222         // Check if all cycles have been completed
223         if (relative_time ≥ num_cycles * cycle_duration) {
224             // Stop the timer after completing all cycles
225             timer_>cancel();
226             RCLCPP_INFO(node->get_logger(), "Movement completed.");
227         }
228     }
229 );
230
231 spinner.join();
232 return 0;
233 }

```

Listing 2: C++ node that uses CHOMP and MoveIt 2 to generate and sample the Cartesian reference trajectory

```

1  #include <moveit/move_group_interface/move_group_interface.h>
2  #include <moveit/planning_scene_interface/planning_scene_interface.h>
3  #include <moveit_visual_tools/moveit_visual_tools.h>
4  #include <geometry_msgs/msg/pose_stamped.hpp>
5  #include <memory>
6  #include <rclcpp/rclcpp.hpp>
7  #include <thread>
8  #include <Eigen/Geometry>
9  #include <tf2_eigen/tf2_eigen.hpp>
10 #include <moveit/planning_scene_monitor/planning_scene_monitor.h>
11 #include <moveit/robot_trajectory/robot_trajectory.h>
12 #include <trajectory_msgs/msg/joint_trajectory_point.hpp>
13 #include <fstream>
14
15 int main(int argc, char * argv[])
16 {
17     rclcpp::init(argc, argv);
18
19     auto const node = std::make_shared<rclcpp::Node>(
20         "desired_trajectory_sampling",
21         rclcpp::NodeOptions().automatically_declare_parameters_from_overrides(true)
22     );
23
24     auto const logger = rclcpp::get_logger("desired_trajectory_sampling");

```

```
25
26 rclcpp::executors::SingleThreadedExecutor executor;
27 executor.add_node(node);
28 auto spinner = std::thread([&executor]() { executor.spin(); });
29
30 using moveit::planning_interface::MoveGroupInterface;
31 auto arm_group_interface = MoveGroupInterface(node, "niryo_arm");
32
33 // Obtain the current position of the end-effector before planning
34 auto current_pose = arm_group_interface.getCurrentPose();
35
36 // Specify a planning pipeline to be used for further planning
37 arm_group_interface.setPlanningPipelineId("chomp");
38 arm_group_interface.setPlanningTime(1.0);
39 arm_group_interface.setMaxVelocityScalingFactor(1.0);
40 arm_group_interface.setMaxAccelerationScalingFactor(1.0);
41
42 // Construct and initialize MoveItVisualTools
43 auto moveit_visual_tools =
44     moveit_visual_tools::MoveItVisualTools{ node, "world",
45         rviz_visual_tools::RVIZ_MARKER_TOPIC,
46         arm_group_interface.getRobotModel() };
47
48 // Erase all existing visual markers from the RViz visualization.
49 moveit_visual_tools.deleteAllMarkers();
50
51 // Load the remote control interface for MoveIt visual tools.
52 moveit_visual_tools.loadRemoteControl();
53
54 // Lambda function to visualize a trajectory as a line in RViz
55 auto const draw_trajectory_tool_path =
56     [&moveit_visual_tools, jmg = arm_group_interface.getRobotModel()
57     ->getJointModelGroup("niryo_arm")]( auto const trajectory)
58     {moveit_visual_tools.publishTrajectoryLine(trajectory, jmg);};
59
60 // Set a target pose for the end effector of the arm
61 auto const arm_target_pose1 = [&node]{
62     geometry_msgs::msg::PoseStamped msg;
63     msg.header.frame_id = "world";
64     msg.header.stamp = node->now();
65     msg.pose.position.x = 0.111;
66     msg.pose.position.y = 0.272;
67     msg.pose.position.z = 0.268;
68     msg.pose.orientation.x = -0.351;
69     msg.pose.orientation.y = 0.150;
70     msg.pose.orientation.z = 0.850;
71     msg.pose.orientation.w = 0.364;
72     return msg;
73 }();
74
75 auto const arm_target_pose2 = [&node]{
76     geometry_msgs::msg::PoseStamped msg;
77     msg.header.frame_id = "world";
78     msg.header.stamp = node->now();
79     msg.pose.position.x = 0.120;
```

```
80     msg.pose.position.y = -0.268;
81     msg.pose.position.z = 0.268;
82     msg.pose.orientation.x = 0.0;
83     msg.pose.orientation.y = 0.382;
84     msg.pose.orientation.z = 0.0;
85     msg.pose.orientation.w = 0.923;
86     return msg;
87 }();
88
89 // File in which I want to save the data
90 std::ofstream ee_file("/home/francesco/lab_ROS2/ws_moveit2/src/niryo_moveit2_
91 config/cpp_script/ref_ee_traj_100ms.txt");
92
93 // Function
94 auto plan_and_execute = [&](const geometry_msgs::msg::PoseStamped&
95     arm_target_pose) -> bool {
96     // Find the robot's kinematic model
97     const moveit::core::JointModelGroup* joint_model_group =
98     arm_group_interface.getCurrentState()->getJointModelGroup("niryo_arm");
99     // Create a robotic state to calculate the IK
100     moveit::core::RobotState
101         target_state(*arm_group_interface.getCurrentState());
102     // Calculate the inverse kinematics to find the joint values
103     // corresponding to arm_target_pose
104     bool found_ik = target_state.setFromIK(
105         joint_model_group, arm_target_pose.pose, true);
106
107     if (found_ik) {
108         std::vector<double> joint_target;
109         target_state.copyJointGroupPositions(joint_model_group, joint_target);
110         RCLCPP_INFO(node->get_logger(), "IK found! Joint target:");
111         for (size_t i = 0; i < joint_target.size(); ++i) {
112             RCLCPP_INFO(node->get_logger(), "  Joint %zu: %f", i, joint_target[i]);
113         }
114         // Set the joint target obtained from the IK
115         arm_group_interface.setJointValueTarget(joint_target);
116     } else {
117         RCLCPP_ERROR(node->get_logger(), "IK not found for desired pose!");
118         RCLCPP_INFO(node->get_logger(), "Desired position: x=%f, y=%f, z=%f",
119             arm_target_pose.pose.position.x,
120             arm_target_pose.pose.position.y,
121             arm_target_pose.pose.position.z);
122         return false;
123     }
124
125     // Create a plan to that target pose
126     auto const [success, plan] = [&arm_group_interface] {
127         moveit::planning_interface::MoveGroupInterface::Plan msg;
128         auto const ok = static_cast<bool>(arm_group_interface.plan(msg));
129         return std::make_pair(ok, msg);
130     }();
131
132     // Try to execute the movement plan if it was created successfully
133     // If the plan wasn't successful, report an error
134     // Execute the plan
```

```

135     if (success)
136     {
137         // Construct the RobotTrajectory object from the message
138         robot_trajectory::RobotTrajectory robot_traj(
139             arm_group_interface.getRobotModel(),
140             "niryo_arm"
141         );
142         robot_traj.setRobotTrajectoryMsg(
143             *arm_group_interface.getCurrentState(),
144             plan.trajectory_
145         );
146
147         std::vector<std::vector<double>> sampled_joint_trajectory;
148         std::vector<std::vector<double>> sampled_ee_trajectory;
149
150         double dt_sampling = 0.1; // 100 ms
151         double traj_duration = robot_traj.getDuration();
152
153         for (double t = 0.0; t ≤ traj_duration; t += dt_sampling) {
154             // Create a shared_ptr to RobotState
155             moveit::core::RobotStatePtr state = std::make_shared<
156             moveit::core::RobotState>(robot_traj.getFirstWayPoint());
157             robot_traj.getStateAtDurationFromStart(t, state);
158
159             // Extract end-effector poses
160             const moveit::core::LinkModel* ee_link =
161                 state->getLinkModel("tool_link");
162             const Eigen::Isometry3d& ee_pose =
163                 state->getGlobalLinkTransform(ee_link);
164             const Eigen::Vector3d& position = ee_pose.translation();
165             const Eigen::Quaterniond orientation(ee_pose.rotation());
166
167             std::vector<double> ee_pose_vec = {
168                 position.x(), position.y(), position.z(),
169                 orientation.x(), orientation.y(), orientation.z(), orientation.w()
170             };
171             sampled_ee_trajectory.push_back(ee_pose_vec);
172         }
173
174         // Log pose EE
175         for (size_t i = 0; i < sampled_ee_trajectory.size(); ++i) {
176             std::ostringstream ee_line;
177             for (double val : sampled_ee_trajectory[i]) {
178                 ee_line << val << " ";
179             }
180             ee_file << ee_line.str() << "\n"; // Write to file
181         }
182         ee_file.close();
183
184         RCLCPP_INFO(logger, "Trajectory sampled in %lu points from CHOMP.",
185             sampled_joint_trajectory.size());
186
187         draw_trajectory_tool_path(plan.trajectory_);
188         moveit_visual_tools.trigger();
189         moveit_visual_tools.trigger();

```

```

190
191     // Execute the planned motion
192     arm_group_interface.execute(plan);
193     rclcpp::sleep_for(std::chrono::seconds(1));
194
195     }
196     else{
197         moveit_visual_tools.trigger();
198         RCLCPP_ERROR(logger, "Planning failed!");
199     }
200     return true;
201 };
202
203 // Run target
204 if (!plan_and_execute(arm_target_pose1)) {
205     RCLCPP_ERROR(logger, "Error in reaching the first position,
206         I am interrupting the programme.");
207     return 1;
208 }
209
210 spinner.join();
211 return 0;
212 }

```

Listing 3: Sample lines from `ref_ee_traj_100ms.txt` containing desired end-effector poses

```

1 0.2142 -0.19909 0.268737 -0.0771229 0.37248 0.182894 0.906565
2 0.23217 -0.177917 0.2687 -0.0946741 0.368471 0.225768 0.896825
3 0.249598 -0.152645 0.268661 -0.113831 0.363079 0.272554 0.883703
4 0.265536 -0.123034 0.268617 -0.13446 0.356031 0.322925 0.866534
5 0.278884 -0.0889988 0.268569 -0.156389 0.347043 0.376454 0.844622

```

Listing 4: NLP creation using CasADi and IPOPT

```

1 casadi::SXDict nlp;
2 nlp["x"] = OPT_VARS;
3 nlp["f"] = J;
4 nlp["g"] = G;
5 nlp["p"] = SX::vertcat({
6     x0,
7     reshape(Jp_sym, 18, 1),
8     reshape(Jo_sym, 18, 1),
9     S_ref_flat});
10 Function solver = nlpsol("nlp", "ipopt", nlp);

```

Listing 5: Terminal log showing the obstacle position used by the solver at each MPC cycle

```

1 MPC cycle t=0: Obstacle Position for NLP: [0.310, 0.178, 0.250]
2 MPC cycle t=1: Obstacle Position for NLP: [0.310, 0.173, 0.250]
3 MPC cycle t=2: Obstacle Position for NLP: [0.310, 0.168, 0.250]

```

Listing 6: Matlab script for observer gain calculation and stability verification

```

1  ts = 0.1; % Sampling period (s)
2  S = t/ts; % Scaling factor
3  alpha1 = a1 * S;
4  alpha2 = a2 * S^2;
5  alpha3 = a3 * S^3;
6  % Found alpha1 = 2, alpha2 = 1.6, alpha3 = 0.128.
7
8  % (2) Phi matrix that governs the observer's estimation error dynamics:
9  %     E(k+1) = Phi * E(k)
10 Phi = [1 - ts*alpha1, ts, 0 ;
11        - ts*alpha2, 1, ts;
12        - ts*alpha3, 0, 1];
13
14 % (3) Observer Stability Verification
15 eigenvalues = eig(Phi); % Calculate the eigenvalues
16 max_modulus = max(abs(eigenvalues)); % Calculate the spectral radius
17
18 fprintf('max_modulus: %f\n', max_modulus);
19 if max_modulus < 1
20     fprintf('The observer is stable (p(Phi) < 1).\n');
21 else
22     fprintf('WARNING: The observer is NOT stable (p(Phi) ≥ 1).\n');
23 end

```

Listing 7: Matlab script for computation of Φ_0 via LMI

```

1  alpha1 = 2; alpha2 = 1.6; alpha3 = 0.128;
2  Phi = [1 - ts*alpha1, ts, 0;
3         -ts*alpha2, 1, ts;
4         -ts*alpha3, 0, 1];
5
6  % Choose eta such that p(phi) < eta < 1
7  eta = 0.9999;
8
9  % Initialize CVX to find matrix W.
10 cvx_begin sdp
11
12     % Decision variable W (symmetric)
13     variable W(3, 3) symmetric
14
15     % Constraints
16     W ≥ 1e-6 * eye(3); % W must be positive definite with eigenvalues ≥ 1e-6
17     Phi' * W * Phi - (eta^2) * W ≤ 0; % LMI inequality
18
19 cvx_end
20
21 % Check and print results
22 if cvx_status == 'Solved'
23     disp('W found:');
24     disp(W);
25 else
26     disp('Error in LMI resolution. ');
27     disp(cvx_status);

```

```

28 end
29
30 % Verify if all eigenvalues of W are positive
31 eigenvalues_W = eig(W);
32 disp('Eigenvalues of W:');
33     disp(eigenvalues_W);
34     if all(eigenvalues_W > 0) % Check if they are strictly positive
35         fprintf('All eigenvalues of W are positive.\n');
36     else
37         fprintf('WARNING: Not all eigenvalues of W are positive!\n');
38     end
39
40 LMI_check = Phi' * W * Phi - (eta^2) * W;
41 % Verify if all eigenvalues of LMI_check are negative
42 eigenvalues_LMI_check = eig(LMI_check);
43 disp('Eigenvalues of (Phi^T * W * Phi - eta^2 * W):');
44     disp(eigenval_LMI_check);
45     % Check if all are ≤ 0 (with a small tolerance)
46     if all(eigenvalues_LMI_check ≤ 1e-9)
47         fprintf('LMI is satisfied (all eigenvalues ≤ 0).\n');
48     else
49         fprintf('WARNING: LMI might not be satisfied (some eigenvalues > 0).\n');
50     end
51
52 % Find c1 and c2 (minimum and maximum eigenvalues of W)
53 c1 = min(eigenvalues_W);
54 c2 = max(eigenvalues_W);
55 fprintf('c1 (lambda_min(W)) = %f\n', c1);
56 fprintf('c2 (lambda_max(W)) = %f\n', c2);
57
58 % Calculate Phi_0
59 Phi_0 = sqrt(c2 / c1);
60 fprintf('Phi_0 (sqrt(c2/c1)) calculated = %f\n', Phi_0);
61
62 % RESULT: c1 = 3.386523 , c2 = 19.977028 , Phi_0 = 2.428781

```

Listing 8: C++ node implementing the MPC algorithm for dynamic obstacle avoidance

```

1 #include <casadi/casadi.hpp>
2 #include <Eigen/Dense>
3 #include <iostream>
4 #include <moveit/move_group_interface/move_group_interface.h>
5 #include <rclcpp/rclcpp.hpp>
6 #include <thread>
7 #include <std_msgs/msg/float64_multi_array.hpp>
8 #include <geometry_msgs/msg/pose_stamped.hpp>
9 #include <mutex>
10 #include <fstream>
11 #include <sstream>
12 #include <vector>
13 #include <string>
14
15 using namespace casadi;
16

```

```

17 // Function to convert a CasADi DM vector to std::vector<double>
18 std::vector<double> DM_to_std_vector(const DM& dm) {
19     std::vector<double> result(dm.numel());
20     for (int i = 0; i < dm.numel(); ++i) {
21         result[i] = static_cast<double>(dm(i));
22     }
23     return result;
24 }
25
26 // Function for the conjugate of a CasADi SX quaternion
27 // q is assumed to be [qx, qy, qz, qw]
28 SX casadi_quat_conj(const SX& q) {
29     return SX::vertcat({-q(0), -q(1), -q(2), q(3)});
30 }
31
32 // Function for the multiplication of two CasADi SX quaternions
33 // Hamilton Product - q1, q2 are assumed to be [qx, qy, qz, qw]
34 SX casadi_quat_mult(const SX& q1, const SX& q2) {
35     SX x1 = q1(0), y1 = q1(1), z1 = q1(2), w1 = q1(3);
36     SX x2 = q2(0), y2 = q2(1), z2 = q2(2), w2 = q2(3);
37
38     SX x = w1*x2 + x1*w2 + y1*z2 - z1*y2;
39     SX y = w1*y2 - x1*z2 + y1*w2 + z1*x2;
40     SX z = w1*z2 + x1*y2 - y1*x2 + z1*w2;
41     SX w = w1*w2 - x1*x2 - y1*y2 - z1*z2;
42
43     return SX::vertcat({x, y, z, w});
44 }
45
46 // Update function for the obstacle state observer (GPIO)
47 void updateObstacleEstimate(
48     // measured obstacle position at time k
49     const Eigen::Vector3d& o_k_measured,
50     // observer timestep and gains
51     double ts_obs, double alpha1_obs, double alpha2_obs, double alpha3_obs,
52     Eigen::Vector3d& o_hat_curr, // current estimates (updated in-place)
53     Eigen::Vector3d& v_hat_curr,
54     Eigen::Vector3d& a_hat_curr,
55     bool& is_initialized) // flag for first initialization (updated in-place)
56 {
57     // Initialization block: executed only on the first valid call
58     if (!is_initialized) {
59         // The first position estimate is the current measurement
60         o_hat_curr = o_k_measured;
61         // Assume initial velocity and acceleration are zero
62         // due to lack of previous information
63         v_hat_curr.setZero();
64         a_hat_curr.setZero();
65         is_initialized = true;
66         return;
67     }
68
69     // For subsequent calls: The "_curr" variables passed by reference
70     // contain the previous cycle's estimates (e.g., o_hat_{k|k-1}).
71     Eigen::Vector3d o_hat_prev_local = o_hat_curr;

```

```

72 Eigen::Vector3d v_hat_prev_local = v_hat_curr;
73 Eigen::Vector3d a_hat_prev_local = a_hat_curr;
74 // Compute the position estimation error: difference between
75 // current measurement and previous estimate.
76 Eigen::Vector3d position_estimation_error = o_k_measured - o_hat_prev_local;
77
78 // Apply GPIO update equations to compute current estimates o_hat_{k|k}.
79 // The variables o_hat_curr, v_hat_curr, a_hat_curr
80 // are overwritten with new values.
81 o_hat_curr = o_hat_prev_local +
82     ts_obs * (v_hat_prev_local + alpha1_obs * position_estimation_error);
83 v_hat_curr = v_hat_prev_local +
84     ts_obs * (a_hat_prev_local + alpha2_obs * position_estimation_error);
85 a_hat_curr = a_hat_prev_local +
86     ts_obs * (alpha3_obs * position_estimation_error);
87 }
88
89 // Predict the obstacle trajectory over the horizon
90 std::vector<Eigen::Vector3d> predictObstacleTrajectory(
91     const Eigen::Vector3d& o_hat_now, // current estimated position
92     const Eigen::Vector3d& v_hat_now, // current estimated velocity
93     const Eigen::Vector3d& a_hat_now, // current estimated acceleration
94     int N_horizon, double ts_obs) // prediction horizon and observer timestep
95 {
96     std::vector<Eigen::Vector3d> predicted_o_trajectory;
97     predicted_o_trajectory.reserve(N_horizon + 1);
98
99     // Add the current estimate as the first element of the
100     // predicted trajectory (corresponds to o_hat_{k|k}).
101     predicted_o_trajectory.push_back(o_hat_now);
102
103     Eigen::Vector3d o_pred_j = o_hat_now;
104     Eigen::Vector3d v_pred_j = v_hat_now;
105     Eigen::Vector3d a_pred_j = a_hat_now;
106
107     // Loop for N prediction steps
108     for (int j = 0; j < N_horizon; ++j) {
109         // Obstacle prediction model (constant acceleration)
110         Eigen::Vector3d o_next = o_pred_j + ts_obs * v_pred_j;
111         Eigen::Vector3d v_next = v_pred_j + ts_obs * a_pred_j;
112         Eigen::Vector3d a_next = a_pred_j;
113
114         // Predicting o_{j+1} with respect to the start of the horizon
115         predicted_o_trajectory.push_back(o_next);
116
117         // Update state for the next prediction iteration
118         o_pred_j = o_next;
119         v_pred_j = v_next;
120         a_pred_j = a_next;
121     }
122     return predicted_o_trajectory;
123 }
124
125 int main(int argc, char *argv[]) {
126     // Initialize ROS 2

```

```
127     rclcpp::init(argc, argv);
128
129     // Create the ROS 2 node
130     auto const node = std::make_shared<rclcpp::Node>(
131         "mpc_controller",
132         rclcpp::NodeOptions().automatically_declare_
133         parameters_from_overrides(true)
134     );
135
136     // Publisher for joint velocities
137     rclcpp::Publisher<std_msgs::msg::Float64MultiArray>::SharedPtr velocity_pub;
138
139     // Logger
140     auto const logger = rclcpp::get_logger("mpc_controller");
141
142     // ROS 2 executor to process messages in the background
143     rclcpp::executors::SingleThreadedExecutor executor;
144     executor.add_node(node);
145     auto spinner = std::thread([&executor]() { executor.spin(); });
146
147     // Initialize the publisher for joint velocities
148     velocity_pub = node->create_publisher<std_msgs::msg::Float64MultiArray>
149     ("/niryo_velocity_controller/commands", 10);
150
151     // MoveGroup interface for the manipulator
152     using moveit::planning_interface::MoveGroupInterface;
153     auto arm_group_interface = MoveGroupInterface(node, "niryo_arm");
154
155     // Get the current end-effector pose
156     auto current_pose = arm_group_interface.getCurrentPose();
157     Eigen::Vector3d position(current_pose.pose.position.x,
158                             current_pose.pose.position.y,
159                             current_pose.pose.position.z);
160     Eigen::Quaterniond orientation(current_pose.pose.orientation.w,
161                                   current_pose.pose.orientation.x,
162                                   current_pose.pose.orientation.y,
163                                   current_pose.pose.orientation.z);
164
165     // Get current joint positions
166     std::vector<double> current_joint_values =
167         arm_group_interface.getCurrentJointValues();
168
169     // Build the full initial state
170     DM x_e_current = DM::vertcat({
171         position.x(), position.y(), position.z(),
172         orientation.x(), orientation.y(), orientation.z(), orientation.w()});
173
174     std::vector<casadi::DM> joints_dm(current_joint_values.size());
175     std::transform(current_joint_values.begin(), current_joint_values.end(),
176         joints_dm.begin(), [](double val) { return casadi::DM(val); });
177     DM theta_current = casadi::DM::vertcat(joints_dm);
178
179     DM x_current = casadi::DM::vertcat({x_e_current, theta_current});
180
181     std::vector<double> joint_lower = {
```

```

182     -2.9998, -1.8325, -1.3400, -2.0900, -1.9200, -2.5300};
183
184     std::vector<double> joint_upper = {
185         2.9998, 0.6101, 1.5700, 2.0900, 1.9228, 2.5300};
186
187     std::vector<double> joint_velocity_max = {
188         10.0, 8.0, 8.0, 2.0, 2.0, 2.0 };
189     // 1.0, 0.8, 0.8, 0.2, 0.2, 0.2 -- Scaled by a factor of 0.1
190
191     // For Jacobian calculation
192     const moveit::core::RobotModelConstPtr& robot_model =
193         arm_group_interface.getRobotModel();
194     const moveit::core::JointModelGroup* joint_model_group =
195         robot_model->getJointModelGroup("niryo_arm");
196
197     const int N = 5; // Horizon
198     const double dt = 0.1; // Timestep
199     const int max_timesteps = 100; // Maximum simulation duration
200
201     // Reference end-effector trajectory read from file
202     std::vector<std::vector<double>> reference_ee_traj;
203
204     std::ifstream infile("/home/francesco/lab_ROS2/ws_moveit2/src/
205     niryo_moveit2_config/cpp_script/ref_ee_traj_100ms.txt");
206     if (!infile.is_open()) {
207         RCLCPP_ERROR(node->get_logger(),
208             "Error opening file ref_ee_traj_100ms.txt");
209         return 1;
210     }
211
212     std::string line;
213     while (std::getline(infile, line)) {
214         std::istringstream iss(line);
215         std::vector<double> pose(7);
216         for (int i = 0; i < 7; ++i) {
217             iss >> pose[i];
218         }
219         reference_ee_traj.push_back(pose);
220     }
221     infile.close();
222
223     // Final padding: repeat the last pose to ensure at least
224     // max_timesteps + N - 1 elements
225     int required_length = max_timesteps + N - 1;
226     while (reference_ee_traj.size() < required_length) {
227         reference_ee_traj.push_back(reference_ee_traj.back());}
228
229     // Debug: print the entire loaded matrix
230     for (size_t i = 0; i < reference_ee_traj.size(); ++i) {
231         std::ostringstream oss;
232         oss << "EE_ref[" << i << "]: ";
233         for (double val : reference_ee_traj[i]) {
234             oss << val << " ";
235         }
236         RCLCPP_INFO(node->get_logger(), "%s", oss.str().c_str());

```

```

237     }
238
239     // (1) Variables for obstacle pose
240     // Stores the full PoseStamped message
241     geometry_msgs::msg::PoseStamped latest_obstacle_pose_msg;
242     // Protects access to latest_obstacle_pose_msg
243     std::mutex obstacle_pose_mutex;
244     bool new_obstacle_data_available = false; // Flag to signal new data
245
246     // (2) Subscriber and Callback
247     auto obstacle_pose_callback =
248         [&](const geometry_msgs::msg::PoseStamped::SharedPtr msg) {
249             std::lock_guard<std::mutex> lock(obstacle_pose_mutex);
250             latest_obstacle_pose_msg = *msg;
251             new_obstacle_data_available = true; // Signal that new data has arrived
252             RCLCPP_DEBUG(node->get_logger(),
253                 "Obstacle position updated: [%.3f, %.3f, %.3f]",
254                     msg->pose.position.x,
255                     msg->pose.position.y,
256                     msg->pose.position.z);
257         };
258
259     auto obstacle_pose_subscriber = ...
260         node->create_subscription<geometry_msgs::msg::PoseStamped>(
261             "/obstacle_pose", 10, obstacle_pose_callback);
262
263     // STATE VARIABLES FOR THE OBSTACLE OBSERVER
264     // These store the estimated obstacle state between MPC cycles.
265     Eigen::Vector3d observer_o_hat; // Current estimated position [ox, oy, oz]
266     Eigen::Vector3d observer_v_hat; // Current estimated velocity [vx, vy, vz]
267     Eigen::Vector3d observer_a_hat; // ... .. acceleration [ax, ay, az]
268     bool observer_is_initialized = false; // Flag for first initialization
269
270     // The observer works with the same timestep as the MPC controller
271     const double obs_ts = dt;
272
273     // Observer gains evaluated in Matlab
274     const double obs_alpha1 = 2.0;
275     const double obs_alpha2 = 1.6;
276     const double obs_alpha3 = 0.128;
277
278     // == Symbols ==
279     SX Jp_sym = SX::sym("Jp", 3, 6); // Jacobian for linear velocity
280     SX Jo_sym = SX::sym("Jo", 3, 6); // Jacobian for angular velocity
281     SX S_ref_flat = SX::sym("S_ref_flat", 7 * N); // Desired EE ref. (vector)
282     SX S_ref = reshape(S_ref_flat, 7, N); // Desired EE reference (matrix)
283     SX x0 = SX::sym("x0", 13); // Initial state (xee + theta)
284
285     // == Decision variables ==
286     SX U = SX::sym("U", 6, N); // Control commands
287     SX X = SX::sym("X", 13, N + 1); // Predicted states: x_0 ... x_N
288
289     // Decay rate gamma_k for the EE (Nx1)
290     SX gamma_ee_k_sym = SX::sym("gamma_ee_k", N);

```

```

291 // Symbolic parameter for the PREDICTED OBSTACLE TRAJECTORY (3(N+1) x 1)
292 SX O_pred_flat_sym = SX::sym("O_pred_flat", 3 * (N+1)); // (vector)
293 SX O_pred_matrix_sym = reshape(O_pred_flat_sym, 3, N+1); // (matrix)
294
295 // === Weight constants ===
296 SX Q = SX::diag(SX::vertcat({SX::ones(3)*2000, SX::ones(3)*50}));
297 SX R = SX::diag(SX::ones(6) * 2);
298
299 SX P_gamma_ee = SX::diag(SX::ones(1) * 1000);
300
301 // === Cost and constraints ===
302 SX J = SX::zeros();
303 std::vector<SX> g; // Constraints
304 std::vector<double> lbg_vec;
305 std::vector<double> ubg_vec;
306
307 // Known initial state
308 g.push_back(X(Slice(), 0) - x0); // X(Slice(), 0) - x0 = 0
309 for (int i = 0; i < 13; ++i) {
310     lbg_vec.push_back(0.0);
311     ubg_vec.push_back(0.0);
312 }
313
314 // PARAMETERS FOR CBF (m)
315 // (0.1 cm) Minimum distance to maintain between EE and obstacle surface.
316 double d_min_cbf = 0.001;
317 // (8.66 cm) Circumscribed sphere radius of the obstacle
318 double Ro_cbf = 0.0866;
319 double rd_cbf = 0.02; // (2 cm)
320 // Safety distance ( 0.001 + 0.0866 + 0.02 = 0.1076 m = 10.76 cm )
321 double r_safe_cbf = d_min_cbf + Ro_cbf + rd_cbf;
322
323 for (int i = 0; i < N; ++i) {
324     SX x_i = X(Slice(), i);
325     SX u_i = U(Slice(), i);
326     SX p_i = x_i(Slice(0, 3)); // Predicted position
327     SX q_i = x_i(Slice(3, 7)); // Predicted quaternion [qx,qy,qz,qw]
328     SX theta_i = x_i(Slice(7, 13));
329
330     // Decay rate for the EE at this step i
331     SX gamma_ee_i = gamma_ee_k_sym(i);
332
333     // Joint angle prediction
334     SX theta_next = theta_i + dt * u_i;
335
336     // EE position prediction
337     SX p_dot_i = mtimes(Jp_sym, u_i);
338     SX p_next = p_i + dt * p_dot_i;
339
340     // EE orientation prediction
341     // 1. Compute angular velocity omega_i (3x1)
342     SX omega_i = mtimes(Jo_sym, u_i); // Use Jo_sym
343
344     // 2. Form a pure quaternion from omega_i for multiplication.
345     // For convention [qx,qy,qz,qw], omega_pure is [omegaz,omegay,omegax,0]

```

```

346     SX omega_pure_i = SX::vertcat({omega_i(0), omega_i(1), omega_i(2), 0.0});
347
348     // 3. Compute the term (omega_pure_i (x) q_i) using casadi_quat_mult
349     //     (note: q_i is [qxi, qyi, qzi, qwil])
350     SX q_derivative_term = casadi_quat_mult(omega_pure_i, q_i);
351
352     // 4. Integrate quaternion:
353     // q_next = q_i + 0.5 * dt * ( omega_pure_i (x) q_i )
354     SX q_next = q_i + 0.5 * dt * q_derivative_term;
355
356     // 5. Symbolically normalize q_next
357     // Squared norm (dot=dot product)
358     SX sq_norm_q_next = dot(q_next, q_next);
359     SX q_next_normalized = q_next / sqrt(sq_norm_q_next + 1e-12);
360     // Small epsilon inside sqrt to avoid sqrt(0).
361
362     // Recompose EE state and full state
363     SX x_e_next_final = SX::vertcat({p_next, q_next_normalized});
364     SX x_next_final = SX::vertcat({x_e_next_final, theta_next});
365
366     // Dynamics constraint: x_{i+1} = x_next_final
367     g.push_back(X(Slice(), i + 1) - x_next_final);
368     for (int j = 0; j < 13; ++j) {
369         lbg_vec.push_back(0.0);
370         ubg_vec.push_back(0.0);
371     }
372
373     // Joint position and velocity constraints
374     for (int j = 0; j < 6; ++j) {
375         // Constraint: theta_next ≥ lower
376         g.push_back(theta_next(j) - joint_lower[j]);
377         lbg_vec.push_back(0.0); // ≥ 0
378         ubg_vec.push_back(casadi::inf);
379
380         // Constraint: theta_next ≤ upper
381         g.push_back(joint_upper[j] - theta_next(j));
382         lbg_vec.push_back(0.0); // ≥ 0
383         ubg_vec.push_back(casadi::inf);
384
385         // u_i(j) ≥ -max_velocity
386         g.push_back(u_i(j) + joint_velocity_max[j]);
387         lbg_vec.push_back(0.0); // ≥ 0
388         ubg_vec.push_back(casadi::inf);
389
390         // u_i(j) ≤ +max_velocity
391         g.push_back(joint_velocity_max[j] - u_i(j));
392         lbg_vec.push_back(0.0); // ≥ 0
393         ubg_vec.push_back(casadi::inf);
394     }
395
396     // Constraints for CBF (0 < gamma_ee_i ≤ 1)
397     double epsilon_gamma = 1e-6; // Small value for > 0
398
399     // gamma_ee_i ≥ epsilon_gamma (instead of >0)
400     g.push_back(gamma_ee_i - epsilon_gamma);

```

```

401     lbg_vec.push_back(0.0);
402     ubg_vec.push_back(casadi::inf);
403
404     g.push_back(1.0 - gamma_ee_i); // gamma_ee_i ≤ 1
405     lbg_vec.push_back(0.0);
406     ubg_vec.push_back(casadi::inf);
407
408     // --- ADDITION OF CBFSC SAFETY CONSTRAINT FOR THE EE ---
409     // Extract predicted obstacle positions
410     // Obstacle position o^{i|k}
411     SX o_obstacle_current_simb = O_pred_matrix_sym(Slice(), i);
412     // Obstacle position o^{i+1|k}
413     SX o_obstacle_next_simb    = O_pred_matrix_sym(Slice(), i + 1);
414
415     // Compute H_current = H(x_e,i|k, o_hat_i|k)
416     SX H_ee_current = norm_2(p_i - o_obstacle_current_simb) - r_safe_cbf;
417
418     // Compute H_next = H(x_e,i+1|k, o_hat_i+1|k)
419     SX H_ee_next = norm_2(p_next - o_obstacle_next_simb) - r_safe_cbf;
420
421     // Full CBFSC constraint formula:
422     // H_next - (1 - gamma_ee_i) * H_current ≥ 0
423     g.push_back(H_ee_next - (1 - gamma_ee_i) * H_ee_current);
424     lbg_vec.push_back(0.0); // Lower bound of constraint (≥ 0)
425     ubg_vec.push_back(casadi::inf); // Upper bound of constraint (infinite)
426
427     // === Error calculation ===
428     SX p_ref_i = S_ref(Slice(0,3), i);
429     SX q_ref_i = S_ref(Slice(3,7), i);
430
431     // 1. Handle q vs -q ambiguity
432     SX q_pred_aligned = if_else(dot(q_ref_i, q_i) < 0, -q_i, q_i);
433
434     // 2. Compute q_err = q_ref * q_pred_aligned_conjugate
435     SX q_pred_conj = casadi_quat_conj(q_pred_aligned);
436     SX q_err = casadi_quat_mult(q_ref_i, q_pred_conj);
437
438     // 3. Extract 3D error
439     SX err_orient_vec = q_err(Slice(0,3)) * 2; // Vector part of q_err * 2
440
441     // 4. Position error
442     SX err_pos_vec = p_ref_i - p_i;
443
444     // 5. Total 6D error vector
445     SX e_total_i = SX::vertcat({err_pos_vec, err_orient_vec});
446     J += mtimes(mtimes(e_total_i.T(), Q), e_total_i) +
447         mtimes(mtimes(u_i.T(), R), u_i) +
448         P_gamma_ee * pow(gamma_ee_i, 2);
449 }
450
451 // === Concatenate decision variables ===
452 SX OPT_VARS = SX::vertcat({
453     reshape(U, 6 * N, 1),
454     reshape(X, 13 * (N + 1), 1),
455     reshape(gamma_ee_k_sym, N, 1)});

```

```

456
457 // === Concatenate constraints ===
458 SX G = SX::vertcat(g);
459
460 // === NLP ===
461 casadi::SXDict nlp;
462 nlp["x"] = OPT_VARS;
463 nlp["f"] = J;
464 nlp["g"] = G;
465 nlp["p"] = SX::vertcat({
466     x0, // 13x1 Parameters: initial state,
467     reshape(Jp_sym, 18, 1), // 3x6 Jacobian position,
468     reshape(Jo_sym, 18, 1), // 3x6 Jacobian orientation,
469     S_ref_flat, // 7N x 1 target,
470     O_pred_flat_sym}); // 3(N+1) x 1 predicted obstacle trajectory.
471
472 Function solver = nlpsol("nlp", "ipopt", nlp);
473
474 RCLCPP_INFO(logger, "NLP ready to be solved with IPOPT");
475
476 // === Loop MPC ===
477 int num_U_vars = 6 * N;
478 int num_X_vars = 13 * (N + 1);
479 int num_gamma_ee_vars = N;
480 int total_decision_vars = num_U_vars + num_X_vars + num_gamma_ee_vars;
481
482 // Variable to save the solution from the previous cycle
483 // Initial initialization
484 DM previous_solution = DM::zeros(total_decision_vars);
485
486 // Initialize with initial states and zero velocities
487 for (int i = 0; i < N + 1; ++i) {
488     for (int j = 0; j < 13; ++j) {
489         previous_solution(num_U_vars + i * 13 + j) = x_current(j);
490         // Use x_current for states
491     }
492 }
493
494 // Initialize gamma
495 for (int i = 0; i < N; ++i) {
496     // The gamma part starts after U and X
497     previous_solution(num_U_vars + num_X_vars + i) = 0.001;
498 }
499
500 // (1) Initialize monotonic clock
501 rclcpp::Clock steady_clock(RCL_STEADY_TIME);
502 rclcpp::Time iteration_start = steady_clock.now();
503
504 bool first_run = true; // Flag for first execution
505 casadi::DM last_u = DM::zeros(6, 1); // u_{k-1}, initialized to zero
506
507 for (int t = 0; t < max_timesteps; ++t) {
508
509     if (first_run) {
510         first_run = false; // Use the initial state already defined

```

```

511     } else {
512
513         // Build msg u_{n-1}
514         std_msgs::msg::Float64MultiArray vel_msg;
515         vel_msg.data.resize(6);
516         for (int i = 0; i < 6; ++i) {
517             vel_msg.data[i] = static_cast<double>(last_u(i));
518         }
519
520         // Check if 100ms have passed or wait
521         if (t ≥ 2) {
522             auto iteration_end = steady_clock.now();
523             auto duration = iteration_end - iteration_start;
524             auto elapsed_ms = duration.nanoseconds() / 1'000'000;
525             if (elapsed_ms < 100) {
526                 std::this_thread::sleep_for(std::chrono::milliseconds
527                     (100 - elapsed_ms));
528             } else {
529                 RCLCPP_WARN(logger, "Cycle %d, Cycle time greater than
530                     100ms, precisely: %ld ms", t, elapsed_ms);
531             }
532         }
533
534         // Apply u_{n-1}
535         velocity_pub->publish(vel_msg);
536         iteration_start = steady_clock.now();
537
538         // Log control action
539         RCLCPP_INFO(logger, "Control Action:");
540         for (size_t i = 0; i < vel_msg.data.size(); ++i) {
541             RCLCPP_INFO(logger, "  u[%zu]: %f", i, vel_msg.data[i]);
542         }
543
544         // Update x_current
545         auto pose_start = steady_clock.now(); // DEBUG 1
546         auto current_pose = arm_group_interface.getCurrentPose();
547         auto pose_end = steady_clock.now(); // DEBUG 1
548         RCLCPP_INFO(logger, "Time getCurrentPose(): %ld ms",
549             (pose_end - pose_start).nanoseconds() / 1'000'000); // DEBUG 1
550         Eigen::Vector3d position(current_pose.pose.position.x,
551             current_pose.pose.position.y,
552             current_pose.pose.position.z);
553         Eigen::Quaterniond orientation(current_pose.pose.orientation.w,
554             current_pose.pose.orientation.x,
555             current_pose.pose.orientation.y,
556             current_pose.pose.orientation.z);
557
558         auto joint_start = steady_clock.now(); // DEBUG 2
559         std::vector<double> current_joint_values =
560             arm_group_interface.getCurrentJointValues();
561         auto joint_end = steady_clock.now(); // DEBUG 2
562         RCLCPP_INFO(logger, "Time getCurrentJointValues(): %ld ms",
563             (joint_end - joint_start).nanoseconds() / 1'000'000);
564         // DEBUG 2
565

```

```

566         x_e_current = DM::vertcat({
567             position.x(), position.y(), position.z(), orientation.x(),
568             orientation.y(), orientation.z(), orientation.w()
569         });
570
571         joints_dm.clear(); // Clear the existing vector
572         for (double val : current_joint_values) {
573             joints_dm.push_back(casadi::DM(val)); // Populate with new values
574         }
575         theta_current = casadi::DM::vertcat(joints_dm);
576         x_current = casadi::DM::vertcat({x_e_current, theta_current});
577     }
578
579     auto state_start = steady_clock.now(); // DEBUG 3
580     // === Dynamically compute the geometric Jacobian ===
581     moveit::core::RobotStatePtr current_state =
582         arm_group_interface.getCurrentState();
583
584     std::vector<double> current_joint_values =
585         DM_to_std_vector(theta_current);
586     // Update the robot state
587     current_state->setJointGroupPositions(
588         joint_model_group, current_joint_values);
589     current_state->update();
590
591     // Compute the geometric Jacobian (6x6)
592     Eigen::MatrixXd J_ee(6, joint_model_group->getVariableCount());
593     current_state->getJacobian(
594         joint_model_group,
595         current_state->getLinkModel("tool_link"),
596         Eigen::Vector3d::Zero(),
597         J_ee,
598         false //6x6
599     );
600     auto state_end = steady_clock.now(); // DEBUG 3
601     RCLCPP_INFO(logger, "Time update state + Jacobian: %ld ms",
602         (state_end - state_start).nanoseconds() / 1'000'000); // DEBUG 3
603
604     // Extract Jp (first 3 rows) and Jo (last 3 rows) from J_ee
605     Eigen::MatrixXd Jp_ee = J_ee.topRows(3); // 3x6
606     Eigen::MatrixXd Jo_ee = J_ee.bottomRows(3); // 3x6
607
608     // Convert Jp_ee to casadi::DM Jp_live_ee
609     DM Jp_live_ee = DM::zeros(3, joint_model_group->getVariableCount());
610     for (int r = 0; r < 3; ++r) {
611         for (int c = 0; c < joint_model_group->getVariableCount(); ++c) {
612             Jp_live_ee(r, c) = Jp_ee(r, c);
613         }
614     }
615
616     // Convert Jo_ee to casadi::DM Jo_live_ee
617     DM Jo_live_ee = DM::zeros(3, joint_model_group->getVariableCount());
618     for (int r = 0; r < 3; ++r) {
619         for (int c = 0; c < joint_model_group->getVariableCount(); ++c) {
620             Jo_live_ee(r, c) = Jo_ee(r, c);

```

```

621     }
622 }
623
624 // Dynamically select only N poses starting from t
625 std::vector<double> flattened_ref;
626 for (int i = t; i < t + N; ++i) {
627     const auto& pose = reference_ee_traj[i];
628     flattened_ref.insert(flattened_ref.end(), pose.begin(), pose.end());
629 }
630 DM reference_ee_traj_dm = DM(flattened_ref); // column vector (7xN)x1
631
632 // --- OBSERVER AND OBSTACLE PREDICTION SECTION ---
633 Eigen::Vector3d o_k_measured_for_observer;
634 bool use_this_measurement = false;
635
636 { // Block to limit the scope of lock_guard
637     std::lock_guard<std::mutex> lock(obstacle_pose_mutex);
638     // Use the mutex defined in main
639     if (new_obstacle_data_available) {
640         // Flag defined in main and updated by the callback
641         o_k_measured_for_observer <<
642             latest_obstacle_pose_msg.pose.position.x,
643             latest_obstacle_pose_msg.pose.position.y,
644             latest_obstacle_pose_msg.pose.position.z;
645         new_obstacle_data_available = false;
646         use_this_measurement = true;
647     } else if (observer_is_initialized) {
648         // Use the observer state variables defined in main
649         o_k_measured_for_observer = observer_o_hat;
650         // Fallback: use the last estimate as "measurement"
651         RCLCPP_DEBUG(logger, "MPC cycle t=%d: No new obstacle pose,
652             using last estimate for observer input.", t);
653         use_this_measurement = true;
654         // Still have an input for the observer
655     } else {
656         RCLCPP_WARN(logger, "MPC cycle t=%d: Observer not initialized
657             and no obstacle data! Setting far.", t);
658         o_k_measured_for_observer =
659             Eigen::Vector3d(100.0, 100.0, 100.0); // Emergency fallback
660         use_this_measurement = true;
661         // Still need to initialize the observer
662     }
663 }
664
665 // Update observer estimates if we have a valid input
666 if (use_this_measurement || !observer_is_initialized) {
667     // Always call if not initialized for the first set
668     updateObstacleEstimate(o_k_measured_for_observer,
669         obs_ts, obs_alpha1, obs_alpha2, obs_alpha3,
670         observer_o_hat, observer_v_hat, observer_a_hat,
671         observer_is_initialized);
672 }
673 // Now observer_o_hat, observer_v_hat, observer_a_hat
674 // contain the latest estimates
675

```

```

676 // Predict the obstacle trajectory for the horizon N
677 std::vector<Eigen::Vector3d> predicted_o_horizon_eigen =
678     predictObstacleTrajectory(
679         observer_o_hat, observer_v_hat, observer_a_hat, N, obs_ts);
680
681 // Convert the predicted obstacle trajectory to a flattened casadi::DM
682 std::vector<double> flattened_o_pred_vec;
683 flattened_o_pred_vec.reserve(3 * (N+1));
684 if (observer_is_initialized && !predicted_o_horizon_eigen.empty()) {
685     // Make sure there is something to convert
686     for (const auto& pos_o : predicted_o_horizon_eigen) {
687         flattened_o_pred_vec.push_back(pos_o.x());
688         flattened_o_pred_vec.push_back(pos_o.y());
689         flattened_o_pred_vec.push_back(pos_o.z());
690     }
691     // If the observer is not initialized or the prediction is empty,
692     // send "far" positions.
693 } else {
694     RCLCPP_WARN(logger, "MPC cycle t=%d: Observer not ready or empty
695         prediction, sending far obstacle positions.", t);
696     for(int i=0; i<N+1; ++i) {
697         flattened_o_pred_vec.push_back(100.0); // ox_far
698         flattened_o_pred_vec.push_back(100.0); // oy_far
699         flattened_o_pred_vec.push_back(100.0); // oz_far
700     }
701 }
702 casadi::DM O_pred_flat_dm(flattened_o_pred_vec);
703
704 // Log to verify the observer and prediction
705 RCLCPP_INFO(logger, "Cycle t=%d: o_measured_used_by_obs
706     [%.3f, %.3f, %.3f]", t, o_k_measured_for_observer.x(),
707     o_k_measured_for_observer.y(), o_k_measured_for_observer.z());
708 RCLCPP_INFO(logger, "o_estimated_current [%.3f, %.3f, %.3f]",
709     observer_o_hat.x(), observer_o_hat.y(), observer_o_hat.z());
710 RCLCPP_INFO(logger, "v_estimated_current [%.3f, %.3f, %.3f]",
711     observer_v_hat.x(), observer_v_hat.y(), observer_v_hat.z());
712 RCLCPP_INFO(logger, "a_estimated_current [%.3f, %.3f, %.3f]",
713     observer_a_hat.x(), observer_a_hat.y(), observer_a_hat.z());
714 if (!predicted_o_horizon_eigen.empty()) {
715     RCLCPP_INFO(logger,
716         "o_pred_horizon[1] (for t+dt) [%.3f, %.3f, %.3f]",
717         predicted_o_horizon_eigen[1].x(),
718         predicted_o_horizon_eigen[1].y(),
719         predicted_o_horizon_eigen[1].z());
720 }
721
722 // 1. Reference position (x_ref, y_ref, z_ref)
723 // for the FIRST step of the horizon
724 std::vector<double> p_ref_first_step_vec = {
725     reference_ee_traj[t][0],
726     reference_ee_traj[t][1],
727     reference_ee_traj[t][2]};
728
729 RCLCPP_INFO(logger, "p_ref (current, t=%d): [%.4f, %.4f, %.4f]", t,
730     p_ref_first_step_vec[0],

```

```

731         p_ref_first_step_vec[1],
732         p_ref_first_step_vec[2]);
733
734     // 2. Current robot position (predicted for the FIRST step)
735     std::vector<double> p_pred_current_vec(3);
736     for(int j=0; j<3; ++j) {
737         p_pred_current_vec[j] = static_cast<double>(x_e_current(j));
738     }
739     RCLCPP_INFO(logger, "p_pred (current robot): [%.4f, %.4f, %.4f]",
740         p_pred_current_vec[0], p_pred_current_vec[1], p_pred_current_vec[2]);
741
742     // 3. Reference quaternion for the FIRST step of the horizon (i=0)
743     std::vector<double> q_ref_first_step_vec = {
744         reference_ee_traj[t][3], reference_ee_traj[t][4],
745         reference_ee_traj[t][5], reference_ee_traj[t][6]
746     };
747     RCLCPP_INFO(logger, "q_ref (current, t=%d):
748         [%.4f, %.4f, %.4f, %.4f]", t,
749         q_ref_first_step_vec[0], q_ref_first_step_vec[1],
750         q_ref_first_step_vec[2], q_ref_first_step_vec[3]);
751
752     // 4. Current robot quaternion (predicted for the FIRST step, i=0)
753     std::vector<double> q_pred_current_vec(4);
754     for(int j=0; j<4; ++j) {
755         q_pred_current_vec[j] = static_cast<double>(x_e_current(3+j));
756     }
757     RCLCPP_INFO(logger, "q_pred (current robot): [%.4f, %.4f, %.4f, %.4f]",
758         q_pred_current_vec[0], q_pred_current_vec[1],
759         q_pred_current_vec[2], q_pred_current_vec[3]);
760
761     std::map<std::string, DM> arg;
762     // Use the previous cycle's solution as a starting point
763     arg["x0"] = previous_solution;
764     arg["p"] = DM::vertcat({
765         x_current, // 13x1
766         reshape(Jp_live_ee, 18, 1), // 18x1 for 6 DoF
767         reshape(Jo_live_ee, 18, 1), // 18x1 for 6 DoF
768         reference_ee_traj_dm, // 7N x 1
769         O_pred_flat_dm}); // 3(N+1) x 1 (predicted obstacle traj.)
770     arg["lbq"] = DM(lbg_vec);
771     arg["ubq"] = DM(ubg_vec);
772
773     try {
774         auto solver_start = steady_clock.now(); // DEBUG
775         auto result = solver(arg);
776         auto solver_end = steady_clock.now(); // DEBUG
777         auto solver_duration_ms =
778             (solver_end - solver_start).nanoseconds() / 1'000'000; // DEBUG
779
780         // log solver time
781         RCLCPP_INFO(logger, "Solver time: %ld ms", solver_duration_ms);
782
783         DM sol = result.at("x");
784         // Save the current solution for the next cycle - warm-starting
785         previous_solution = sol;

```

```
786         DM u0 = reshape(sol(Slice(0, 6)), 6, 1); // u_k
787         last_u = u0; // save for the next cycle
788
789         // Print the first gamma value of the horizon (gamma_{0|k})
790         RCLCPP_INFO(logger, " gamma_ee_optimal[0] (for t+dt): %.4f",
791             static_cast<double>(sol(num_U_vars + num_X_vars)));
792
793         // Stop logic
794         if (t == max_timesteps - 1) {
795             // Zero velocities
796             std_msgs::msg::Float64MultiArray zero_vel_msg;
797             zero_vel_msg.data = {0.0, 0.0, 0.0, 0.0, 0.0, 0.0};
798             velocity_pub->publish(zero_vel_msg);
799             RCLCPP_INFO(logger, "Zero velocities published. Robot stopped.");
800             RCLCPP_WARN(logger, "Maximum number of timesteps reached
801                 (%d). Stopping MPC.", max_timesteps);
802         }
803     } catch (std::exception &e) {
804         RCLCPP_ERROR(logger, "Optimization failed at timestep
805             %d: %s", t, e.what());
806         RCLCPP_ERROR(logger, "Solver parameters at failure:");
807         for (int i = 0; i < arg["p"].numel(); ++i) {
808             RCLCPP_ERROR(logger, " p[%d] = %.3f", i,
809                 static_cast<double>(arg["p"](i)));
810         }
811         // Zero velocities
812         std_msgs::msg::Float64MultiArray zero_vel_msg;
813         zero_vel_msg.data = {0.0, 0.0, 0.0, 0.0, 0.0, 0.0};
814         velocity_pub->publish(zero_vel_msg);
815         RCLCPP_INFO(logger, "Zero velocities published. Robot stopped.");
816         break;
817     }
818 }
819
820 spinner.join();
821 return 0;
822 }
```